Python Lecture Note

1. Python과 외부 실행 환경, 파일시스템, CLI

Dong Hyun KIM

1 Python과 외부 실행 환경, 파일시스템, CLI

1.1 Python 실행 환경과 외부 시스템 개요

1.1.1 sys는 파일 경로용 패키지가 아니다

```
sys ≠ 시스템의 파일이나 디렉터리를 다루는 패키지

sys = 현재 실행 중인 Python interpreter와
      그 실행 환경에 접근하는 모듈
```

```python
import sys

print(sys.version) # 현재 어떤 Python인가?
print(sys.executable) # 현재 어떤 Python 실행 파일을 쓰는가?
print(sys.argv) # 이 프로그램을 실행할 때 무엇을 전달받았는가?
print(sys.path) # import할 모듈을 어디서 찾는가?
```

위 4개의 print문의 인수는 전부 파일 조작 기능이 아니다.

```python
from pathlib import Path # 파일 경로 다루는 패키지
import os # 운영체제 기능을 다루는 패키지
```

그러므로 아래와 같이 생각하자.

```
sys
├── Python interpreter
├── 실행 인수
├── import 검색 경로
├── stdin / stdout / stderr
└── 프로그램 종료 등


os
├── 운영 체제
├── 환경 변수
├── 프로세스 관련 기능
└── 일부 파일 시스템 기능


pathlib
└── 파일/디렉터리 경로
```

1.1.2 그런데 왜 sys를 배우다 보면 경로가 나오는가?

대표적인 원인은 `sys.path`이다.

> `sys.path` ≠ 내 컴퓨터의 파일 경로를 탐색하는 기능
>
> **`sys.path` = Python이 `import` 할 모듈을 찾는 위치들의 목록**

예를 들어:

```
import numpy
```

라는 코드를 실행했다고 하면 Python 입장에서는 문제가 발생한다.

`numpy`가 어디 있는데?

컴퓨터에는 수십만 개의 파일이 있을 수 있다.

Python이 C 드라이브나 SSD 전부를 뒤지는 것이 아니다.

Python은 정해진 검색 위치를 가지고 있고 그것을 확인하는 게:

```
import sys

print(sys.path)
```

위 코드의 결과는 대략 아래와 같다.

```
[
'/Users/user/project',
'/Users/user/anaconda3/envs/transformer/lib/python310.zip',
'/Users/user/anaconda3/envs/transformer/lib/python3.10',
...
'/Users/user/anaconda3/envs/transformer/lib/python3.10/site-packages'
]
```

여기에서 `site-packages`는 `pip install transformers`같은 걸 하면 설치된 Python 패키지가 들어가는 대표적인 위치다.

그렇기에:

```
import transformers
```

가 가능한 것이다.

1.1.3 이것을 conda 가상환경과 연결해보자

윈도우에서 Conda 환경이:

```
C:\Users\user\anaconda3\envs\transformer\
```

에 있을 때 터미널에서 conda 가상환경을 활성화하고 파이썬에서:

```
import sys

print(sys.executable)
```

을 하면 결과는

```
C:\Users\user\anaconda3\envs\transformer\python.exe
```

가 나온다.

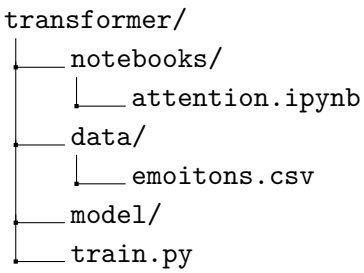
즉,

Conda environment
↓
transformer
↓
그 환경의 Python interpreter
↓
C:\Users\user\anaconda3\envs\transformer\python.exe

가 연결된다.

1.1.4 파일 경로는 또 다른 문제다

예를 들어 프로젝트가:



라고 할 때, train.py에서 데이터를 읽고 싶다면:

```
import pandas as pd

df = pd.read_csv(
    'data/emotions.csv'
)
```

여기서 'data/emotions.csv'는 sys.path와 직접적인 관계가 없다.

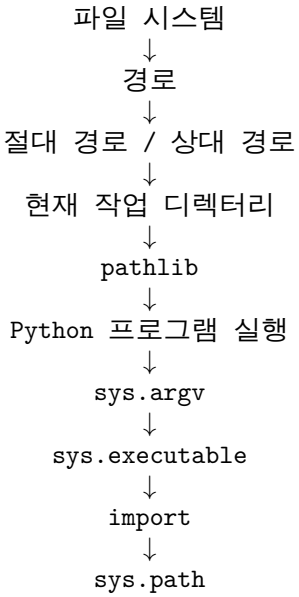
이 것은 **파일 시스템 경로**다.

그래서 중요한 개념이 등장한다.

> 현재 작업 디렉터리
>
> Current Working Directory
>
> CWD

1.1.5 그래서 우리가 제일 먼저 깊게 배워야 하는 것은 sys가 아니다

추천 순서는:

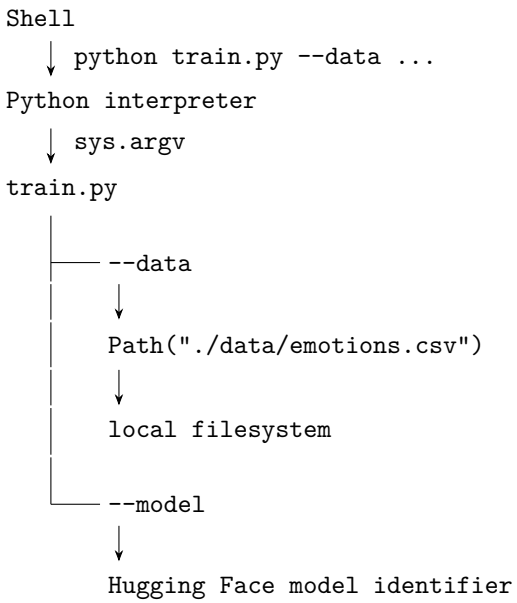


1.1.6 최종적으로는 이런 코드를 이해하는 것이 목표다

예를 들어 몇 단계 뒤에는 터미널에서:

```
python train.py \
    --data ./data/emotions.csv \
    --model distilbert-base-uncased \
    --epochs 5 \
    --output ./models/emotion-model
```

를 실행할 때:



그리고 내부에서는:

```
sys.executable # 어떤 Python이 실행 중인지 알 수 있음
sys.path # Python이 import할 package를 어디서 찾는지 확인 할 수 있음
os.environ # 환경 변수에 접근할 수도 있음
```

이 정도가 되면 CLI, Conda, VS Code, Jupyter, Python import, 파일 경로, URL, ML 학습 스크립트가 전부 하나의 시스템으로 보이기 시작한다.

1.2 파일 시스템

1.2.1 파일 시스템이란 무엇인가?

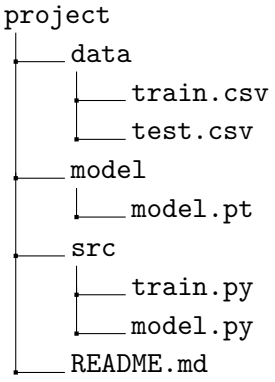
컴퓨터 저장장치에 아래와 같은 파일들이 있다고 가정해보자.

```
사진.jpg
train.py
data.csv
requirements.txt
model.pt
```

파일이 수십만 개가 되면 한곳에 다 넣을 수 없다.

그래서 디렉터리(directory)를 이용해서 계층적으로 정리한다.

예를 들어:



이것이 우리가 앞으로 계속 보게 될 **파일 시스템의 트리 구조**다.

여기서:

`project` → 디렉터리

`data` → 디렉터리

`train.csv` → 파일

`src` → 디렉터리

`train.py` → 파일

이며, 디렉터리 안에 다른 디렉터리가 들어갈 수 있기 때문에 트리 구조가 만들어진다.

1.2.2 경로(Path)는 트리를 따라가는 방법이다

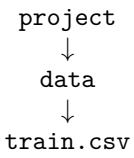
위 구조에서 `train.csv`가 어디 있는지를 설명하고 싶다고 해보자.

그러면 위치를 이렇게 표현할 수 있다.

```
project/data/train.csv
```

이것이 `path(경로)`다.

즉,



라는 파일 시스템의 이동 경로를 문자열 형태로 표현한 것이다.

그래서 처음에는 이렇게 생각해도 좋다.

```
Path = 파일 시스템 트리에서 어떤 위치까지 찾아가는 주소
```

단, 메모리의 주소값(address)과는 다른 개념임을 주의하자.

1.2.3 그런데 어디서 출발하느냐가 문제가 된다

서울에서 누군가에게

"우리집은 오른쪽으로 가서 두 번째 골목이에요."

라고 하면 이상하다.

반면,

"서울특별시 → ○○구 → ○○동 → ..."

처럼 처음부터 위치를 지정하면 출발 위치와 관계없이 찾아갈 수 있다.

파일 시스템도 똑같아서 경로에는 크게 두 종류가 있다.

```
절대 경로 (absolute path)

상대 경로 (relative path)
```

1.2.4 절대 경로 Absolute Path

절대 경로는 **파일 시스템의 출발점부터 목적지까지 전부 적은 경로**다.

Mac/Linux라면 예를 들어:

```
/Users/user/Desktop/python/project/data/train.csv
```

Windows 라면:

```
C:\Users\user\Desktop\python\project\data\train.csv
```

이런 식이다.

Mac의 것을 구조화하면:

```
/
└── Users
    └── user
        └── Desktop
            └── python
                └── project
                    └── data
                        └── train.csv
```

따라서:

```
/Users/user/Desktop/python/project/data/train.csv
```

의 뜻은 다음과 같다.

```
파일 시스템의 맨 위에서부터 train.csv까지 찾아가라.
```

1.2.5 맨 앞의 /는 특별하다

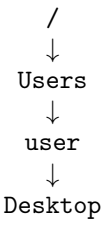
Mac/Linux에서 / 는 단순한 구분자가 아니다.

파일 시스템의 최상위 디렉터리인 **root directory**를 의미한다.

그래서:

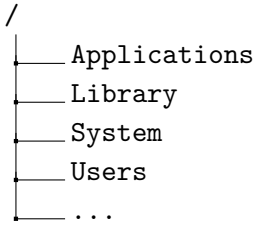
```
/Users/user/Desktop
```

를 해석하면:



이다.

여기서 root를 흔히 이렇게 그린다.



Mac을 사용하다보면 `/Users/...` 같은 경로를 자주 보게 되는 이유다.

1.2.6 Windows는 조금 다르다

Windows는 흔히 드라이브가 먼저 등장한다:

```
C:\Users\user\Desktop
```

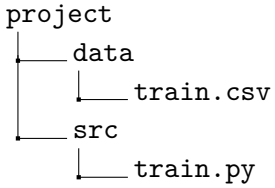
1.2.7 상대 경로 Relative Path

현재 내가 있는 위치가:

`project/`

라고 하자.

구조는:



현재 위치가 `project`라면 `train.csv`를 굳이:

`/Users/user/Desktop/python/project/data/train.csv`

라고 전부 적을 필요가 없이:

`data/train.csv`

라고 적으면 된다.

왜냐하면 출발점이 이미 `project`이기 때문이다.

이것이 **상대 경로**다.

1.2.8 그러면 "현재 위치"가 대체 무엇인가?

여기가 Python에서 굉장히 중요하다.

이것을:

```
Current Working Directory, CWD
```

라고 한다.

한국어로는 **현재 작업 디렉터리**정도로 번역이 된다.

예를 들어 터미널이:

```
~/Desktop/python/project %
```

라고 되어 있다면 현재 작업 디렉터리가 대략:

```
/Users/user/Desktop/python/project
```

라고 생각할 수 있다.

Python에서도 확인할 수 있다.

```
from pathlib import Path

print(Path.cwd())
```

예를 들어:

```
/Users/user/Desktop/python/project
```

가 출력된다.

1.2.9 상대 경로는 CWD를 기준으로 해석한다

이 원칙이 정말 중요하다.

```
from pathlib import Path

p = Path("data/train.csv")
```

여기서:

```
data/train.csv
```

은 상대 경로다.

만약:

```
Path.cwd()
```

가:

```
/users/user/Desktop/python/project
```

라면 실질적으로 가리키는 곳은:

```
/users/user/Desktop/python/project
                +
          data/train.csv
```

즉:

```
/users/user/Desktop/python/project/data/train.csv
```

이다.

```
/Users/user/Desktop/python/project (CWD)
                                  │
                                  └── data
                                        │
                                        └── train.csv
```

이 원리를 잡으면 나중에 Pandas에서 발생하는 수많은 파일 경로 오류를 이해할 수 있다.

1.2.10 . 은 "현재 디렉터리"다

경로에서 이것도 많이 본다.

.

점 하나는:

```
현재 디렉터리
```

를 의미한다.

따라서:

```
./data/train.csv
```

는:

현재 디렉터리
↓
data
↓
train.csv

라는 뜻이다.

그래서:

`data/train.csv`

와

`./data/train.csv`

는 이 상황에서는 사실상 같은 위치를 나타낸다.

1.2.11 .. 는 부모 디렉터리다

점 두 개:

`..`

는:

현재 디렉터리의 한 단계 위

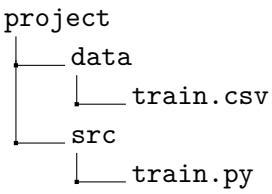
를 의미한다.

예를 들어 위치가:

`project/src`

라고 하자.

구조는:



현재 위치:

`project/src`

에서:

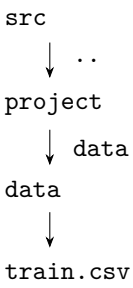
`..`

하면 `project`로 올라간다.

그러므로:

`../data/train.csv`

는:



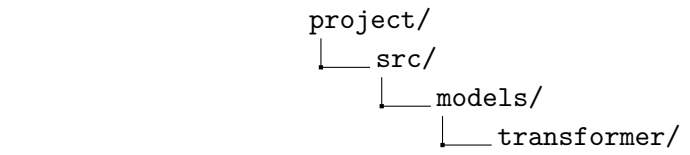
는 뜻이다.

즉,

$$\texttt{src} \rightarrow \text{한 단계 위} \rightarrow \texttt{data} \rightarrow \texttt{train.csv}$$

1.2.12 여러 번 올라갈 수도 있다

예를 들어:



현재 위치가:

```
project/src/models/transformer
```

라고 하자.

그러면:

```
..
```

는:

```
project/src/models
```

이고,

```
../..
```

는:

```
project/src
```

이고,

```
../../..
```

는:

```
project
```

이다.

따라서:

```
../../../data/train.csv
```

같은 것도 가능하다.

하지만 실제 프로젝트에서 이런 경로가 난무하기 시작하면 구조를 다시 생각해볼 필요가 있다.

1.2.13 ~는 또 다른 의미다

Mac/Linux 터미널에서 자주 보는:

~

는 보통 사용자의 **home directory**를 의미한다.

Mac 계정 이름을 예시로 dhk 라고 하면:

~

가 대략:

```
/Users/dhk
```

를 의미한다.

그래서:

```
~/Desktop
```

은:

```
/Users/dhk/Desktop
```

같은 의미가 된다.

Windows에도 사용자 홈 개념이 있고 Python의 pathlib으로 운영체제에 맞게 처리할 수 있다.

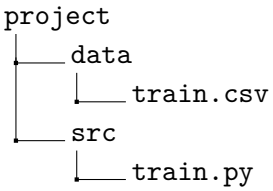
예를 들어:

```
from pathlib import Path
print(Path.home())
```

이라고 하면 사용자 홈 디렉터리를 얻을 수 있다.

1.2.14 여기서 중요한 함정

다음 파일이 있다고 해보자.



train.py에:

```
import pandas as pd

df = pd.read_csv("../data/train.csv")
```

라고 작성했다.

많은 사람이 이렇게 생각한다.

> train.py가 src안에 있으니까 .. 하면 project겠네.

항상 그런 것은 아니다.

상대 경로의 기본 기준은 **Python 파일이 위치한 디렉터리가 아니라 현재 작업 디렉터리(CWD)다.**

이게 처음 파일 경로를 배울 때 가장 많이 혼동하는 부분 중 하나다.

1.2.15 실행하는 위치가 중요하다

예를 들어 터미널에서:

```
project/src %
```

인 상태에서:

```
python train.py
```

를 실행했다고 하자.

CWD:

```
project/src
```

따라서:

```
../data/train.csv
```

는:

```
project/src
   ↓ ..
project
   ↓ data
project/data/train.csv
```

가 되며 잘 작동한다.

그런데 이번에는:

```
project %
```

에서:

```
python src/train.py
```

를 실행했다고 해보자.

`train.py` 파일은 여전히:

```
project/src/train.py
```

에 있다.

하지만 CWD는 `project`다.

따라서:

`../data/train.csv`

를 해석하면:

project
↓
project의 부모
↓
data/train.csv

를 찾게 된다.

1.2.16 이것 때문에 파일은 분명히 있는데 못찾는 경우가 발생한다

아마 Python이나 ML을 하면서 `FileNotFoundError`를 본적이 있을 것이다.

파일이 정말 없는 경우도 있지만 상당히 흔한 상황은:

> 내가 생각한 기준 디렉터리 ≠ Python의 현재 작업 디렉터리

인 것이다.

그래서 파일을 못 찾을 때 가장 먼저:

```
from pathlib import Path

print(Path.cwd())
```

를 찍어보는 습관이 상당히 유용하다.

1.2.17 이제 `pathlib.Path`를 사용해보자

예전 Python 코드에서는:

```
import os
```

와 문자열 경로 조작이 굉장히 많았다.

현재는 경로 자체를 다룰 때 `pathlib`이 매우 편하다.

```
from pathlib import Path
```

그리고:

```
p = Path("data/train.csv")
```

라고 할 수 있다.

여기서 중요한 사고방식이 있다.

```
"data/train.csv"
```

는 그냥 문자열이다.

반면:

```
Path("data/train.csv")
```

는 경로를 표현하고 조작하기 위한 Path 객체다.

1.2.18 문자열과 Path 객체의 차이

문자열:

```
p = "data/train.csv"
```

Python 입장에서는 그냥 문자들의 나열이다.

```
    d a t a / t r a i n . c s v
```

그 이상을 특별히 알 필요가 없다.

반면:

```
p = Path("data/train.csv")
```

라고 하면 경로 관련 기능을 사용할 수 있다.

예를 들어:

```
p.exist()
```

파일이 실제 존재하는지 확인한다.

```
p.is_file()
```

파일인지 확인한다.

```
p.is_dir()
```

디렉터리인지 확인한다.

```
p.name
```

이라면:

```
train.csv
```

를 얻는다.

1.2.19 경로를 / 로 조립할 수도 있다

pathlib의 편한 기능이다.

```
from pathlib import Path

project = Path("/Users/user/Desktop/project")

data = project / "data"
train = data / "train.csv"
```

결과적으로:

```
/Users/user/Desktop/project/data/train.csv
```

라는 경로가 만들어진다.

처음 보면:

```
project / "data"
```

가 이상할 수 있다.

숫자 나눗셈이 아니다.

> Path객체가 / 연산자를 **경로 연결이라는 의미로 정의해 놓은 것**이다.

그래서:

```
Path("data") / "train.csv"
```

는:

```
data/train.csv
```

라는 경로가 된다.

1.2.20 절대 경로인지 확인할 수도 있다

```
from pathlib import Path

a = Path("/Users/user/Desktop/data.csv")
b = Path("data/data.csv")

print(a.is_absolute())
print(b.is_absolute())
```

결과:

```
True
False
```

왜냐하면:

```
/Users/user/Desktop/data.csv
↑
root부터 시작
```

하지만:

```
data/data.csv
```

는 출발점이 없다.

따라서 CWD가 필요하다.

1.2.21 상대 경로를 절대 경로로 확인해보자

```
from pathlib import Path

p = Path("data/train.csv")

print(p)
print(p.resolve())
```

예를 들어 CWD가:

```
/Users/user/Desktop/project
```

라면:

```
print(p)
```

는:

```
data/train.csv
```

이고,

```
print(p.resolve())
```

는 대략:

```
/Users/user/Desktop/project/data/train.csv
```

가 된다.

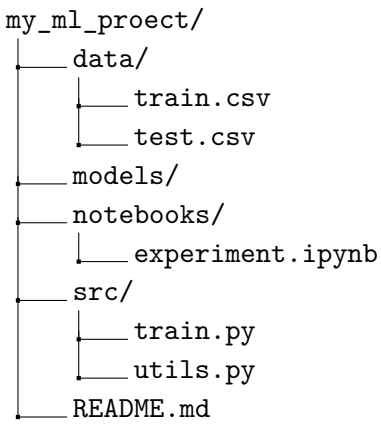
즉 resolve()를 사용하면:

"이 상대 경로가 결국 실제로 어디를 가리키는 거야?"

를 확인하기 좋다.

1.2.22 실제 ML 프로젝트를 하나 만들어보자

앞으로 이 예제를 확장해보자



터미널에서:

```
my_ml_project %
```

에 있다고 하자.

즉, CWD:

```
my_ml_project
```

이다.

그러면 Python에서:

```
from pathlib import Path

cwd = Path.cwd()
print(cwd)
```

대략:

```
/Users/user/.../my_ml_project
```

이다.

그리고:

```
data_dir = cwd / "data"
train_file = data_dir / "train.csv"
```

그러면:

```
my_ml_project/data/train.csv
```

를 가리키게 된다.

1.2.23 Pandas와도 바로 연결된다

Path 객체는 Pandas 같은 현대 Python 라이브러리에 바로 사용할 수 있다.

```
from pathlib import Path
import pandas as pd

data_path = Path("data") / "train.csv"

df = pd.read_csv(data_path)
```

이제 코드의 의미를 완전히 분해할 수 있다.

```
Path("data")
```

현재 작업 디렉터리를 기준으로 data라는 경로를 나타내고,

```
/ "train.csv"
```

를 통해 그 아래의 파일 경로를 만들고,

```
pd.read_csv(data_path)
```

가 해당 위치의 csv를 읽는다.

1.2.24 디렉터리도 만들 수 있다

예를 들어 학습된 모델을:

```
models/
```

에 저장하고 싶다고 하자.

```
from pathlib import Path

model_dir = Path("models")
model_dir.mkdir()
```

그러면 디렉터리를 만들 수 있다.

이미 존재할 가능성이 있다면:

```
model_dir.mkdir(exist_ok=True)
```

를 사용할 수 있다.

더 깊은 구조:

```
outputs/models/checkpoints
```

를 한 번에 만들고 싶다면:

```
path = Path("outputs/models/checkpoints")
path.mkdir(parens=True, exist_ok=True)
```

같은 식으로 사용할 수 있다.

이런 패턴은 ML/DL 프로젝트에서 굉장히 흔하다.

1.2.25 이제 파일 경로와 URL을 절대로 섞지 말자

이것도 앞으로 매우 중요하다.

다음은 **로컬 파일 경로**다.

```
/users/user/project/data/train.csv
```

또는:

```
C:\Users\user\project\data\train.csv
```

컴퓨터의 파일 시스템에서 위치를 찾는다.

반면:

`https://example.com/data/train.csv`

는 URL이다.

이것은:

내 SSD
↓
디렉터리 탐색

의 문제가 아니다.

대략적으로:

URL
↓
네트워트
↓
서버
↓
HTTP 요청
↓
응답

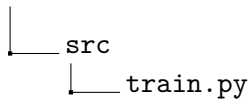
의 세계다.

따라서 나중에 우리가 Path와 URL을 명확히 분리해서 배울 것이다.

1.2.26 지금까지의 개념을 하나의 그림으로 합치자

파일 시스템이:

```
/
└── Users
    └── user
        └── Desktop
            └── my_ml_project
                ├── data
                │   └── train.csv
                └── models
```



라고 하자.

현재 작업 디렉터리:

```
/Users/user/Desktop/my_ml_project
```

이라면:

절대 경로
`/Users/user/Desktop/my_ml_project/data/train.csv`

와:

상대 경로
`data/train.csv`

는 같은 파일을 가리킬 수 있다.

그리고:

`.`

는:

```
/Users/user/Desktop/my_ml_project
```

를 의미하므로:

```
./data/train.csv
```

도 같은 파일이다.

반면:

`..`

는:

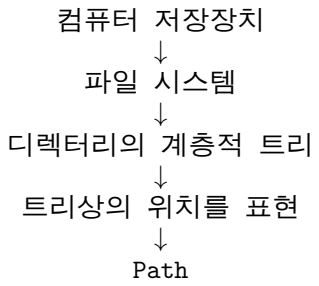
```
/Users/user/Desktop
```

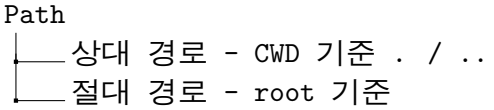
을 의미한다.

1.2.27 여기까지에서 가장 중요한 사고방식

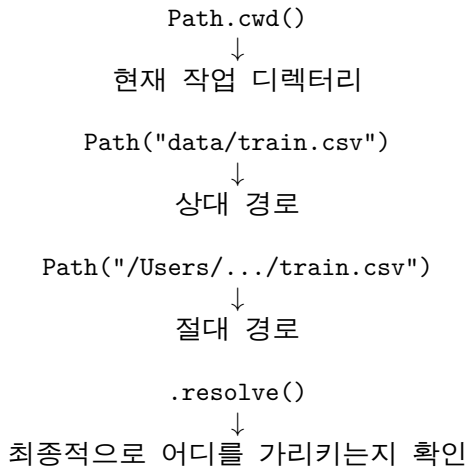
위 내용들을 단순히 기호로 외우면 안된다.

다음 구조를 머릿속에 만드는 것이 목표다.





그리고 Python에서는:



이렇게 연결된다.

1.2.28 다음 단계가 특히 중요하다

바로 sys로 가지 않고 **경로에서 가장 많이 혼동하는 세 가지 기준점**을 분리하는 게 좋다.

1. 현재 작업 디렉터리 → `Path.cwd()`
2. 실행 중인 .py 파일의 위치 → `__file__`
3. Python이 import 대상을 찾는 위치 → `sys.path`

셋 다 화면에는 /Users/... 같은 경로가 나오기 때문에 같은 종류처럼 보이지만, 실제 의미는 완전히 다르다.

여기까지 분리되면 그 다음 sys.argv를 배울 때

```
python src/train.py --data data/trian.csv
```

에서 **shell → Python interpreter → .py 파일 → Path → 실제 파일**이 한번에 연결되기 시작한다.

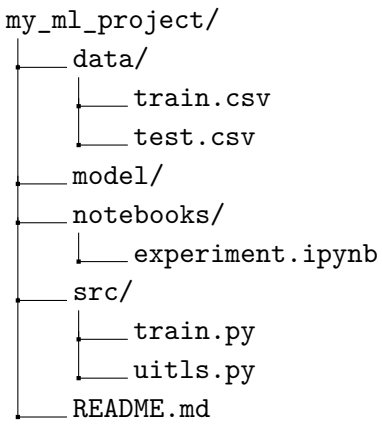
1.3 sys.path, pathlib

Python에서 "경로"라는 값이 보인다고 해서 모두 같은 목적의 경로는 아니다.

무엇을 찾기 위한 경로인지 구분해야 한다.

1.3.1 먼저 실습용 프로젝트를 하나로 고정한다

앞으로 이 구조를 계속 사용한다.



Mac에서 실제 프로젝트 위치가 다음이라고 가정하자.

```
/Users/user/Desktop/my_ml_project
```

따라서 `train.py`의 절대 경로는

```
/Users/user/Desktop/my_ml_project/src/train.py
```

이다.

이제 세 가지를 보자

```
Path.cwd()

__file__

sys.path
```

겉으로는 셋 다 `/Users/...` 같은 경로가 튀어나올 수 있다.

하지만 질문 자체가 다르다

Path.cwd()
지금 어디에서 작업 중인가?

__file__
지금 실행하는 이 파일은 어디 있는가?

sys.path
mport 할 때 Python이 어디를 검색하는가?

1.3.2 Path.cwd() - 나는 지금 어디서 작업하고 있는가?

```
from pathlib import Path

print(Path.cwd())
```

CWD는 Current Working Directory, 즉 **현재 작업 디렉터리**다.

터미널에서:

```
/Users/user/Desktop/my_ml_project %
```

인 상태에서:

```
python src/train.py
```

를 실행했다고 하자.

그러면 보통:

```
Path.cwd()
```

결과는:

```
/Users/user/Desktop/my_ml_project
```

이다.

train.py는 여기 있다.

```
/Users/user/Desktop/my_ml_project/src/train.py
```

그런데 CWD는:

```
/Users/user/Desktop/my_ml_project
```

이며, 둘은 다르다.

1.3.3 `__file__` - 이 Python 파일 자체는 어디 있는가?

`train.py`안에 다음 코드를 넣어보자.

```
print(__file__)
```

그러면 실행 중인 **해당 Python 파일의 경로**를 알 수 있다.

더 확실하게 절대 경로 형태로 보고 싶다면:

```
from pathlib import Path
print(Path(__file__).resolve())
```

결과는:

```
/Users/user/Desktop/my_ml_project/src/train.py
```

같은 식이다.

따라서:

```
Path.cwd()
```

```
/Users/user/Desktop/my_ml_project
```

와:

```
Path(__file__).resolve()
```

```
/Users/user/Desktop/my_ml_project/src/train.py
```

는 전혀 다른 것을 말하고 있다.

1.3.4 이 차이를 그림으로 보면 간단하다

현재:

```
/
└── Users/
    └── user/
        └── Desktop/
            └── my_ml_project - CWD
```

여기에서:

```
python src/train.py
```

를 실행했다.

그러면:

```
my_ml_project <- Path.cwd()
├── data/
├── src/
    └── train.py  <- __file__
```

즉:

> CWD
> 내가 프로그램을 실행하고 있는 작업 기준 위치
>
> __file__
> 실행되는 이 Python 파일 자체의 위치

이다.

1.3.5 그래서 __file__을 이용하면 파일 기준 경로를 만들 수 있다

이 패턴은 실제 프로젝트에서 매우 유용하다

train.py가:

```
my_ml_project/src/train.py
```

에 있다.

```
from pathlib import Path

current_file = Path(__file__).resolve()

print(current_file)
```

결과:

```
/Users/user/Desktop/my_ml_project/src/train.py
```

그런데 우리는 파일이 아니라 src 디렉터리를 원한다고 해보자.

```
current_dir = current_file.parent
```

결과:

```
/Users/user/Desktop/my_ml_project/src
```

한 번 더:

```
project_root = current_file.parent.parent
```

결과:

```
/Users/user/Desktop/my_ml_project
```

가 된다.

따라서:

```
data_path = project_root / "data" / "train.csv"
```

하면:

```
/Users/user/Desktop/my_ml_project/data/train.csv
```

가 된다.

전체 코드:

```python
from pathlib import Path

current_file = Path(__file__).resolve()
project_root = current_file.parent.parent
data_path = project_root / "data" / "train.csv"
print(data_path)
```

이 코드는 CWD에 덜 의존한다.

1.3.6 왜 그런지 살펴보자

train.py를 다음처럼 작성하자.

```python
from pathlib import Path

print("CWD :", Path.cwd())
print("FILE:", Path(__file__).resolve())
```

프로젝트:

```
my_ml_project/
└── src/
    └── train.py
```

이제 실행 방법을 바꿔보자.

1.3.7 실험 A - 프로젝트 root에서 실행

터미널:

```
my_ml_project %
```

여기서:

```
python src/train.py
```

결과:

```
CWD : /Users/user/Desktop/my_ml_project
FILE: /Users/user/Desktop/my_ml_project/src/train.py
```

즉:

```
CWD
my_ml_project/

FILE
my_ml_project/src/train.py
```

1.3.8 실험 B - src 안으로 들어가서 실행

터미널에서:

```
cd src
```

그러면:

```
my_ml_project/src %
```

이제:

```
python train.py
```

결과:

```
CWD : /Users/user/Desktop/my_ml_project/src
FILE: /Users/user/Desktop/my_ml_project/src/train.py
```

비교해보자.

실험 A

```
CWD = my_ml_project
FILE = my_ml_project/src/train.py
```

실험 B

```
CWD = my_ml_project/src
FILE = my_ml_project/src/train.py
```

__file__은 그대로인데 CWD가 바뀌었다.

이게 핵심이다.

1.3.9 그래서 상대 경로의 결과가 달라진다

train.py에 다음 코드가 있다고 하자.

```
from pathlib import path

data = Path("data/train.csv")
print(data.resolve())
```

프로젝트 root에서 실행하면

```
my_ml_project %
```

에서:

```
python src/train.py
```

하면:

```
CWD
/Users/user/Desktop/my_ml_project
```

이므로:

```
data/train.csv
```

는:

```
/Users/user/Desktop/my_ml_project/data.train.csv
```

가 된다.

1.3.10 그런데 src에서 실행하면?

```
my_ml_project/src %
```

에서:

```
python train.py
```

하면 CWD가:

```
/Users/user/Desktop/my_ml_project/src
```

다.

따라서:

```
Path("data/train.csv")
```

는:

```
/Users/user/Desktop/my_ml_project/src/data/train.csv
```

를 의미한다.

그런데 실제 파일은:

```
/Users/user/Desktop/my_ml_project/data/train.csv
```

에 있다.

그러면 FileNotFoundError가 발생할 수 있다.

1.3.11 여기서 아주 중요한 결론

많은 초보자가 이렇게 생각한다.

```
Path("data/train.csv")
```

Python 파일이 있는 위치에서 data/train.csv를 찾겠지?

아니다.

기본적으로 상대 파일 경로를 해석할 때 중요한 기준은:

.py 파일 위치

가 아니라:

Current Working Directory, CWD

이다.

그래서

같은 train.py
+
같은 Path("data/train.csv")

여도 **어디에서 프로그램을 실행했느냐에 따라 가리키는 곳이 달라질 수 있다.**

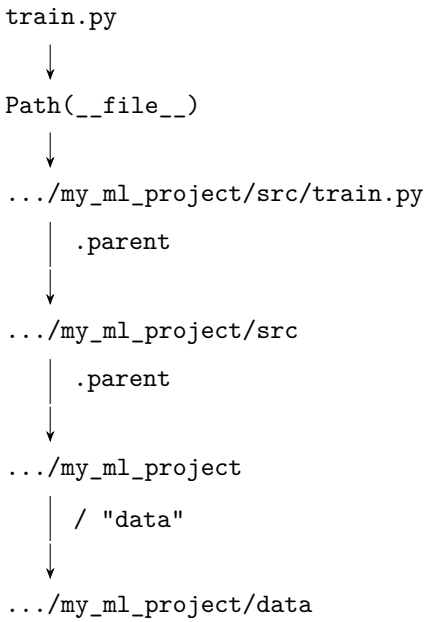
1.3.12 그러면 파일 위치를 기준으로 만들고 싶으면?

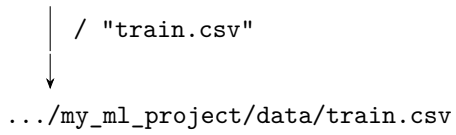
그때 __file__을 이용할 수 있다.

```
from pathlib import Path

project_root = Path(__file__).resolve().parent().parent()
data_path = project_root / "data" / "train.csv"
```

구조적으로:





이제 실행 위치가 바뀌어도 프로젝트 구조 자체가 유지된다면 같은 파일을 가리킬 수 있다.

1.3.13 이제 세 번째인 sys.path

여기서 이름 때문에 혼란이 생긴다.

```
import sys

print(sys.path)
```

결과는 대략:

```
[
'/Users/user/Desktop/my_ml_project/src',
'/Users/user/anaconda3/envs/transformer/lib/python310.zip',
'/Users/user/anaconda3/envs/transformer/lib/python3.10',
'/Users/user/anaconda3/envs/transformer/lib/python3.10/site-packages',
...
]
```

이건 train.csv를 찾기 위한 목록이 아니다.

Python이:

```
import numpy
import pandas
import transformers
```

같은 **모듈/패키지를 import할 때 검색하는 위치**와 관련된 목록이다.

즉:

```
Path.cwd()
작업 위치

__file__
이 Python 파일 위치

sys.path
import 검색 위치들
```

이다.

1.3.14 sys.path는 하나의 경로가 아니라 목록이다

이 차이도 중요하다.

```
Path.cwd()
```

는 하나의 현재 작업 위치를 나타낸다.

반면:

```
sys.path
```

는 여러 검색 위치를 담고 있다.

개념적으로:

```
sys.path
├── 위치 A
├── 위치 B
├── 위치 C
├── site-packages
└── ...
```

Python이:

```
import pandas
```

를 만나면 이 검색 체계를 이용해 pandas를 찾는다.

그래서 Conda 환경의:

`.../envs/transformer/lib/python3.10/site-packages`

같은 위치가 중요해진다.

그곳에 transformer, pandas, numpy 등이 설치되어 있기 때문이다.

1.3.15 그래서 sys.path와 파일 읽기는 분리해야 한다

예를 들어:

```
import pandas as pd

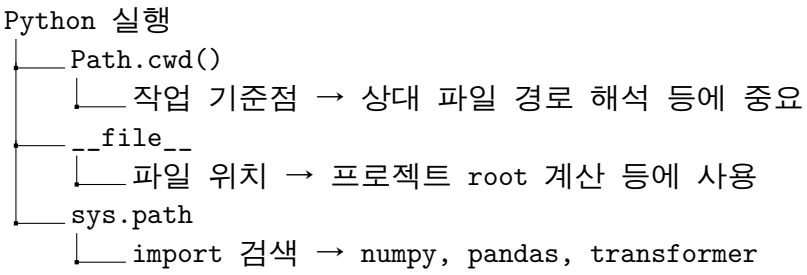
df = pd.read_csv("data/train.csv")
```

에서 `data/train.csv`를 찾는 문제와 pandas를 찾는 문제는 다르다.

전자는 파일 시스템 + 파일 경로 문제다.

후자는 Python import system + sys.path 문제다.

1.3.16 지금까지를 하나의 그림으로 통합



1.3.17 pathlib.Path를 제대로 다뤄보자

여기서부터는 실제 도구 사용이다.

우선:

```
from pathlib import Path
```

한다.

1.3.18 Path객체 만들기

```
p = Path("data/train.csv")
```

현재로서는 파일을 읽은 게 아니다.

그냥:

data/train.csv라는 경로를 나타내는 객체

를 만든 것이다.

이 차이가 중요하다.

```
Path("data/train.csv")
```

한다고 파일이 열리는 것도 아니고 생성되는 것도 아니다.

1.3.19 현재 작업 디렉터리

```
Path.cwd()
```

예:

```
/Users/user/Desktop/my_ml_project
```

1.3.20 홈 디렉터리

```
Path.home()
```

Mac에서는 예를 들어:

```
/Users/user
```

같은 결과가 나온다.

1.3.21 경로 결합

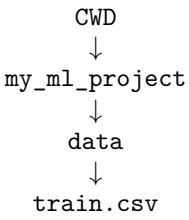
가장 자주 쓴다.

```
project = Path.cwd()
data_dir = project / "data"
train_file = data_dir / "train.csv"
```

또는 한 번에:

```
train_file = Path.cwd() / "data" / "train.csv"
```

개념:



1.3.22 파일/디렉터리 존재 여부

```
path = Path("data/train.csv")

print(path.exists())
```

존재하면 `True`, 없으면 `False`

ML 코드에서 굉장히 유용하다.

```
if not path.exist():
    print("데이터 파일이 없습니다.")
```

1.3.23 파일인지 확인

```
path.is_file()
```

예:

```
Path("data/train.csv").is_file()
```

실제 파일이면 True, 아니면 False

1.3.24 디렉터리인지 확인

```
Path("data").is_dir()
```

실제 디렉터리면 True, 아니면 False

따라서:

exists()
→ 존재하는가?

is_file()
→ 파일인가?

is_dir()
→ 디렉터리인가?

로 구분하면 된다.

1.3.25 파일 이름 가져오기

```
path = Path("/Users/user/Desktop/project/data/train.csv")

print(path.name)
```

결과:

```
train.csv
```

1.3.26 확장자 가져오기

```
print(path.suffix)
```

결과:

```
.csv
```

ML/DL에서는:

```
.csv
.json
.pt
.pth
.ckpt
.safetensors
```

같은 확장자를 다룰 일이 많다.

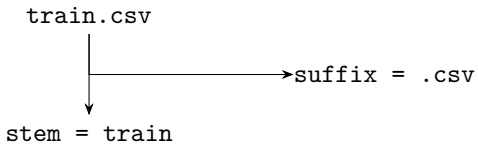
1.3.27 확장자를 제외한 파일 이름

```
print(path.stem)
```

결과:

```
train
```

즉:



그리고:

```
name = train.csv
```

이다.

1.3.28 부모 디렉터리

```
print(path.parent())
```

경로가:

```
/Users/user/project/data/train.csv
```

라면:

```
/Users/user/project/data
```

가 나온다.

다시:

```
path.parent.parent
```

하면:

```
/Users/user/project
```

이다.

아까 `__file__`과 결합했던 이유가 이것이다.

1.3.29 확장자 변경

```python
path = Path("data/train.csv")
new_path = path.with_suffix(".json")
print(new_path)
```

결과:

```
data/train.json
```

주의할 점은 이 코드가 **실제 파일의 확장자를 변경한 것은 아니다.**

새로운 경로 객체를 만든 것이다.

이 구분은 중요하다.

```
path.with_suffix(".json")
```

은:

.json으로 끝나는 새로운 Path를 만들어라.

이지, **SSD의 파일 이름을 실제로 바꿔라가 아니다.**

1.3.30 파일 이름 자체 변경

경로 표현만 바꾸려면:

```
path = Path("data/train.csv")
new_path = path.with_name("train_v2.csv")
```

결과:

```
data/train_v2.csv
```

이것 역시 **경로 객체 생성**이다.

실제 파일 변경과는 다르다.

1.3.31 실제 디렉터리 만들기

이제부터는 실제 파일 시스템에 영향을 준다.

```
output_dir = Path("output")
output_dir.mkdir()
```

실제로 output/디렉터리가 생성된다.

이미 있으면 오류가 날 수 있으므로 보통:

```
output_dir.mkdir(exist_ok=True)
```

라고 한다.

1.3.32 중첩 디렉터리 생성

```
checkpoint_dir = Path("outputs/models/checkpoints")

checkpoint_dir.mkdir(
    parents = True,
    exist_ok = True
```

```
)
```

그러면 필요한 부모 디렉터리까지 만든다.

```
outputs/
├── models/
└── checkpoints/
```

ML 코드에서 정말 자주 보는 패턴이다.

1.3.33 간단한 텍스트 파일 쓰기

Path자체로도 가능하다.

```
path = Path("result.txt")

path.write_text("accuracy: 0.95")
```

실제로 result.txt가 생성된다.

읽으려면:

```
text = path.read_text()
print(text)
```

결과:

```
accuracy: 0.95
```

1.3.34 파일 삭제

```
path = Path("result.txt")
path.unlink()
```

실제 파일을 삭제한다.

따라서 unlink()는 앞에서 본:

```
with_suffix(x)
```

```
with_name()
```

보다 훨씬 주의해야 한다.

`unlink()`는 실제 파일 시스템을 변경하기 때문이다.

1.3.35 빈 디렉터리 삭제

```
directory = Path("outputs")
directory.rmdir()
```

단, 기본적으로 비어있는 디렉터리여야 한다.

파일이 여러 개 들어 있는 폴더를 통째로 제거하는 것은 별도의 문제로 다루자.

1.3.36 디렉터리 안을 살펴보기

프로젝트가:

```
data/
├── train.csv
├── labels.json
└── README.txt
```

라고 하자.

```
data_dir = Path("data")

for path in data_dir.iterdir():
    print(path)
```

대략:

```
data/train.csv
data/test.csv
data/labels.json
data/README.txt
```

가 나온다.

즉 `iterdir()`는:

> 이 디렉터리 바로 아래 뭐가 있는지 순회

한다.

1.3.37 특정 파일만 찾기 - `glob`

csv만 찾고 싶다.

```
data_dir = Path("data")
for path in data_dir.glob("*.csv"):
    print(path)
```

결과:

```
data/train.csv
datat/test.csv
```

여기서:

```
*.csv
```

는:

> 이름은 무엇이든 상관없고 `.csv`로 끝나는 것

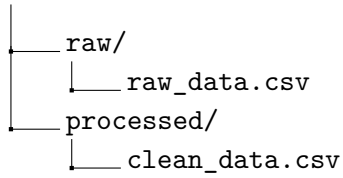
이라는 패턴이다.

*를 wildcard라고 부른다.

1.3.38 하위 디렉터리까지 전부 찾기 - `rglob`

구조가:

```
data/
└── train.csv
```



라고 하자.

```
for path in Path("data").rglob("*.csv"):
    print(path)
```

그러면 하위 디렉터리까지 재귀적으로 검색해서:

```
data/train.csv
data/raw_data.csv
data/processed/clean_data.csv
```

를 찾을 수 있다.

ML 데이터셋 관리에서 꽤 유용하다.

1.3.39 이름 변경 - 실제 파일 시스템 조작

아까:

```
with_name()
```

은 새로운 경로만 만들었다.

실제 파일 이름을 변경하려면 예를 들어:

```
old = Path("data/train.csv")
new = Path("data/train_v2.csv")

old.rename(new)
```

라고 할 수 있다.

실제로:

$$\texttt{train.csv} \rightarrow \texttt{train_v2.csv}$$

로 변경된다.

이 차이를 기억하자.

```
with_name()
with_suffix()
      ↓
Path 표현 생성

rename()
unlink()
mkdir()
   ↓
실제 파일 시스템 변경
```

1.3.40 Path에서 아주 중요한 개념적 구분

이걸 객체 관점에서 보면 더 명확하다.

```
p = Path("data/train.csv")
```

여기서 p는 **파일 그 자체가 아니다.**

p는:

파일 시스템상의 어떤 위치를 표현하는 Python 객체

이다.

그래서:

```
p.exists()
```

라고 물어보는 것이다.

즉:

```
Path 객체
   ↓ 이 위치에
data/train.csv
   ↓ 실제 파일 시스템을 확인
```

파일이 있는가?

이다.

이 관점은 Python의 **name → object** 사고방식과도 잘 맞는다.

```
p = Path("data/train.csv")
```

에서 p라는 이름이 Path객체를 가리키고 있는 것이다.

그리고 그 Path객체가 파일 시스템상의 위치를 표현한다.

1.3.41 실전 ML 코드로 합쳐보자

예를 들어:

```
from pathlib import Path
import pandas as pd

project_root = Path(__file__).resolve().parent.parent

data_dir = project_root / "data"
model_dir = project_root / "models"

train_path = data_dir / "train.csv"

if not train_path.exists():
    raise FileNotFoundError(f'데이터가 없습니다: {train_path}')

mdoel_dir.mkdir(
    parents = True,
    exist_ok = True
)

df = pd.read_csv(train_path)
```

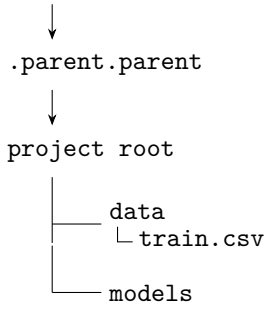
이제 이 코드가 거의 전부 읽혀야 한다.

구조는:

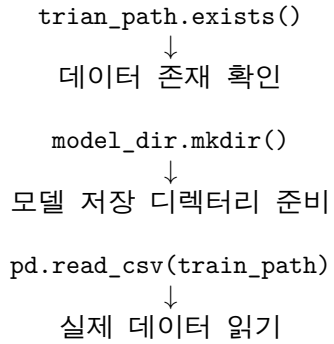
현재 Python 파일

↓

__file__



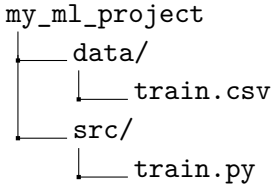
그리고:



가 된다.

1.3.42 마지막으로 세 개를 확실히 분리하자

프로젝트:



프로젝트 root에서:

```
python src/train.py
```

를 실행했다.

```
Path.cwd()
→ /Users/user/Desktop/my_ml_project
의미: 내 작업 기준점은 어디인가?

Path(__file__).resolve()
→ /Users/user/Desktop/my_ml_project/src/train.py
의미: 현재 이 Python 파일은 어디 있는가?

sys.path
→ [
    ...,
    ".../site-packages",
    ...
    ]
의미: Python이 import할 모듈/패키지를 어디에서 찾는가?
```

이 세 가지는 **모두 경로를 다루지만 서로 다른 질문에 대한 답**이다.

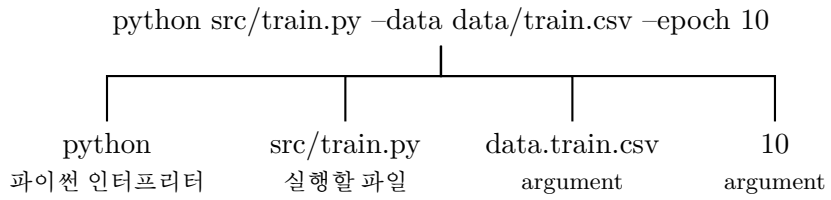
1.3.43 여기서 다음 단계는 sys.argv이다

이제 드디어 **커맨드라인**으로 들어갈 기반이 거의 만들어졌다.

다음에는 이 한 줄을 완전히 해부하면 된다.

```
python src/train.py --data data/train.csv --epochs 10
```

지금까지 배운 걸 기준으로 이미 앞부분은 이해할 수 있다.



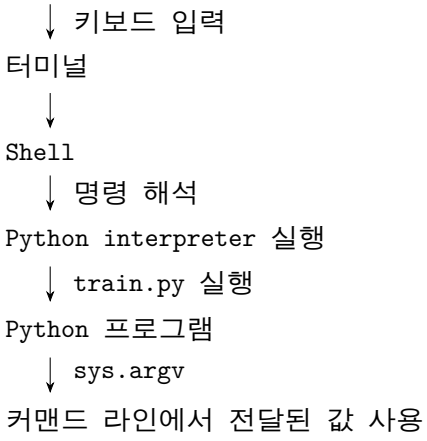
터미널에:

```
python src/train.py --data data/train.csv --epochs 10
```

을 입력한다.

이걸 구조적으로 보면:

사용자



여기서 새로운 개념들이 등장한다.

Terminal
Shell
Command
Argument
Option
Python interpreter
sys.argv

하나씩 분리하자.

1.3.44 터미널(Terminal)은 무엇인가?

Mac에서 Terminal, iTerm, VS Code의 Terminal 같은 화면을 본다고 하자.

예를 들어:

```
(transformer) user@MacBook my_ml_project %
```

그리고 여기에:

```
python src/train.py
```

라고 입력한다.

흔히 이걸 보고:

터미널이 Python을 실행한다.

라고 말하지만 조금 더 정확하게 보면 **터미널과 Shell은 다르다.**

터미널은 사용자와 Shell이 상호작용할 수 있는 **입출력 인터페이스**에 가깝다.

쉽게 말하면:

```
Terminal
----------------------------
python train.py
hello
----------------------------
```

처럼 명령을 입력하고 결과를 볼 수 있는 창을 제공한다.

1.3.45 실제로 명령을 해석하는 것은 Shell

Mac에서는 흔히 zsh를 사용한다.

Linux에서는 zsh, bash 등이 있고,

Windows에서는 PowerShell, cmd 등을 만날 수 있다.

Shell의 역할은 대략:

> 사용자가 입력한 명령을 해석해서 적절한 프로그램을 실행하는 것

이다.

그래서:

```
python src/train.py
```

라고 입력하면 Shell이 이것을 해석한다.

1.3.46 공백이 상당히 중요하다

다음 명령을 보자.

```
python src/train.py --data data/train.csv --epochs 10
```

Shell입장에서는 대략 공백을 기준으로 여러 개의 토큰으로 분리된다.

```
python
src/train.py
--data
data/train.csv
--epochs
10
```

즉 개념적으로:

```
[python]
[src/train.py]
[--data]
[data/train.csv]
[--epochs]
[10]
```

처럼 된다.

물론 따옴표, escape 등의 규칙 때문에 실제 Shell parsing은 이것보다 더 복잡하지만 지금은 이정도면 충분하다.

1.3.47 첫 번째 python은 무엇인가?

```
python src/train.py
└── 이것은 실행하려는 command다.
```

Shell입장에서는:

```
python이라는 프로그램을 실행하라.
```

라는 의미다.

여기서 Python interpreter가 등장한다.

1.3.48 Python interpreter란?

우리가 작성하는:

```
print("hello")
```

는 그 자체로 CPU가 바로 실행하는 기계어가 아니다.

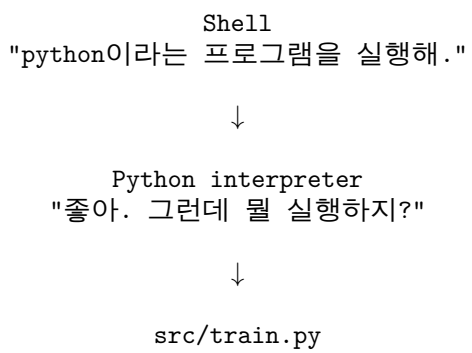
Python 코드를 읽고 실행하는 프로그램이 필요하다.

그 프로그램이 **Python interpreter**다.

따라서:

```
python src/train.py
```

는 대략:



라는 구조다.

1.3.49 Conda와도 바로 연결된다

예를 들어:

```
conda activate transformer
```

를 했다.

프롬프트가:

```
(transformer) %
```

처럼 바뀌었다.

그리고:

```
python
```

을 실행한다.

여기서 중요한 질문은:

수많은 Python 중 어떤 Python이 실행되는가?

이다.

Conda 환경이 활성화되어 있다면 보통 해당 환경의 Python interpreter가 선택된다.

Windows transformer 환경을 예로 들면:

```
C:\Users\user\anaconda3\envs\transformer\python.exe
```

같은 실행 파일이다.

Python 안에서도 확인할 수 있다.

```
import sys

print(sys.executable)
```

즉:

```
Shell
  ↓ python
Conda transformer 환경의 python.exe
  ↓
Python interpreter
```

라고 연결된다.

1.3.50 여기서 sys.executable의 정체가 나온다

이제 sys가 등장한다.

```
import sys

print(sys.executable)
```

sys.executable은 대략:

> 현재 이 Python 프로그램을 실행하고 있는
>
> Python interpreter 실행 파일은 어디 있는가?

를 알려준다.

예를 들어:

```
C:\Users\user\anaconda3\envs\transformer\python.exe
```

그러니까 이것도 단순한 파일 경로 이야기가 아니다.

현재 실행 중인 Python 환경을 확인하는 것이다.

그래서 환경 문제를 디버깅할 때 굉장히 유용하다.

예를 들어:

> 분명 transforemrs를 설치했는데 import transformers가 안되는데?

이럴 때:

```
import sys

print(sys.executable)
```

을 보면 엉뚱한 Python을 실행하고 있었다는 걸 발견하는 경우가 있다.

1.3.51 이제 두 번째 토큰 src/train.py

전체 명령:

```
python src/train.py --data data/train.csv --epochs 10
```

여기에서:

```
python
└── src/train.py
```

이다.

Shell은 Python 프로그램을 실행하면서 나머지 문자열들을 Python 쪽에 전달한다.

Python interpreter 입장에서 첫 번째 중요한 값은:

```
src/train.py
```

이다.

즉:

```
이 Python 스크립트를 실행해라.
```

1.3.52 여기서 앞에서 배운 CWD가 바로 등장한다

현재 터미널 위치가:

```
my_ml_project/
```

라고 하자.

구조:

```
my_ml_project/
├── data/
│   └── train.csv
└── src/
    └── train.py
```

그리고:

```
python src/train.py
```

를 실행한다.

`src/train.py`는 **상대 경로**다.

무엇을 기준으로?

앞에서 배운 CWD다.

```
CWD
↓
my_ml_project
+
src/train.py
↓
my_ml_project/sr/train.py
```

앞에서 공부한 파일 경로와 커맨드 라인이 연결되었다.

1.3.53 그런데 뒤에 이상한 것들이 더 있다

```
python src/train.py --data data/train.csv --epochs 10
```

Python 파일 뒤에:

```
--data
data.train.csv
--epochs
10
```

이 있다.

이것들이 **커맨드라인 인수**(`command-line arguments`)다.

프로그램을 실행하면서 프로그램에 추가 정보를 전달하는 것이다.

1.3.54 왜 굳이 커맨드라인으로 정보를 전달할까?

예를 들어 `train.py`에 이렇게 작성할 수도 있다.

```
data_path = "data/train.csv"
epochs = 10
```

그런데 다음번에는 다른 데이터를 학습하거나 epochs를 변경하고 싶다.

그러면:

```
data_path = "data/test.csv"
epochs = 20
```

으로 수정해야 한다.

불편하다.

그래서 프로그램 자체와 실행 설정을 분리한다.

```
python src/train.py --data data/train.csv --epochs 10
```

다음에는:

```
python src/train.py --data data/new_data.csv --epochs 30
```

라고 실행할 수 있다.

`train.py` 코드는 수정하지 않았다.

1.3.55 이게 커맨드라인 인수의 핵심 목적이다

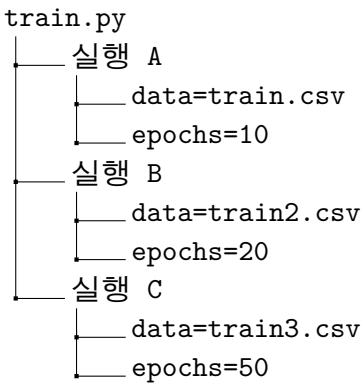
프로그램 코드
`train.py`

+

실행할 때 필요한 설정
`data/train.csv`
`epochs = 10`

를 분리한다.

즉:



가 가능해진다.

ML/DL에서 CLI를 많이 사용하는 이유 중 하나다.

1.3.56 이제 Argument와 Option을 구분하자

간단한 명령:

```
python train.py data/train.csv
```

여기서:

```
data/train.csv
```

는 argument다.

프로그램에 전달하는 값이다.

그런데 값이 많아지면 문제가 생긴다.

```
python train.py data/train.csv 10 32 model.pt
```

이게 뭘 의미하는지 알아보기 어렵다.

data/train.csv → 데이터?

10 → epochs?

32 → batch size?

model.pt → outputs?

그래서 이름을 붙여준다.

```
python train.py \
    --data data/train.csv \
    --epochs 10 \
    --batch-size 32 \
    --output model.pt
```

훨씬 명확하다.

1.3.57 --data, --epochs가 option이다

보통:

```
--data
--epochs
--batch-size
--output
```

같은 것을 option이라고 부른다.

그리고:

```
--data data/train.csv
```

를 보면:

```
--data
└── 어떤 설정인지 나타내는 option

data/train.csv
└── 전달할 value
```

라고 볼 수 있다.

다만 CLI 관련 용어는 라이브러리/문서에 따라

argument, option, parameter 등을 조금 다르게 쓰기도 한다.

중요한 것은 구조다.

1.3.58 위치로 의미를 판단하는 positional argument도 있다

예를 들어:

```
python copy.py source.txt destination.txt
```

이라고 하자.

프로그램이:

첫 번째 값 = source

두 번째 값 = destination

이라고 약속했다면:

```
source.txt
destination.txt
```

는 위치 자체가 의미를 가진다.

이것을 **positional argument라고 한다.**

예를 들어:

```
git add README.md
```

같은 명령에서도 비슷한 사고방식을 볼 수 있다.

1.3.59 Option은 이름을 붙여 전달한다

반면:

```
python train.py --epochs 10
```

에서는:

`--epochs`

라는 이름을 통해 10의 의미를 알 수 있다.

그래서 개념적으로:

`Positional - 위치로 의미 결정`

`python train.py data.csv 10`

위의 내용 대비:

`Option - 이름으로 의미 명시`

`python train.py --data data.csv --epochs 10`

라고 생각하면 된다.

1.3.60 이제 드디어 sys.argv

Python은 자신에게 전달된 커맨드라인 인수에 접근할 방법이 필요하다.

그것이:

```
import sys
print(sys.argv)
```

이다.

`argv`는 전통적으로:

`argument vector`

에서 온 이름이다.

여기서 vector를 NumPy 벡터처럼 수학적으로 생각할 필요는 없다.

전달된 argument들의 순서 있는 목록정도로 이해하면 충분하다.

1.3.61 직접 실험해보자

train.py:

```
import sys

print(sys.argv)
```

터미널:

```
python train.py
```

결과는 대략:

```
['train.py']
```

여기서 중요한 사실이 나온다.

sys.argv[0]에는 보통 실행 대상 스크립트 이름/경로가 들어간다.

1.3.62 인수를 하나 전달해보자

```
python train.py hello
```

그러면:

```
['train.py', 'hello']
```

가 된다.

즉:

```
sys.argv
├── [0] = 'train.py'
└── [1] = 'hello'
```

이다.

1.3.63 여러 개 전달

```
python train.py hello world 10
```

그러면:

```
[
    'train.py',
    'hello',
    'world',
    '10'
]
```

가 된다.

인덱스로 접근이 가능하다.

```
import sys

print(sys.argv[0])
print(sys.argv[1])
print(sys.argv[2])
print(sys.argv[3])
```

결과

```
train.py
hello
world
10
```

1.3.64 아주 중요한 점: 전부 문자열이다

```
python train.py 10
```

이라고 했다고 하자.

사람은 10을 보고 숫자라고 생각한다.

하지만:

```
import sys
```

```
print(sys.argv[1])
print(type(sys.argv[1]))
```

결과:

```
10
<class 'str'>
```

이다.

즉:

터미널에서 전달
10
↓
sys.argv
↓
"10"

이다.

커맨드라인 인수는 문자열 형태로 들어오므로 필요하면 변환해야 한다.

1.3.65 epochs라면 숫자로 변환해야 한다

```
python train.py 10
```

train.py:

```
import sys

epochs = int(sys.argv[1])

print(epochs)
pirnt(type(epochs))
```

결과:

```
10
<class 'int'>
```

즉 input()과 비슷한 면이 있다.

```
x = input()
```

도 기본적으로 문자열을 받는다.

커맨드라인 인수도:

```
sys.argv[1]
```

에서 문자열을 받는다.

1.3.66 파일 경로와 결합해보자

터미널:

```
python train.py data/train.csv
```

Python:

```
import sys
from pathlib import Path

data_path = Path(sys.argv[1])

print(data_path)
print(data_path.resolve())
```

여기서 흐름이 중요해진다.

```
Shell
   ↓ "data/train.csv"
Python
   ↓ sys.argv[1]
"data/train.csv"
   ↓ Path(...)
Path("data/train.csv")
   ↓ CWD 기준
실제 파일 시스템상의 위치
```

지금가지 공부한 두 파트가 완전히 연결되었다.

1.3.67 Pandas까지 붙이면

```
import sys
import pandas as pd
from pathlib import Path

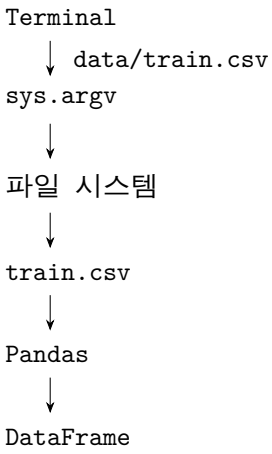
data_path = Path(sys.argv[1])

df = pd.read_csv(data_path)
print(df.head())
```

실행:

```
python train.py data/train.csv
```

그러면:



이 된다.

이제 CLI가 ML 코드와 연결되기 시작한다.

1.3.68 그런데 sys.argv만 직접 쓰면 금방 불편해진다

예를 들어:

```
python train.py data/train.csv 10 32 model.pt
```

코드는:

```
data = sys.argv[1]
epochs = int(sys.argv[2])
batch_size = int(sys.argv[3])
outuput = sys.argv[4]
```

가 된다.

문제가 많다.

순서를 기억해야 하고,

```
python train.py data/train.csv 32 10 model.pt
```

처럼 순서를 잘못 넣어도 사람이 알아채기 어렵다.

인수가 빠지면 `IndexError`가 날 수도 있고 도움말도 없다.

1.3.69 그래서 argparse가 등장한다

Python 표준 라이브러리에:

```
import argparse
```

가 있다.

이걸 사용하면:

```
python train.py \
    --data data/train.csv \
    --epochs 10 \
    --batch-size 32
```

같은 CLI를 훨씬 체계적으로 만들 수 있다.

예를 들어:

```
import argparse

parser = argparse.ArgumentParser()

parser.add_argument("--data")
parser.add_argument("--epochs, type=int)
```

```
args = parser.parse_args()

print(args.data)
print(args.epochs)
```

실행:

```
python train.py --data data/train.csv --epochs 10
```

그러면:

```
args.data
```

는:

```
datat/train.csv
```

이고:

```
args.epochs
```

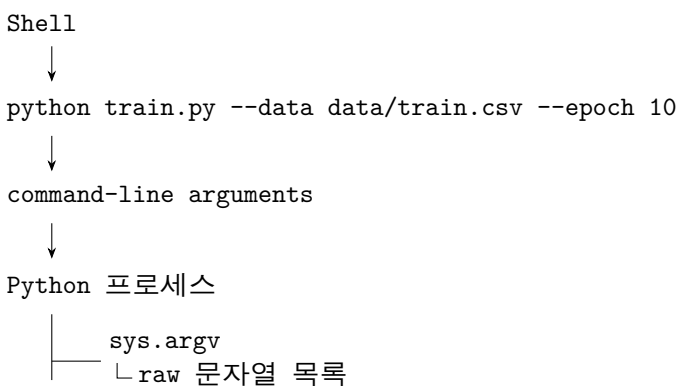
는 `type=int`를 지정했으므로 정수 10 이다.

1.3.70 argparse가 마법을 쓰는 것은 아니다

이것도 구조적으로 이해하는 게 중요하다.

밑바닥에는 여전히 커맨드라인에서 전달된 argument가 있다.

개념적으로:



```
└── argparse
    └ 해석/검증/변환
```

sys.argv는 비교적 **날것 그대로 접근**하는 방법이고,

argparse는 그것을 **구조적으로 해석해주는 도구**라고 보면 된다.

1.3.71 Option을 조금 더 제대로 만들어보자

```
import argparse
from pathlib import Path

parser = argparse.Argumentparser()

parser.add_argument(
    "--data",
    type=Path,
    required=True
)

parser.add_argument(
    "--epochs",
    type=int,
    default=10
)

args = parser.parse_args()
```

이제:

```
python train.py --data data/train.csv
```

라고만 실행해도:

```
args.data
```

는 Path객체가 되고:

```
args.epochs
```

는 기본값 10이 된다.

1.3.72 required=True

```
parser.add_argument(
    "--data",
    required=True
)
```

라고 하면:

--data는 반드시 전달해야 한다.

라는 의미다.

따라서:

```
python train.py
```

라고 실행하면 argparse가 오류를 알려준다.

직접:

```
if len(sys.argv) < 2:
...
```

같은 검사를 만들 필요가 줄어든다.

1.3.73 default

```
parser.add_argument(
    "--epochs",
    type=int,
    default=10
)
```

는:

사용자가 epochs를 안 주면 10을 사용해라.

라는 뜻이다.

따라서:

```
python train.py --data data/train.csv
```

라면:

```
args.epochs == 10
```

이고

```
python train.py --data data/train.csv --epochs 50
```

라면:

```
args.epochs == 50
```

이다.

1.3.74 도움말도 자동으로 만들 수 있다

```
parser.add_argument(
    "--epochs",
    type=int,
    default=10,
    help="Number of training epochs"
)
```

그리고:

```
python train.py --help
```

라고 실행하면 CLI 사용법을 보여준다.

그래서 제대로 만들어진 Python 프로그램에서:

```
-h
--help
```

를 자주 볼 수 있는 것이다.

1.3.75 이제 실제 ML 형태로 만들어보자

```
import argparse
from pathlib import Path

parser = argparse.ArgumentParser(
    description="Train a classification model"
)

parser.add_argument(
    "--data",
    type=Path,
    required=True,
    help="Path to training data"
)

parser.add_argument(
    "--epochs",
    type=int,
    default=10
)

...
```

실행:

```
python src/train.py \
    --data data/train.csv \
    --epochs 20 \
    --batch-size 64 \
    --output models/run1
```

그러면 프로그램 내부에서는:

```
args.data
→ Path("data/train.csv")

args.epochs
→ 20

args.batch_size
→ 64

args.output
→ Path("models/run1")
```

처럼 사용할 수 있다.

1.3.76 이제 이 명령 전체가 읽혀야 한다

```
python src/train.py --data data/train.csv --epochs 10
```

앞에서부터 읽자.

python → Shell에게 Python interpreter를 실행하라고 요청

src/train.py → Python에게 실행할 스크립트 전다

--data → data라는 option

data/train.csv → 해당 option에 전달되는 값

--epochs → epochs라는 option

10 → 전달되는 값

그리고 Python 안에서는 원시적으로 sys.argv를 통해 이 정보에 접근할 수 있고,

더 체계적인 프로그램에서는 argparse로 파싱한다.

1.3.77 지금까지 공부한 파일 경로까지 전부 합치면

```
python src/train.py --data data/train.csv --epochs 10
```

을 실행했다고 하자.

Terminal
↓
Shell
↓ command 해석
python
↓ 어떤 Python?
Conda 환경의 Python interpreter
↓ sys.executable로 확인 가능
src/train.py
↓ CWD 기준 상대 경로

이 구조가 이번 파트의 핵심이다.

1.3.78 그리고 sys의 역할도 조금씩 선명해진다

지금까지 등장한 sys 기능은 두 개다.

```
sys.executable
→ 현재 어떤 Python interpreter가 나를 실행하고 있는가?

sys.argv
→ 이 프로그램을 실행할 때 어떤 command-line argument가 전달되었는가?

그리고 앞에서 잠깐 본:
sys.path
→ import할 모듈을 어디에서 찾는가?
```

따라서 sys라는 모듈의 공통점이 보이기 시작한다.

```
sys
└── 현재 실행중인 Python과 실행 환경에 대한 접근
    ├── executable - interpreter
    ├── argv - 실행 인수
    └── path - import 검색
```

sys를 파일 경로 라이브러리라고 생각하면 안 되는 이유도 상당히 명확해진다.

1.3.79 argparse에 대한 추가 내용

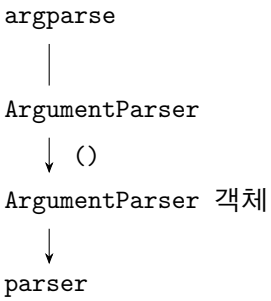
```
import argparse
```

그냥 Python 표준 라이브러리의 argparse 모듈을 가져온 것이다.

우리가 사용하고 싶은 것은 이 안에 있는 argparse.ArgumentParser다.

```
parser = argparse.ArgumentParser()
```

여기서 객체 관점으로 보는 게 중요하다.



즉:

```
parser = argparse.ArgumentParser()
```

는 ArgumentParser를 호출하여 객체를 만들고 parser라는 이름에 바인딩한 것이다.

이 객체의 역할은:

> 커맨드라인 입력을 어떻게 해석할 것인지 관리하는 객체

라고 생각하면 된다.

```
parser.add_argument("--name")
```

이제 parser에게 말하는 것이다.

> --name 이라는 입력을 받을 거야.

그리고:

```
parser.add_argument("--age", type=int)
```

는:

> --age 라는 입력도 받을 거야. 그리고 이 값은 int로 변환해.

이다.

그러니까 add_argument()는 이름 그대로다.

"내가 어떤 argument를 받아야 하지?"
↓
add_argument("--name")
↓
add_argument("--age", type=int)

입력 규칙을 parser에 하나씩 등록하는 것이다.

하지만 아직 실제 값을 읽은 게 아니며 이게 중요하다.

```
parser.add_argument("--name")
parser.add_argument("--age", type=int)
```

여기까지 했다고:

```
Kim
30
```

가 들어온 게 아니다.

그냥:

앞으로 이런 입력을 받을 예정이다.

라는 규칙만 만들어놓은 것이다.

프로그래밍적으로 표현하면:

프로그램의 입력 설계

단계다.

이제:

```
args = parser.parse_args()
```

가 나온다.

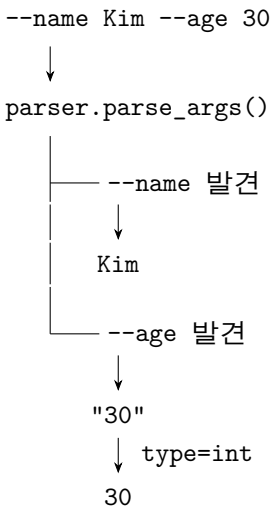
여기서 실제 커맨드라인을 읽고 parser가 등록된 규칙에 따라 해석한다.

우리가:

```
python hello.py --name Kim --age 30
```

라고 실행했다.

그러면 개념적으로:



그리고 결과 객체를 반환한다.

그 객체에 `args`라는 이름을 붙인 것이다.

이후:

```
print(args)
```

를 해보자.

대략:

```
Namespace(name='Kim', age=30)
```

가 나온다.

즉 `parser`가 지저분한 `command-line` 입력을:

```
--name Kim --age 30
```

에서:

```
Namespace
├── name = "Kim"
└── age = 35
```

라는 Python에서 다루기 쉬운 객체로 정리해 준 것이다.

그래서:

```
args.name
args.age
```

로 접근할 수 있다.

이게 sys.agrv[1],[2],[3]를 기억하는 것보다 훨씬 편한 이유다.

1. ArgumentParser()

   Argument + Parser 는 argument를 parse하는 것

   parse : **구조를 분석해서 의미 있는 형태로 해석한다**정도의 의미

   따라서:

   ArgumentParser = command-line argument를 해석하는 객체

   라고 보면 외우지 않아도 이름 자체가 기능 설명임을 알 수 있다.

2. args = parser.parse_args()

   이것 또한 마찬가지다

   parse + args 이므로 arguments를 해석하라.

3. argparse는 option의 - 를 attribute의 _ 형태로 만들어준다.

Command Line		Python
--batch-size	→	args.batch_size
--learning rate	→	args.learning_rate

1.3.80 기억할 최소 골격과 자주 나타나는 오류

당장은 이것만 직접 칠 수 있게 되는 것이 목표다.

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--name")
parser.add_argument("--age", type=int)
args = parser.parse_args()
print(args.name)
```

그리고 터미널:

```
python example.py --name Kim --age 30
```

이 작은 프로그램을 **보지 않고 한 번 작성해보는 것**을 추천한다.

지금부터 몇 번 직접 타이핑하면서 아래 패턴을 손에 넣으면 된다.

만든다
ArgumentParser()

↓

등록한다
add_argument()

↓

해석한다
parser_args()

↓

사용한다
args.xxx

\ 는 단순히 줄바꿈 기호가 아니다.

zsh/bash 같은 Shell에서 줄 끝의

\

는 **다음 줄까지 현재 명령이 계속된다는 뜻**이다.

좀 더 정확하게 말하면 바로 뒤의 newline을 escape해서

그 줄바꿈을 명령 구분으로 취급하지 않게 만든다.

그리고 이 `\` 를 작성하며 나오는 `>` 또한 오류가 아니다.

터미널에서:

```
(transformer) ... % python train.py \
>
```

처럼 `>` 가 나온다면, 이건 zsh가:

명령이 아직 끝나지 않았으니 다음 줄을 입력하세요.

라고 기다리고 있는 것이다.

`>` 를 직접 입력하는 것은 아니다.

Shell이 표시해주는 continuation prompt다.

Python REPL의 »>와 혼동하면 안된다.

1.3.81 지금까지 배운 네 가지를 정리하고 추가 내용

지금까지 배운 각각은:

`type`
→ 어떤 `Python` 타입으로 변환할까?

`default`
→ 사용자가 입력하지 않으면 무엇을 사용할까?

`required`
→ 반드시 입력해야 하는가?

`help`
→ `--help`에서 어떻게 설명할까?

이다.

그런데 ML/DL 프로그램을 만들다 보면 이것만으로 부족하다.

특히 자주 만날 것은

```
action
choices
nargs
짧은 option alias
dest
positional argument
```

정도다.

1.3.82 action - 값을 받지 않는 스위치

ML에서 이런 옵션을 만들고 싶다고 하자.

```
python train.py --use-augmentation
```

뜻은:

```
augmentation을 사용한다.
```

여기서는 굳이 `--use-augmentation True`라고 할 필요가 없다.

`--use-augmentation`이 **존재하는 것 자체를 True**로 만들 수 있다.

```
parser.add_argument(
    "--use-augmentation",
    action="store_true"
)
```

이제:

```
python train.py
```

라면:

```
args.use_augmentation
```

은 `False`다.

반대로:

```
python train.py --use-augmentation
```

하면:

```
args.use-augmentation
```

은 `True`가 된다.

이런 걸 흔히 flag라고 생각하면 된다.

GPU 관련 설정, logging, augmentation, debug mode 같은 on/off 설정에 잘 어울린다.

1.3.83 ML에서는 `choices`를 어디에 쓸까?

예를 들어:

```
parser.add_argument(
    "--optimizer",
    choices=["adamw", "adam", "sgd"],
    default="adamw"
)

parser.add_argument(
    "--scheduler",
    choices=["linear", "cosine", "constant"],
    default="linear"
)

parser.add_argument(
    "--device",
    choices=["cpu", "cuda", "mps"],
    default="cpu"
)
```

같은 식이다.

그러면 CLI 자체가 프로그램이 허용하는 설정을 설명한다.

1.3.84 nargs - argument 하나에 여러 값 받기

이번에는 hidden layer 크기를 받고 싶다고 하자.

예를 들어:

$$512 \rightarrow 256 \rightarrow 128$$

사용자가:

```
python train.py --hidden-sizes 512 256 128
```

라고 입력하게 하고 싶다.

그때:

```
parser.add_argument(
    "--hidden-sizes",
    type=int,
    nargs="+"
)
```

라고 할 수 있다.

그러면:

```
args.hidden_sizes
```

는:

```
[512, 256, 128]
```

이 된다.

즉 하나의 option에 여러 값을 전달했다.

1.3.85 nargs="+"의 +는 무슨 뜻인가?

여기서는:

하나 이상

이라는 뜻이다.

그래서:

```
--hidden-sizes 512
```

도 가능하고:

```
--hidden-szie 512 256 128
```

도 가능하다.

하지만 값 없이:

```
--hidden-sizes
```

만 쓰면 안된다.

1.3.86 짧은 option이름도 같이 만들 수 있다

예를 들어:

```
parser.add_argument(
    "-e",
    "--epochs",
    type=int,
    default=10
)
```

그러면 둘 다 가능하다.

```
python train.py --epochs 20
python train.py -e 20
```

둘 다:

```
args.epochs == 20
```

이다.

짧은 옵션은 자주 사용하는 CLI에서 편리하다.

1.3.87 `dest` - 최종 Python attribute 이름을 직접 정하기

우리가 사용했던:

```
parser.add_argument("--learning-rate")
```

는 자동으로:

```
args.learning_rate
```

가 된다고 했다.

이 최종 목적지 이름을 `argparse`에서는 **destination**, 줄여서 `dest`라고 한다.

직접 지정할 수도 있다.

```
parser.add_argument(
    "--learning_rate",
    dest="lr,
    type=float
)
```

그러면:

```
python train.py --learning-rate 0.001
```

라고 입력했는데 Python에서는:

```
args.lr
```

로 접근한다.

즉:

```
--learning-rate
├── dest="lr"
└── args.lr
```

이다.

1.3.88 dest는 왜 존재할까?

CLI에서 사람이 읽기 좋은 이름과 Python 내부에서 쓰기 좋은 이름을 분리할 수 있기 때문이다.

예를 들어 터미널에서는

```
--learning-rate
```

가 명확하다.

하지만 학습 코드에서는:

```
lr
```

라는 이름을 사용하고 싶을 수도 있다.

그러면 위와 같은 코드로 변경이 가능하다.

다만 특별한 이유가 없다면 자동 변환되는 기본 동작을 그대로 써도 충분하다.

1.3.89 Positional argument도 만들 수 있다

지금까지 전부:

```
--epochs
--batch-size
--learning-rate
```

처럼 –가 붙었다.

그런데:

```
parser.add_argument("data")
```

처럼 –없이 정의하면 positional argument가 된다.

그러면 실행할 때:

```
python train.py data/train.csv
```

라고 한다.

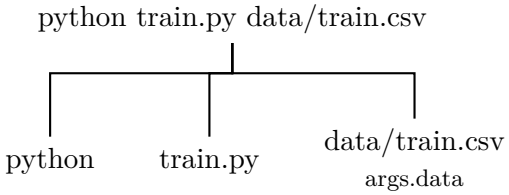
Python에서는:

```
args.data
```

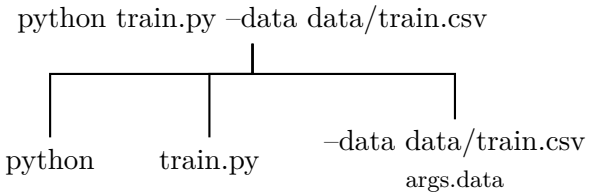
로 사용한다.

즉:

positional argument



optional argument



이다.

1.3.90 무엇을 positional로 만들고 무엇을 optional로 만들까?

정답이 하나로 고정된 것은 아니지만 프로그램 설계의 문제다.

예를 들어 프로그램이:

CSV 하나를 반드시 분석한다.

라는 도구라면:

```
python analyze.py data.csv
```

처럼 파일을 positional로 두는 것도 자연스럽다.

반면 ML 학습 프로그램처럼 설정이 많다면:

```
python train.py \
    --data data/train.csv \
    --epochs 20 \
    --batch-size 64
```

처럼 명시적인 optional으로 두면 읽기 쉽다.

1.3.91 실제 ML CLI를 한번 읽어보자

이제 꽤 현실적인 코드를 보자.

```
import argparse
from pathlib import Path

parser = argparse.ArgumentParser(
    description="Train a classification model"
)

parser.add_argument(
    "--data",
    type=Path,
    required=True,
    help="Path to training dataset"
)

parser.add_argument(
    "-e",
    "--epochs",
    type=int,
    default=10
)

parser.add_argument(
    "--optimizer",
    choices=["adam", "adamw", "sgd"],
    default="adamw"
)

parser.add_argument(
    "--hidden-sizes",
    type=int,
    nargs="+"
)

parser.add_argument(
```

```
    "--use-augmentation",
    action="store_true"
)
```

처음 argparse를 봤을 때는 이 코드가 꽤 낯설었지만

지금은 위에서부터 읽을 수 있다.

```
parser.add_argument(
    "--data",
    type=Path,
    required=True
)
```

data 경로는 반드시 받아라. 그리고 Path 객체로 변환해라.

```
parser.add_argument(
    "-e",
    "--epochs",
    type=int,
    default=10
)
```

-e 또는 --epochs 로 받을 수 있고 정수이며 안 주면 10.

```
parser.add_argument(
    "--optimizer",
    choices=["adam", "adamw", "sgd"],
    default="adamw"
)
```

optimizer는 셋 중 하나만 허용하고 기본값은 adamw.

```
parser.add_argument(
    "--hidden-sizes",
    type=int,
    nargs="+"
)
```

정수를 하나 이상 받아서 여러 값으로 저장.

```
parser.add_argument(
    "--use-augmentation",
    action="store_true"
)
```

option이 등장하면 True가 되는 스위치.

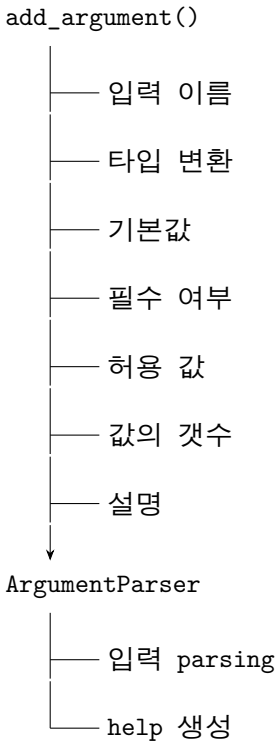
1.3.92 --help의 가치도 이제 커진다

이렇게 잘 정의해놓으면:

```
python train.py --help
```

만 입력해도 argparse가 프로그램 사용법을 만들어준다.

즉 add_argument()에 우리가 입력한 정보는 단순히 값을 읽는 데만 사용되는 게 아니다.



그래서 CLI 자체가 하나의 **입력 인터페이스**가 된다.

바꾸고 싶은 것	CLI 사용 예시	argparse에서 필요한 것
학습 횟수	`--epochs 20`	`type=int`
batch 크기	`--batch-size 64`	`type=int`
learning rate	`--learning-rate 0.001`	`type=float`
데이터 위치	`--data train.csv`	`type=Path`
모델 저장 위치	`--output model.pt`	`type=Path`
optimizer 종류	`--optimizer adamw`	`choices=...`
augmentation 켜기	`--use-augmentation`	`action="store_true"`
device 선택	`--device mps`	`choices=...`
worker 수	`--num-workers 4`	`type=int`
hidden layer 여러 개	`--hidden-sizes 512 256`	`nargs="+", type=int`

1.4 import, module, package, sys.path

우리는 당연하게 아래와 같은 것을 사용한다.

```
import argparse
from pathlib import Path
import pandas as pd
import torch
```

처음에는 그냥:

라이브러리를 가져온다.

정도로 배운다.

그런데 이제는 그 설명을 버리고 좀 더 정확하게 볼 때이다.

예를 들어:

```
import pandas
```

라고 하면 Python은 대략적으로

"pandas라는 이름의 module/package가 필요하네."



하는 과정이 있다.

여기서 우리가 앞에서 배운 **파일 경로**와 드디어 다시 만난다.

1.4.1 sys.path를 먼저 보자

Python에서:

```
import sys

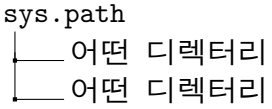
print(sys.path)
```

를 해보면 대략 이런 리스트가 나온다.

```
[
'/Users/user/Desktop/my_project/src',
'/Users/user/anaconda3/envs/transformer/lib/python310.zip',
'/Users/user/anaconda3/envs/transformer/lib/python3.10',
'/Users/user/anaconda3/envs/transformer/lib/python3.10/site-packages'
]
```

중요한 것은 정확한 문자열을 외우는 게 아니다.

sys.path 자체가 리스트다.



```
└── 어떤 디렉터리
    └── site-packages
```

Python이:

```
import pandas
```

를 만나면 이런 **탐색 위치들을 이용해서 pandas를 찾는다.**

그래서 아주 중요한 구분이 생긴다.

```
Path.cwd()
    ↓
"현재 어디에서 작업하고 있는가?"

sys.path()
    ↓
"import할 module/package를 어디에서 찾을 것인가?"
```

둘 다 경로가 등장하니까 처음에는 굉장히 헷갈리지만 **질문 자체가 다르다.**

1.4.2 그리고 site-packages가 등장한다

Conda 환경에서:

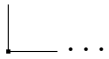
```
pip install pandas
```

했다고 해보자.

패키지는 보통 현재 Python 환경의 site-packages쪽에 설치된다.

개념적으로:

```
transformer/
└── lib/
    └── python 3.10/
        └── site-packages/
            ├── pandas/
            ├── numpy/
            └── transformers/
```



그리고 그 위치가 Python의 import 탐색 대상에 있으니까:

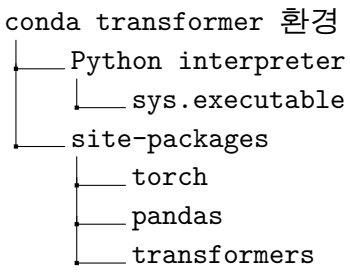
```
import pandas
```

가 가능하다.

전에 확인했던:

```
sys.executable
```

과 연결하면 상당히 중요한 그림이 완성된다.



그래서 VS Code/Jupyter에서 흔히 발생하는:

> 분명히 `pip install` 했는데 `ModuleNotFoundError`가 나는데?

라는 문제도 결국 상당수가:

> 설치한 Python 환경 $\neq$ 현재 코드를 실행하는 Python 환경

문제다.

명령어	의미
`Path.cwd()`	내가 지금 어디서 작업하는가?
`sys.executable`	어떤 Python이 나를 실행하고 있는가?
`sys.path`	그 Python이 import 대상을 어디서 찾는가?
`site-packages`	설치한 외부 package들이 주로 존재하는 곳

이제 직접만든 `util.py`를 Python이 어떻게 찾아서 import하는지 볼 것이다.

여기서부터:

$$\texttt{module} \rightarrow \texttt{package} \rightarrow \texttt{sys.path} \rightarrow \texttt{ModuleNotFoundError}$$

가 한 번에 연결된다.

1.4.3 먼저 module이 뭔가?

아주 단순하게 시작하면 **Python 파일 하나가 하나의 module이 될 수 있다.**

프로젝트가:

```
my_project/
├── train.py
└── utils.py
```

이렇게 있다고 하자.

`utils.py`:

```python
def add(a, b):
    return a + b
```

`train.py`:

```python
import utils

results = utils.add(10,20)
print(result)
```

여기서:

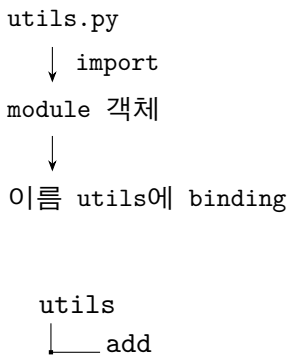
```
import utils
```

는 대충:

utils라는 module을 찾아서 사용할 수 있게 해줘.

라는 뜻이다.

Python은 utils.py를 발견하면 그 파일을 module 객체로 만들어 다룬다.



그래서:

```
utils.add(10, 20)
```

처럼 접근할 수 있다.

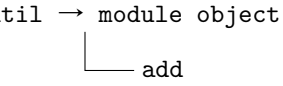
이것도:

name → object binding

과 똑같다.

```
import utils
```

이름 객체



1.4.4 `from utils import add`는 뭐가 다를까?

이번에는:

```
from utils import add

result = add(10, 20)
```

라고 해보자.

개념적으로는 최종 namespace가 다르다.

import utils

현재 namespace

utils → module
        └── add

따라서:

```
utils.add()
```

from utils import add

현재 namespace

add → function object

따라서:

```
add()
```

이렇게 사용한다.

여기서도 완전히 새로운 원리가 나온 게 아니라 **객체와 이름의 binding**이 나온 것이다.

1.4.5 그런데 Python은 utils.py를 어떻게 찾았나?

이것이 이번 파트의 핵심이다.

```
import utils
```

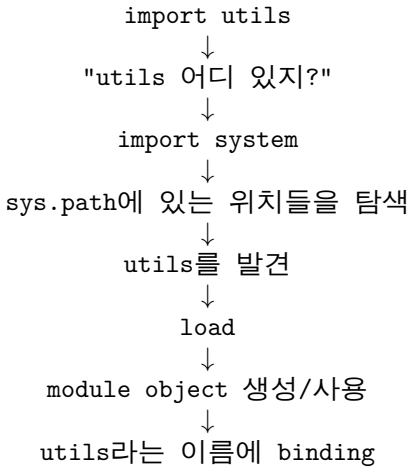
를 실행했다고 해서 Python이 컴퓨터 전체 SSD를 뒤지는 게 아니다.

Python에는 **import를 찾을 위치들**이 있고, 그게 앞에서 본:

```
sys.path
```

이다.

개념적으로:



여기까지가 첫 번째 큰 그림이다.

1.4.6 이제 폴더가 하나 생기면 문제가 재미있어진다

프로젝트를:

```
my_project/
├── train.py
└── src/
```

```
└── utils.py
```

이렇게 바꿔보자.

이제 `train.py`에서:

```
import utils
```

하면 `src/utils.py`를 찾을까?

아니다.

`src/`가 존재한다고 해서

Python이 자동으로 모든 하위 디렉터리를 import 탐색 대상으로 삼는 게 아니다.

이게 굉장히 중요하다.

Python이 `my_project`를 탐색한다고 해서:

`my_project`
↓
모든 하위 폴더를 재귀적으로 검색
↓
`src/utils.py` 발견

하는 방식이 **아니다.**

그래서 import를 이해할 때:

파일이 프로젝트 어딘가에 존재한다.

와:

Python이 그 파일을 import대상으로 찾을 수 있다.

는 전혀 다른 말이다.

1.4.7 그래서 package라는 개념이 필요해진다

우리가 원하는 것은:

```
from src import utils
```

또는:

```
from src.utils import add
```

같은 구조라면 src를 **package**구조로 다루게 된다.

전통적으로 아주 흔하게 보는 구조가:

```
my_project/
├── train.py
└── src/
    ├── __init__.py
    └── utils.py
```

이다.

그러면 개념적으로:

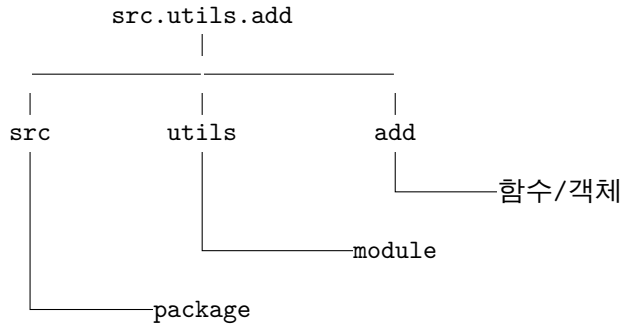
```
src
├── __init__.py
└── utils.py
    └── add()
```

라는 package/module계층이 만들어진다.

그래서:

```
from src.utils import add
```

를 보면 이제 이렇게 읽으면 된다.



사용:

```
from src.utils import add

print(add(10, 20))
```

1.4.8 __init__.py는 뭔데?

이 파일을 처음 보면 이상하다.

__init__.py

클래스에서 배운:

```
def __init__(self):
```

와 이름이 같아서 더 헷갈릴 수 있지만

지금은:

package의 __init__.py

이다.

역사적으로 Python에서는:

```
src/
├── __init__.py
└── utils.py
```

처럼:

```
__init__.py
```

가 있으면 이 디렉터리를 Python package로 취급하는 전통적인 구조를 사용했다.

현대 Python에는 이 것이 없어도 가능한 **namespace package**도 있어서,

```
package라면 무조건 __init__.py가 있어야 한다.
```

라고 외우면 정확하지 않다.

하지만 일반적인 Python 프로젝트 구조를 배우는 지금 단계에서는:

```
디렉터리를 일반적인 Python package로 구성
                ↓
__init__.py를 둔다.
```

정도로 먼저 이해하면 충분하다.

1.4.9 module과 package를 이렇게 구분하면 된다

지금 단계에서는:

```
module
  ↓
Python 코드의 한 단위
대표적으로 .py 파일

utils.py
train.py
model.py
```

반면:

```
package
  ↓
여러 module 등을 계층적으로 조직하는 구조
```

```
src/
├── __init__.py
├── utils.py
├── model.py
└── dataset.py
```

그래서 ML 프로젝트도 점점 커지면:

```
my_project/
├── train.py
└── src/
    ├── __init__.py
    ├── data.py
    ├── model.py
    └── utils.py
```

같아지고,

```
from src.data import load_data
from src.model import MyModel
from src.utils import save_model
```

같은 코드가 나온다.

1.4.10 여기서 ModuleNotFoundError의 의미가 보이기 시작한다

예를 들어:

```
from src.model import MyModel
```

했는데:

```
ModuleNotFoundError
```

가 발생했다면 무조건:

model.py파일이 없다.

라는 뜻이 아니다.

더 정확하게는:

> 현재 import system이 주어진 이름을 해석해서
>
> 해당 module/package를 찾지 못했다.

이다.

따라서 문제를 볼 때 이제 세 가지를 분리해야 한다.

실제 파일은 어디 있는가?
↓
Path / filesystem 문제

현재 어떤 Python으로 실행 중인가?
↓
sys.executable / Conda 문제

그 Python은 import 대상을 어디서 찾는가?
↓
sys.path / package 구조 문제

이 구분이 중요하다.

1.4.11 그리고 여기서 sys.path.append()가 왜 인터넷에 자주 나오는지도 보인다

검색하다보면 이런 코드를 엄청 많이 보게 된다.

```
import sys

sys.path.append("...")
```

이게 하는 일은 말 그대로:

```
[
    A,
    B,
    C
]
```

```
append(D)

[
    A,
    B,
    C,
    D
]
```

즉:

Python아, import할 때 이 디렉터리도 찾아봐.

라고 import 탐색 위치를 런타임에 추가하는 것이다.

그래서 import가 안 될 때:

```
sys.path.append("...")
```

같은 것을 넣으면 갑자기 되는 경우가 있다.

그런데 이건 때때로 **프로젝트 구조나 실행 방법의 문제를 임시로 덮어버리는 것**일 수 있다.

그래서 지금 단계에서는:

ModuleNotFoundError 가 발생했으니 sys.path.append부터 넣자

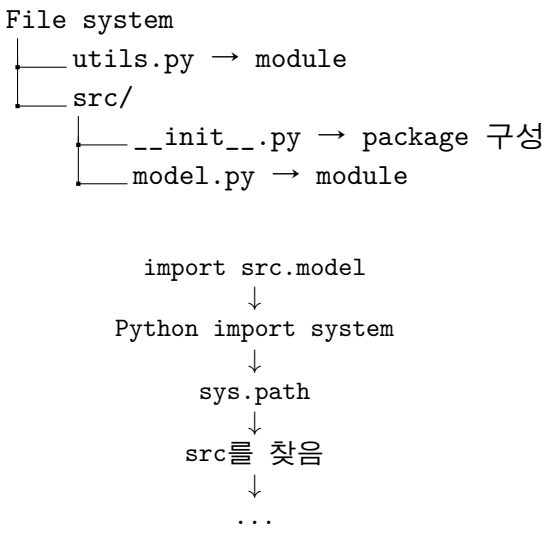
가 아니라

ModuleNotFoundError
↓
파일은 어디 있지?
↓
어떤 Python으로 실행 중이지?
↓
어디서 어떤 방식으로 실행했지?
↓
package 구조는 어떻게 되어 있지?
↓
sys.path는 어떻게 구성되어 있지?

라고 생각하는 습관을 들이는 것이 좋다.

1.4.12 여기까지의 핵심

새로운 내용을 기존 개념과 합치면 이렇게 된다.



이것이 **Filesystem + Path + name/object/binding + 실행환경**이 하나로 합쳐지는 지점이다.

다음 단계에서는 가장 많이 헷갈리는

```
python src/train.py
```

```
python -m src.train
```

위 두 가지가 서로 다르고,

왜 하나에서는 import가 되지만 다른 하나에서는 깨질 수 있는지

를 실제 프로젝트 구조를 놓고 볼 것이다.

1.4.13 같은 train.py인데 실행 방법이 두 가지다

프로젝트를 이렇게 두자:

```
my_project/
└── src/
```

```
    ├── __init__.py
    ├── train.py
    └── utils.py
└── data/
    └── train.csv
```

그리고 `train.py`에서 같은 `package`의 `utils.py`를 사용하고 싶다고 하자.

```
from src.utils import add
```

터미널의 CWD는:

```
my_project/
```

라고 하자.

여기서 두 가지 실행법이 있다.

```
python src/train.py
```

그리고:

```
python -m src.train
```

둘 다 `train.py`의 코드를 실행하는 것 처럼 보이는가?

그렇지만 Python에게 전달하는 **의미가 다르다**.

1.4.14 `python src/train.py`

이건 우리가 지금까지 많이 썼던 방식이다.

```
python src/train.py
```

Python에게 말하는 건:

`src/train.py`라는 **파일을 직접 실행해**.

라는 의미다.

즉 출발점이 **파일 경로**이다.

```
python src/train.py
            └── 파일 경로
```

이때 `train.py`는 실행할 최상위 스크립트로 취급된다.

그리고 중요한 변화가 하나 생긴다.

실행되는 스크립트가 있는 디렉터리인:

```
my_project/src/
```

가 import 탐색에 중요한 위치로 들어오게 된다.

그래서 상황에 따라 package import가 예상과 다르게 동작할 수 있다.

1.4.15 `python -m src.train`

이번엔 완전히 관점이 다르다.

```
python -m src.train
```

여기에는:

```
src/train.py
```

라는 파일 경로를 주지 않았다.

대신:

```
src.train
```

이라는 **module 이름**을 주었다.

`-m`은 간단히 말하면:

> 이 이름의 `module`을 찾아서 실행해.

라는 의미다.

따라서 Python의 사고 과정은:

python -m src.train

현재 import 환경에서

↓

src라는 package를 찾고

↓

그 안의 train module을 찾고

↓

그 module을 실행

이렇게 된다.

즉:

python src/train.py

↓

파일 경로를 직접 실행

python -m src.train

↓

module/package 이름으로 찾아서 실행

이 차이가 핵심이다.

1.4.16 왜 /를 안 쓰고 .을 쓰는가?

이제 이유가 보일 것이다.

```
python src/train.py
```

에서는 **Filesystem Path**니까:

```
src/train.py
```

라고 쓴다.

반면:

```
python -m src.train
```

에서는 **Python module**경로니까:

```
src.train
```

이라고 쓴다.

따라서 둘을 구별해야 한다.

```
Filesystem
──────────────────────────────────────────────
src/train.py
src/utils.py
```

```
Python module namespace
──────────────────────────────────────────────
src.train
src.utils
```

이것은 상당히 중요한 구분이다.

1.4.17 이제 package 내부 import가 나온다

`train.py`와 `utils.py`가 같은 `src` package 안에 있다.

```
src/
├── __init__.py
├── train.py
└── utils.py
```

train.py에서 이렇게 쓸 수도 있다.

```
from src.utils import add
```

이걸 **absolut import**라고 생각하면 된다.

전체 package 기준으로:

```
src
└── utils
    └── add
```

를 지정했기 때문이다.

그런데 이렇게도 쓸 수 있다.

```
from .utils import add
```

여기서 . 은 Filesystem의:

```
./
```

와 똑같은 문자열 문법이라고 생각하면 **안 된다.**

여기서는 **현재 package**라는 의미다.

즉 train.py가 src package 안에 있으므로:

```
from .utils import add
```

는 개념적으로:

현재 package

↓

src

↓

utils

↓

add

라는 뜻이다.

이를 **relative import**라고 한다.

1.4.18 그런데 파일을 직접 실행하면 문제가 생길 수 있다

`train.py`:

```
from .tuils import add

print(add(10, 20))
```

라고 해보자.

그리고:

```
ptyhon src/trian.py
```

로 실행하면 `relative import`관련 오류를 만날 수 있다.

왜냐하면 우리는 Python에게:

`src package 안의 train module`을 실행해.

라고 한 것이 아니다.

그냥:

`이 train.py`를 직접 실행해.

라고 했기 때문이다.

즉 Python입장에서는 `train.py`가 package 안에서

어떤 위치를 갖는 module인지에 대한 문맥이 깨질 수 있다.

반대로:

```
python -m src.train
```

이라고 하면 처음부터

```
src package
    ↓
train module
```

이라는 package 문맥으로 찾았다.

따라서:

```
from .utils import add
```

의 . 도:

```
현재 package = src
```

라고 해석할 수 있다.

이게 -m이 중요한 이유 중 하나이다.

1.4.19 __name__까지 여기서 연결된다

Python 파일에서 많이 보는:

```
if __name__ == "__main__":
    main()
```

도 사실 이 주제이다.

Python이 어떤 module을 **실행의 시작점**으로 실행하면

그 모듈의:

```
__name__
```

을:

"__main__"

으로 설정한다.

예를 들어:

```
def main():
    print("training.")

if __name__ == "__main__":
    main()
```

직접 실행하면:

```
python train.py
```

__name__=="__main__" 이므로 main()이 실행된다.

반면 다른 코드에서:

```
import train
```

하면 보통:

```
__name__=="train"
```

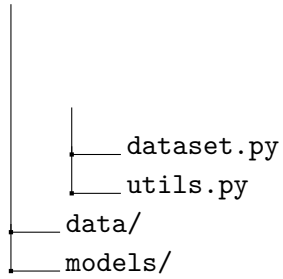
이므로 저 조건은 False가 된다.

그래서 **import 했을 때 학습이 갑자기 시작되는 것을 막고, 실행 진입점만 분리**할 수 있다.

1.4.20 이게 ML 코드에서 왜 중요할까?

프로젝트가 커지면 보통:

```
my_project/
└── src/
    ├── __init__.py
    ├── train.py
    └── model.py
```



이런 식이 된다.

train.py 혼자 모든 걸 하는 게 아니라:

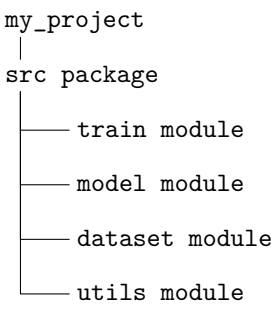
```
from src.model import MyModel
from src.dataset import load_dataset
from src.utils import save_model
```

처럼 여러 module이 연결된다.

그러면 단순히:

파일 A 실행

만 생각하는 것보다:



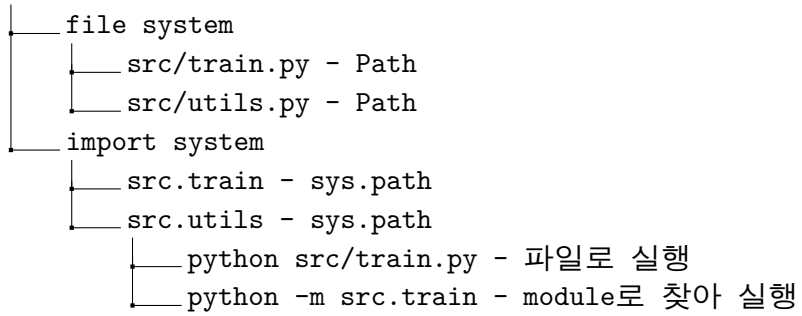
이라는 package 구조로 생각하는 게 중요해진다.

그래서 프로젝트 루트에서:

```
python -m src.train
```

처럼 실행하는 방식이 자연스러워 지는 경우가 있다.

Python 프로젝트



여기까지 이해되면 **import의 뼈대**는 거의 잡힌 것이다.

다음 내용은 여기서 자연스럽게:

__name__

__main__

import 할 때 Python 파일의 코드가 실제로 실행된다는 사실

module caching인 sys.modules

로 넘어가는 것이 좋다.

이것을 배우면:

> 왜 import는 단순히 함수/클래스 정의를 복사해오는 게 아닌가?

까지 이해할 수 있다.

1.4.21 import는 정의를 복사해오는 게 아니다

예를 들어 utils.py가 있다고 하자.

```python
print("utils.py 실행됨")

def add(a, b):
    return a + b
```

그리고 train.py:

```python
import utils
```

```
print(utils.add(10,20))
```

실행하면:

```
utils.py 실행됨
30
```

이게 중요한 이유는,

```
import utils
```

가 단순히 add() 라는 함수 정의만 가져오는 게 아니라는 것이다.

Python은 처음 import 할 때 개념적으로:

import utils

↓

utils라는 모듈을 찾음

↓

utils.py의 코드를 실행

↓

그 실행으로 module namespace 구성

↓

module 객체를 만듦

↓

utils라는 이름에 그 module 객체를 binding

한다고 보면 된다.

1.4.22 왜 파일을 실행해야 할까?

utils.py가:

```python
x = 10

def add(a, b):
    return a + b

class Calculator:
    pass
```

라고 해보자.

Python이 이 파일을 실행하면서 namespace가 만들어진다고 생각하면 된다.

```
utils module의 namespace

x → 10

add → function object

Calculator → class object
```

그리고:

```python
import utils
```

하는 파이썬 파일에서는:

```
현재 namespace

utils → module object
        ├── x
        ├── add
        └── Calculator
```

가 되는 것이다.

그래서:

```python
utils.x
utils.add()
utils.Calculator()
```

처럼 접근할 수 있었던 것이다.

이건 예전에 배운 **name → object binding**이 module 수준으로 확장된 것이다.

1.4.23 그러면 import 할 때 위험한 코드도 실행된다

예를 들어 `train.py`를 이렇게 만들면:

```
def train():
    print("모델 학습 시작")

train()
```

그리고 다른 파일에서:

```
import train
```

만 해도:

```
모델 학습 시작
```

이 출력된다.

왜냐하면 Python은 `train.py`를 import 하기 위해

그 모듈의 코드를 실행했기 때문이다.

바로 여기서 아까 배운 것이 다시 등장한다.

```
def main():
    print("모델 학습 시작")

if __name__ == "__main__":
    main()
```

이제 다른 파일에서:

```
import train
```

하면 `train.py` 자체는 import 과정에서 실행된다.

그래서:

```
def main():
```

도 실행되어 **함수 객체가 만들어지고 main이라는 이름에 binding**된다.

하지만:

```
if __name__ == "__main__":
```

에서는 import 된 모듈이므로:

```
__name__ == "train"

"train" == "__main__"
        ↓
      False
```

따라서:

```
main()
```

이라는 **함수 호출만 실행되지 않은 것**이다.

이 차이가 중요하다.

```
import train

↓

train.py 자체가 실행되지 않는다 (X)

train.py는 실행된다 (O)

하지만 __main__ 조건 내부의 main() 호출이 실행되지 않는다. (O)
```

1.4.24 그렇다면 매번 import할 때마다 파일을 다시 실행할까?

여기서 `sys.modules`가 등장한다.

`utils.py`:

```
print("UTILS EXECUTED")
```

train.py:

```
import utils
import utils
import utils
```

얼핏 생각하면:

```
UTILS EXECUTED
UTILS EXECUTED
UTILS EXECUTED
```

일 것 같지만 일반적으로 한 번만 출력된다.

왜냐하면 Python은 이미 import한 module을 **캐시, Cache**해두기 때문이다.

그 캐시가:

`sys.modules`

이다.

1.4.25 `sys.modules`는 dictionary다

```
import sys

print(type(sys.modules))
```

개념적으로:

```
dict
```

이고 대충 이런 구조라 생각하면 된다.

```
sys.modules
{
    "sys": <module 'sys'>,
    "os": <module 'os'>,
    "pathlib": <module 'pathlib'>,
    "utils": <module 'utils'>,
    ...
}
```

즉:

module 이름

↓

module 객체

를 저장하는 dictionary이다.

1.4.26 두 번째 import utils에서는 어떻게 되는가?

첫 번째:

```
import utils
```

에서는 대략:

import utils

↓

sys.modules에 utils 있나?

↓

없음

↓

모듈 탐색

↓

utils.py 발견

↓

코드 실행

↓

module 객체 생성/초기화

↓

sys.modules["utils"]에 등록

이제:

```
sys.modules

"utils" → utils module object
```

가 되었다고 생각하자.

두 번째:

```
import utils
```

에서는:

import utils

↓

sys.modules에 utils 있나?

↓

있음

↓

기존 module 객체 사용

그래서 일반적으로 utils.py를 처음부터 다시 실행하지 않는다.

1.4.27 그래서 같은 module 객체인지 확인할 수도 있다

```
import sys
import utils

print(utils is sys.modules["utils"])
```

결과:

```
True
```

현재 namespace
utils

↓

module object

↑

sys.modules["utils"]

두 이름/참조가 **동일한 module 객체를 가리키므로**:

```
utils is sys.modules["utils"]
```

가 `True`인 것이다.

1.4.28 이제 전체 import 과정을 조금 더 정확하게 만들 수 있다

단순하게:

import

↓

sys.path 탐색

↓

module 발견

이라고 이전에 배웠던 것이 이제 한 단계 정밀해진다.

import src.utils

↓

이미 로드되어 있는가?

↓

sys.modules 확인

↓

No

↓

import system이 module 탐색

↓

sys.path

↓

module 발견

↓

module 코드 실행

1.4.29 그리고 이것이 __name__과도 연결된다

utils.py가 import 되면서 실행될 때

Python은 module namespace 에 여러 특별한 이름도 준비한다.

개념적으로:

```
utils module namespace

__name__ → "utils"
__package__ → ...
__file__ → ".../utils.py"
add → function object
x → 10
...
```

그래서 utils.py내부 코드가 실행되는 동안에도:

```
print(__name__)
```

이 가능했던 것이다.

__name__ 이라는 마법 같은 함수가 따로 있는게 아니라

Python이 module namespace에 준비해둔 특별한 이름이라고 보면 된다.

여기까지 이해하면 다음에는

> import utils와 from utils import add가
>
> 객체/binding 관점에서 정확히 어떻게 다른지
>
> 그리고 왜 import한 변수는 원본 module에서 나중에 바뀌었는데
>
> 내쪽에서는 안바뀌지? 정도까지 들어가면 된다.

1.4.30 import ... / from ... import ... → 객체와 바인딩 관점에서 분리

먼저 모듈 하나를 만들자.

config.py:

```
learning_rate = 0.001

def show_lr():
    print(learning_rate)
```

다른 파일 train.py에서:

```
import config
```

한다고 하자.

앞에서 배운 과정에 따라 config.py가 처음 import 되면

실행되고 module namespace 가 만들어진다.

```
config module object
└── namespace
```

```
├── learning_rate → 0.001
└── show_lr → function object
```

그리고 train.py쪽에서는:

```
trian.py namespace

config → config module object
```

즉 import config가 현재 namespace 에 만드는 핵심 이름은 config이다.

그래서:

```
config.learning_rate
config.show_lr()
```

라고 접근한다.

1.4.31 from config import learning_rate는 다르다

이번에는:

```
from config import learning_rate
```

라고 해보자.

config module을 찾고 로드하는 과정 자체는 여전히 필요하다.

하지만 마지막에 현재 namespace 로 가져오는 이름이 다르다.

```
config module namespace
learning_rate → 0.001
   ↓ from config import learning_rate
train.py namespace
learning_rate → 같은 0.001 객체
```

즉 현재 namespace 에:

```
config
```

라는 이름을 만드는 게 아니라:

```
learning_rate
```

라는 이름을 직접 binding 하는 것이다.

그래서:

```
print(learning_rate)
```

라고 바로 쓸 수 있다.

1.4.32 여기서 중요한 실험

config.py:

```
learning_rate = 0.001
```

train.py:

```
import config

print(config.learning_rate)

config.learning_rate = 0.01

print(config.learning_rate)
```

결과:

```
0.001
0.01
```

이건 어렵지 않다.

config

↓

module object

↓

learning_rate → 0.001

에서 module namespace 의 learning_rate binding을:

learning_rate → 0.01

로 바꾼 것이기 때문이다.

1.4.33 그런데 이렇게 하면 재미있는 일이 생긴다

```
from config import learning_rate

print(learning_rate)

learning_rate = 0.01

print(learning_rate)
```

당연히:

```
0.001
0.01
```

이다.

그런데 이것은 **config.learning_rate를 0.01로 바꾼 것이 아니다.**

처음에는:

config namespace
learning_rate

↓

0.001

↑

train namespace
learning_rate

두 namespace 의 서로 다른 이름이 같은 객체를 가리키고 있었던 것이다.

그 후 `train.py`에서:

```
learning_rate = 0.01
```

하면 **train namespace의 binding만 바뀐다.**

```
config namespace
learning_rate → 0.001

train namespace
learning_rate → 0.01
```

1.4.34 반대로 `import config`는 module 을 통해 계속 접근한다

```
import config

config.learning_rate = 0.01
```

에서는 구조가:

```
train namespace

config
 |
config module object
 |
 └── learning_rate → 0.01
```

이다.

별도의 `learning_rate`이름을 `train` namespace에 만든 게 아니라

config module namespace에 들어가서 그 binding을 변경한 것이다.

그래서 이 둘은 꽤 중요한 차이가 있다.

```
import config
config.learning_rate
```

vs.

```
from config import learning_rate
learning_rate
```

첫 번째는 매번 config module의 attribute를 보고 있고,

두 번째는 import 시점에 해당 객체를 **현재 namespace의 별도 이름에 binding한 것**이다.

1.4.35 함수도 완전히 똑같다

utils.py:

```
def add(a,b):
    return a + b
```

방법 A

```
import utils

utils.add(10, 20)
```

구조:

현재 namespace

```
utils → module object
        └── add → function object
```

방법 B

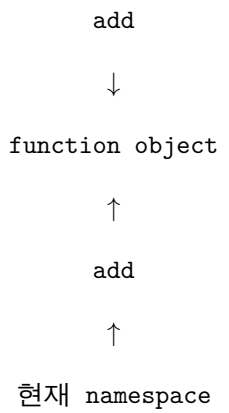
```
from utils import add

add(10, 20)
```

구조:

```
                    utils module

                         ↓
```



즉 함수 코드를 복사해온 것도 아니다.

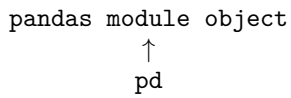
같은 function object를 현재 namespace의 add라는 이름에서도 가리키게 만든 것이다.

1.4.36 as도 이제 별 것 아니다

```
import pandas as pd
```

이것도 특별한 마법처럼 볼 필요가 없다.

대략:



현재 namespace에서 그 module 객체를 pd라는 이름으로 사용하겠다는 것이다.

그래서:

```
pd.read_csv(...)
```

가 되는 것이다.

마찬가지로:

```
from pathlib import Path as P
```

라면 현재 namespace에:

P → Path class object

가 생기는 셈이다.

1.4.37 여기까지 하나로 정리

```
import utils
현재 namespace
 |
utils → module
        |
        |—— add
        |
        |—— ...
```

```
from utils import add
현재 namespace
 |
add → function object
 |
utils module
 |
 |—— add → function object(same)
```

결국 import 문법도:

객체를 찾고 → 어떤 namespace의 어떤 이름에 binding 할 것인가

라는 원리 위에 있다.

다음에는 여기서 한단계 더 나아가서,

mutable 객체를 from ... import ... 했을 때는 왜 조금 다르게 보이는지,

그리고 **import * 가 왜 좋지 않은지** 살펴본다.

1.4.38 mutable 객체를 import 했을 때의 동작은?

이번에는 config에 리스트를 넣어보자.

config.py:

```
hidden_sizes = [512, 256, 128]
```

train.py:

```
from config import hidden_sizes

print(hidden_sizes)
```

처음 import 직후 구조는:

config namespace

hidden_sizes

↓

[512, 256, 128]

↑

hidden_sizes

train namespace

즉 리스트를 복사한 것이 아니다.

두 namespace의 hidden_size라는 이름이 **동일한 list 객체를 가리킨다.**

따라서:

```
import config
from config import hidden_sizes

print(config.hidden_sizes is hidden_sizes)
```

는:

```
True
```

가 된다.

1.4.39 여기서 객체 자체를 변경해보자

```
import config
from config import hidden_sizes

hidden_sizes.append(64)

print(hidden_sizes)
print(config.hidden_sizes)
```

결과:

```
[512, 256, 128, 64]
[512, 256, 128, 64]
```

왜?

append()는 hidden_sizes라는 이름의 binding을 바꾼 것이 아니다.

그 이름이 가리키고 있는 list 객체 자체를 변경한 것이다.

그래서 양쪽에서 변경된 상태가 보인다.

1.4.40 그런데 재대입하면 완전히 다르다

이번에는:

```
import config
from config import hidden_sizes

hidden_sizes = [1024, 512]
print(hidden_sizes)
print(config.hidden_sizes)
```

결과:

```
[1024, 512]
[512, 256, 128]
```

왜냐하면:

```
hidden_sizes = [1024, 512]
```

는 기존 list를 수정한 게 아니라 **train namespace의 `hidden_sizes`라는 이름을 새로운 list 객체에 재바인딩**했기 때문이다.

```
config namespace
hidden_sizes → [512, 256, 128]


train namespace
hidden_sizes → [1024, 512]
```

이건 mutable/immutable 의 내용과 같은 원리이다.

1.4.41 따라서 핵심은 import가 아니다

이게 중요하다.

이 현상을 보고:

`from import`는 `mutable`일 때 특별하게 동작한다.

라고 **이해하면 안된다**.

특별한 import 규칙이 있는 것이 아니다.

핵심은:

객체 자체 변경(mutaion)

vs.

이름의 재바인딩(rebinding)

이다.

예를 들어:

```
hidden_sizes.append(64)
```

는:

이름

↓

기존 list 객체

↓

객체 내부 상태 변경

반면:

```
hidden_sizes = [1024, 512]
```

는:

이름 → 기존 list (X)

이름 → 새로운 list (O)

이다.

1.4.42 dictionary도 똑같다

config.py:

```
settings = {
    "epochs": 10,
    "batch_size": 32
}
```

train.py:

```
import config
from config import settings

settings["epochs"] = 20
print(settings)
print(config.settings)
```

둘 다:

```
{
    "epochs": 20,
    "batch_size": 32
}
```

가 보인다.

왜냐하면:

```
settings["epochs"]=20
```

은 dictionary 객체 자체를 변경했기 때문이다.

반대로:

```
settings = {
    "epochs": 100
}
```

하면 `train`쪽 이름만 새로운 dictionary로 재바인딩 된다.

1.4.43 여기까지 오면 import를 굉장히 단순하게 볼 수 있다

처음에는:

```
import config
from config imoprt hidden_sizes
```

가 Python만의 복잡한 규칙처럼 보이지만, 실제 객체 모델은:

1. `module`도 객체다.
2. `module`은 자신의 `namespace`를 가진다.
3. `import`는 객체를 현재 `namespace`의 이름에 `binding`한다.
4. `from ... import ...` 도 특정 객체를
   현재 `namespace`의 이름에 `binding`한다.
5. 그 이후부터는 평범한 `Python` 객체 규칙이다.

`mutation`

↓

같은 객체를 보는 모든 곳에서 관찰됨

rebinding

↓

해당 namespace의 이름만 다른 객체를 가리킴

이것이 전부이다.

1.4.44 그리고 import *

이제 이런 코드도 이해할 수 있다.

```
from config import *
```

대략적으로는 config에서 공개 대상으로 취급되는 여러 이름을

현재 namespace로 가져오는 형태다.

예를 들어:

```
# config.py

learning_rate = 0.001
batch_size =32

def load_config():
    ...
```

에서:

```
from config import *
```

를 사용하면 현재 namespace에 여러 이름이 한꺼번에 들어올 수 있다.

현재 namespace

```
learning_rate
batch_size
load_config
...
```

문제는 구묘가 커지면:

```
from config import *
from utils import *
from dataset import *
```

같은 코드에서

```
load()
```

를 봤을 때:

```
load가 어디서 들어온 것인가?
```

를 파악하기 어려워진다는 것이다.

같은 이름이 여러 module에 있으면 namespace 충돌도 생길 수 있다.

그래서 일반적인 애플리케이션 코드에서는 `import *`를 피하고:

```
import config
```

또는:

```
from config import learning_rate, batch_size
```

처럼 **어떤 이름을 가져오는지 명시적으로 표현하는 편이 좋다.**

여기까지 왔으면 **import/module/package** 학습의 핵심 부분은 거의 끝났다.

지금까지 연결한 범위는:

```
filesystem
```

↓

Path / CWD

↓

Python 실행

↓

sys.executable

↓

sys.path

↓

module 탐색

↓

module / package

↓

python file.py vs ptyhon -m package.module

↓

__name__ / __package__

↓

module namespace

↓

sys.modules / import cache

↓

import binding

↓

mutable / mutation / rebinding

1.5 Python File I/O

가장 밑바닥인 `open()`과 file object가 정확히 무엇인가부터 시작한다.

1.5.1 `open()`은 무엇을 하는가?

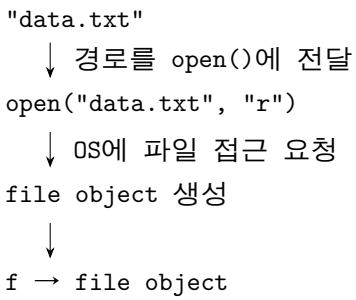
가장 기본적인 코드는:

```
f = open("data.txt", "r")
```

이다.

흔히 이것을 그냥 "`data.txt`를 연다"라고 배우는데,

객체 관점에서 보면 조금 더 정확하게 이해할 수 있다.



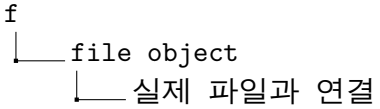
즉,

```
f = open("data.txt", "r")
```

에서 `f`가 파일 자체인 것은 아니다.

`f`는 열려 있는 파일과 상호작용하기 위한 Python의 file object를 가리키는 이름이다.

이건 지금까지 배운 객체 모델과 완전히 동일하다.



그래서 다음과 같은 메서드를 사용할 수 있다.

```
f.read()
f.readline()
f.write(...)
f.close()
```

1.5.2 "r"은 무엇인가?

두 번째 인자는 **mode**이다.

```
open("data.txt", "r")
```

여기서 "r"은 **read**.

즉:

이 파일을 읽기 위한 방식으로 열어라.

라는 뜻이다.

대표적인 mode부터 보면:

```
    "r" read   읽기
    "w" write  쓰기
    "a" append 뒤에 추가
```

예를 들어 파일 내용이:

```
epoch 1
epoch 2
```

일 때,

```
f = open("log.txt", "a")
f.write("\nepoch 3\n")
```

하면 기존 내용을 유지하면서 뒤에 추가해서:

```
epoch 1
```

```
epoch 2
epoch 3
```

가 된다.

반면:

```
f = open("log.txt", "w")
```

의 `"w"`는 **기존 파일이 있으면 내용을 비우고 새로 쓰기 시작할 수 있기 때문에 주의해야 한다.**

1.5.3 파일을 읽어보자

`data.txt`:

```
apple
banana
orange
```

Python:

```
f = open("data.txt", "r")
text = f.read()
print(text)
f.close()
```

`f.read()`는 파일에서 내용을 읽어 Python 객체로 가져온다.

여기서는 text mode이므로 대략:

```
disk
data.txt
   ↓ read
Python process
text → "apple\nbanana\norange"
```

처럼 `str` 객체를 얻게 된다.

그래서:

```
type(text)
```

는:

```
str
```

이다.

1.5.4 그런데 왜 close()가 필요한가?

이게 File I/O에서 중요한 부분이다.

```
f = open(...)
```

하면 Python은 단순히 파일 이름을 기억하는 게 아니라 운영체제의 파일 자원을 사용하게 된다.

그래서 사용이 끝났으면:

```
f.close()
```

로 닫아줘야 한다.

문제는 이런 코드다:

```
f = open("data.txt", "r")

text = f.read()

# 여기에서 에러 발생
some_function()

f.close()
```

some_function()에서 예외가 발생하면?

open

↓

read

↓

에러 발생

↓

프로그램 흐름 중단

↓

`f.close()`까지 못 감

이런 자원 관리 문제 때문에 Python에서 일반적으로

직접 `open()` → `close()`를 관리하기보다는 `with`를 사용한다.

1.5.5 `with open(...) as f`

실무에서 흔히 보는 형태가 이것이다.

```
with open("data.txt", "r") as f:
    text = f.read()

print(text)
```

핵심은:

`with` 블록 진입

↓

파일 열기

↓

`f` → `file object`

↓

`f.read()`

↓

`with` 블록 종료

↓

파일 자동 정리/close

중간에 예외가 발생하더라도 with가 자원 정리를 처리할 수 있다.

그래서 보통:

```
f = open(...)

...

f.close()
```

보다:

```
with open(...) as f:
    ...
```

를 선호한다.

1.5.6 그런데 with는 파일 전용 문법이 아니다

이 부분이 중요하다.

처음 배우면:

with → 파일 열 때 사용하는 문법

처럼 외우기 쉬운데 그렇지 않다.

with는 **context manager**라는 더 일반적인 Python 개념과 관련된다.

큰 그림은:

어떤 자원을 사용하기 전에 준비

↓

사용

↓

같은 패턴을 안전하게 다루는 문법이다.

파일이 대표적인 예일 뿐이다.

```
with open(...) as f:
    ...
```

를 개념적으로 읽으면:

> 이 context에 진입하면서 자원을 준비하고,
> 블록 안에서 사용한 뒤 context를 빠져나갈 때 정리한다.

라고 볼 수 있다.

나중에 class를 더 배우면 `__enter__`, `__exit__`과 연결된다.

1.5.7 Path와도 바로 연결된다

앞에서 배운 pathlib을 그대로 사용할 수 있다.

```
from pathlib import Path

path = Path("data/train.txt")

with path.open("r") as f:
    text = f.read()
```

즉:

```
open("data/train.txt", "r")
```

과

```
path = Path("data/train.txt")
path.open("r")
```

둘 다 파일을 여는 방식이다.

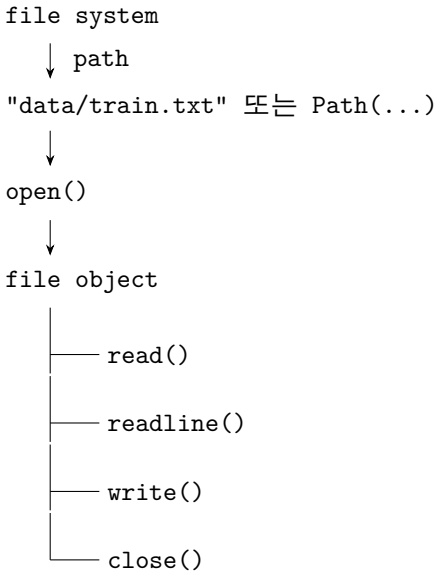
그리고 단순히 텍스트 전체를 읽는 경우에는:

```
text = path.read_text()
```

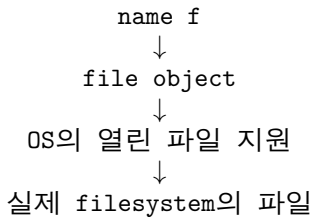
처럼 더 간단한 API도 있다.

1.5.8 여기서 첫 번째로 잡아야 할 핵심

지금은 메서드를 외우는 것보다 이 구조가 중요하다.



그리고:



이렇게 **Python 객체 세계와 실제 filesystem 사이를 연결하는 객체가 file object**이다.

다음은 `read()`, `readline()`, `readlines()`의 차이와

왜 **file object가 iterator이기도 하는지**로 넘어간다.

1.5.9 read() - 파일 전체를 한 번에 읽기

data.txt가 다음과 같다고 하자.

```
apple
banana
orange
```

```
with open("data.txt", "r") as f:
    text = f.read()

print(text)
```

text에는 하나의 str객체가 들어온다.

```
'apple\nbanana\norange'
```

구조적으로는:

```
file
  ↓ f.read()
하나의 str 객체
"apple\nbanana\norange"
```

즉, read()는 **전체 파일** → **하나의 문자열**이라고 생각하면 된다.

그래서 큰 파일에서는 문제가 생길 수 있다.

```
10GB file
  ↓ read()
10GB에 가까운 데이터를 메모리에 한 번에 올리려고 함
```

1.5.10 readline() - 한 줄씩 읽기

이번에는:

```
with open("data.txt", "r") as f:
    line1 = f.readline()
    line2 = f.readline()
```

```
print(line1)
print(line2)
```

대략:

```
line1 == "apple\n"
line2 == "banana\n"
```

이 된다.

여기서 중요한 현상이 하나 있다.

첫 번째:

```
f.readline()
```

은 apple을 읽었는데,

두 번째:

```
f.readline()
```

은 또 apple을 읽는 게 아니라 banana를 읽었다.

왜 그럴까?

1.5.11 file object에는 현재 읽고 있는 위치가 있다

file object는 내부적으로 **현재 파일의 어느 위치까지 읽었는지**를 관리한다.

개념적으로:

```
apple\nbanana\norange
└── 현재위치
```

첫 번째 readline():

```
apple\nbanana\norange
      └── 현재  위치
```

두 번째 `readline()`:

```
apple\nbanana\norange
               └── 현재위치
```

처럼 진행된다.

이 위치를 보통 **file position/file pointer**라고 부른다고 생각하자.

실제로 확인할 수도 있다.

```
with open("data.txt", "r") as f:
    print(f.tell())

    f.readline()

    print(f.tell())
```

`tell()`은 현재 위치를 알려준다.

그리고:

```
f.seek(0)
```

처럼 `seek()`을 사용하면 위치로 이동할 수도 있다.

1.5.12 그래서 `read()`도 두 번 하면?

이제 결과를 예측할 수 있다.

```
with open("data.txt", "r") as f:
    a = f.read()
    b = f.read()

print(a)
print(b)
```

a 에는:

```
apple
banana
```

```
orange
```

가 들어가지만 b 는:

```
''
```

빈 문자열이 된다.

왜냐하면 첫 번째 `read()`가:

```
    [apple banana orange]
    ↑

    f.read()

    [apple banana orange]
                         ↑
                         EOF
```

파일 끝까지 위치를 이동시켰기 때문이다.

`EOF`는 **End Of File**이다.

따라서 두 번째 `read()`에는 더 이상 읽을 것이 없다.

1.5.13 `readlines()` - 모든 줄을 리스트로

이번에는:

```
with open("data.txt", "r") as f:
    lines = f.readlines()

print(lines)
```

결과는 대략:

```
['apple\n', 'banana\n', 'orange']
```

이다.

즉:

```
read()
  ↓
"apple\nbanana\norange"
하나의 str
```

```
readline()
  ↓
"apple\n"
한 줄 str
```

```
readlines()
  ↓
['apple\n', 'banana\n', 'orange']
list[str]
```

여기까지만 보면 `readlines()`가 편해보인다.

하지만 역시 파일 전체의 줄들을 리스트로 만들어야 하기에 큰 파일에서는 메모리를 사용하게 된다.

1.5.14 그런데 Python에서는 파일을 그냥 for에 넣을 수 있다

여기서 `iterator`가 등장한다.

```
with open("data.txt", "r") as f:
    for line in f:
        print(line)
```

이 가능하다.

왜?

file object는 iterato이기 때문이다.

그래서 개념적으로:

```
iterator = iter(f)

line1= next(iterator)
line2= next(iterator)
line3= next(iterator)
```

가 가능하다.

실제로:

```
with open("data.txt", "r") as f:
    print(iter(f) is f)
```

는 일반적인 Python text file object에서:

```
True
```

가 된다.

즉 file object 자신이 `iterator`이다.

1.5.15 iterator protocol이 그대로 나온다

이터레이터 구조가:

`iterable`

↓

`iter()`

↓

`iterator`

↓

`next()`

↓

다음 값

↓

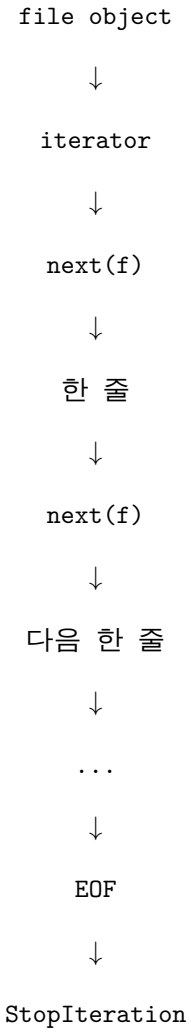
...

↓

`StopIteration`

이다.

file object에서는:



이다.

그래서:

```
with open("data.txt", "r") as f:
    print(next(f))
    print(next(f))
```

하면:

```
apple
```

```
banana
```

순서로 읽힌다.

1.5.16 for line in f가 큰 파일에서 중요한 이유

예를 들어 학습 데이터가:

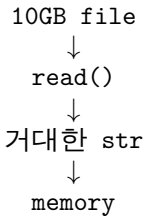
```
10GB trianing_data.txt
```

라고 해보자.

이걸:

```
with open("training_data.txt") as f:
    text = f.read()
```

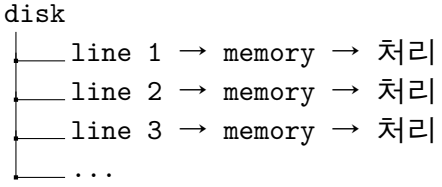
하면 전체 파일을 한꺼번에 읽으려고 한다.



반면:

```
with open("training_data.txt") as f:
    for line in f:
        process(line)
```

이면 개념적으로:



처럼 **전체 파일을 하나의 거대한 Python 문자열/리스트로 만들 필요가 없다.**

물론 실제 OS/Python I/O는 성능을 위해 buffering을 사용하기 때문에

디스크에서 정확히 물리적인 한 줄씩 읽는다는 뜻은 아니다.

하지만 **Python 코드 관점에서는 한 줄씩 순차적으로 소비할 수 있다**는 것이 핵심이다.

1.5.17 여기서 아주 중요한 연결 하나

위에서:

```
with open("data.txt") as f:
    for line in f:
        ...
```

에서 `f`가 `iterator`라고 했다.

그러면:

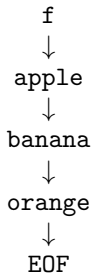
```
with open("data.txt") as f:
    for line in f:
        print(line)

    for line in f:
        print(line)
```

두 번째 `for`문에서는 어떻게 될까?

아무것도 출력되지 않는다.

첫 번째 `for`가 이미:



까지 소비했기 때문이다.

이것이:

> 소진된 iterator는 다시 처음부터 시작하지 않는다.

와 정확히 같은 현상이다.

처음부터 다시 읽으려면 예를 들어:

```
f.seek(0)
```

으로 파일 위치를 처음으로 되돌려야 한다.

1.5.18 지금 단계에서 핵심 정리

함수	내용
read()	남은 내용을 하나의 str로 읽음
readline()	현재 위치부터 한 줄을 str로 읽음
readlines()	남은 줄들을 list[str]로 읽음
for line in f	file iterator를 순차 소비
next(f)	다음 한 줄
tell()	현재 file position 확인
seek(...)	file position 이동

그리고 가장 중요한 개념은:

> 파일 읽기는 단순히 파일에서 값을 복사해오는 것이 아니라,
>
> file object가 현재 읽기 위치라는 상태를 가지고
>
> 그 상태가 진행되는 과정이다.

이것을 이해하면 read() 두 번 했는데 왜 두 번째가 비어있는지,

readline()이 왜 다음 줄로 넘어가는지,

파일을 왜 for문으로 순회할 수 있는지가 전부 하나의 원리로 연결된다

다음 단계에서는 쓰기(w, a, write)와 파일이 실제로 언제 만들어지고 언제 내용이 사라지는지 살펴본다.

1.5.19 "w" - 파일을 쓰기 모드로 열기

```
with open("result.txt", "w") as f:
    f.write("epoch 1")
```

실행하면 result.txt에:

```
epoch 1
```

이 저장된다.

그런데 "w"에는 중요한 성질이 두 가지 있다.

파일이 없응면 새로 만들고, 파일이 이미 있으면 기존 내용을 비운다.

예를 들어 원래:

```
epoch 1
epoch 2
epoch 3
```

였는데:

```
with open("result.txt", "w") as f:
    f.write("new training")
```

하면 결과는:

```
new training
```

이다.

즉 "w"는 기존 내용 뒤에 이어 쓰는 모드가 아니다.

그래서 "w"는 잘못 사용하면 기존 파일 내용을 날릴 수 있다.

1.5.20 "a" - append

기존 내용을 보존하면서 **끝에 추가**하려면:

```
with open("result.txt", "a") as f:
    f.write("epoch 4\n")
```

을 사용한다.

원래:

```
epoch 1
epoch 2
epoch 3
```

이었다면:

```
epoch 1
epoch 2
epoch 3
epoch 4
```

가 된다.

ML 학습 로그를 계속 누적한다고 생각하면 직관적이다.

```
with open("training.log", "a") as f:
    f.write("epoch=10 loss=0.231\n")
```

1.5.21 wirte()는 자동으로 줄바꿈하지 않는다

이것도 자주 실수하는 부분이다.

```
with open("result.txt", "w") as f:
    f.write("apple")
    f.write("banana")
    f.write("orange")
```

결과:

```
applebananaorange
```

print()처럼 자동 줄바꿈을 해주는 것이 아니다.

따라서:

```
f.write("apple\n")
f.write("banana\n")
f.write("orange\n")
```

처럼 줄바꿈 이스케이프 문자를 넣어야 한다.

1.5.22 write()는 무엇을 반환할까?

재미있는 부분인데:

```
n = f.write("hello")
```

n에는 보통:

```
5
```

가 들어간다.

즉 write()는 **작성한 문자 수**를 반환한다.

그래서 write()의 반환값은 None이 아니라는 것도 알아두면 좋다.

1.5.23 여러 문자열을 쓸 때 writelines()

예를 들어:

```
lines = [
    "epoch 1\n",
    "epoch 2\n",
    "epoch 3\n"
]

with open("result.txt", "w") as f:
    f.writelines(lines)
```

결과:

```
epoch 1
epoch 2
epoch 3
```

여기서도 writelines()가 자동으로 \n을 붙여주는 것은 아니다.

즉:

```
["apple", "banana", "orange"]
```

를 `writelines()` 하면:

```
applebananaorange
```

가 된다.

1.5.24 그런데 `write()`하면 즉시 디스크에 기록되는가?

여기서 buffering이 등장한다.

처음에 보기엔:

```
f.write(...)
     ↓
   disk
```

라고 생각하기 쉽지만 실제로는 보통 더 복잡하다.

개념적으로는:

```
Python program

f.write("hello")
     ↓
   buffer
     ↓
적절한 시점
     ↓
    OS
     ↓
filesystem
```

중간에 buffer를 두는 것이다.

왜 굳이 그럴까?

디스크나 OS에 아주 작은 데이터를 쓸 때마다 계속 요청하는 것보다

어느 정도 모아서 처리하는 것이 효율적이기 때문이다.

예를 들어:

```
f.write("a")
f.write("b")
f.write("c")
f.write("d")
```

라고 해서 반드시 물리 저장장치에:

```
a 기록
b 기록
c 기록
d 기록
```

와 같은 식으로 각각 독립적으로 기록되는 것은 아니다.

1.5.25 flush()

그래서 등장하는 것이:

```
f.flush()
```

이다.

개념적으로는:

> Python 쪽에 버퍼링된 출력 데이터를 아래 계층으로 밀어내라.

정도로 이해하면 된다.

예를 들어 장시간 학습 프로그램이:

```
f = open("training.log", "w")
for epoch in range(1000):
    ...
    f.write(f"epoch={epoch}, loss={loss}\n")
```

처럼 돌아간다고 해보자.

프로그램이 몇 시간 동안 실행될 수도 있다.

이때 로그를 외부에서 빨리 확인해야 한다면

buffering 때문에 원하는 시점에 내용이 안 보이는 상황이 있을 수 있다.

그럴 때:

```
f.write(...)
f.flush()
```

처럼 명시적으로 flush할 수도 있다.

1.5.26 그러면 close()와 flush()는 무슨 관계인가?

일반적으로 파일을 정상적으로 close()하면 필요한 flush()도 수행된다.

즉:

```
f.write(...)
f.close()
```

에서는 close 과정에서 버퍼에 남아 있는 데이터를 내보내고 파일 자원을 정리한다.

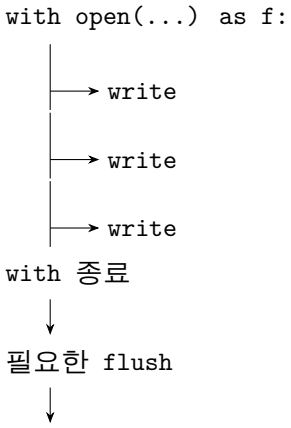
그리고 앞에서 배운:

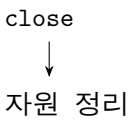
```
with open("result.txt", "w") as f:
    f.write("hello")
```

도 블록을 정상적으로 빠져나오면서 file object가 정리되니까

일반적인 파일 작업에서는 직접 flush()를 계속 호출할 필요가 없다.

그래서 with가 다시 중요해진다.





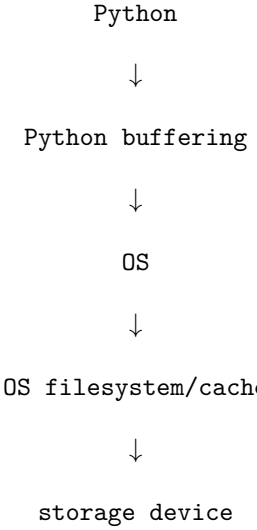
1.5.27 단, flush()가 물리 디스크에 영구 저장 완료는 아니다

이건 지금 단계에서는 구분만 해둔다.

```
f.flush()
```

했다고 해서 반드시 그 순간 **SSD의 물리 셀까지 영구적으로 기록되었다**는 의미는 아니다.

대략 여러 계층이 존재할 수 있다.



flush()는 기본적으로 Python의 버퍼를 아래 계층으로 넘기는 것과 관련된 것이지,

모든 하드웨어 계층까지 물리 기록 완료를 보장한다는 의미는 아니다.

정말 그런 수준의 보장이 필요한 경우라면

os.fsync() 같은 별도의 개념이 등장하는데, 지금은 살펴볼 필요가 없다.

1.5.28 "r", "w", "a"를 하나로 정리하면

"r"
읽기
파일이 없으면 → 오류
기존 내용 보존

"w"
쓰기
파일이 없으면 → 생성
파일이 있으면 → 기존 내용 제거

"a"
추가 쓰기
파일이 없으면 → 생성
파일이 있으면 → 기존 내용 보존 후 끝에 추가

이후에:

"x" 새 파일 생성, 이미 존재하면 오류

"+" 읽기 + 쓰기

"b" binary mode

"t" text mode

까지 붙으면 파일 mode 전체가 완성된다.

번외로, bytes와 encoding, decoding에 대한 내용

"r"
↓
읽겠다

text mode
↓
bytes를 문자로 해석하겠다

encoding="utf-8"
↓
그 bytes를 어떤 규칙으로 문자로 바꿀 것인가

파일의 bytes
↓
UTF-8 규칙으로 decode
↓
Python str
"안녕하세요"

text mode
→ Python이 encoding/decoding을 대신 처리
→ Python 코드에서는 str 사용

binary mode
→ encoding/decoding하지 않음
→ Python 코드에서는 bytes 사용

encoding은 정확히는 str → bytes 로 바꾸는 규칙
decoding은 그 반대인 bytes → str 로 해석하는 규칙

encoding='utf-8' 은 파일 타입을 지정하는 게 아니라
문자(str)와 파일에 저장되는 바이트(bytes) 사이를
어떤 규칙으로 변환할지 지정한다
cp949 도 똑같은 역할인데 변환 규칙이 UTF-8과 다른 것

bytes 는 저수준 프로그래밍 언어가 아니라
컴퓨터가 파일·네트워크 등에서 다루는 바이트 데이터 표현
머릿속에서는 간단하게:
사람이 다루는 문자(str) → encode → bytes → decode → 문자(str)

저수준의 File I/O는 이 정도로 끝내고,

실제로 Python/ML/DL에서 자주만나는 구조화된 파일 형식으로 넘어간다.

1.5.29 JSON이 왜 필요한가?

지금까지는 이런 텍스트를 저장했다.

```
epoch=10
batch_size = 32
```

```
learning_rate = 0.001
```

사람은 이해할 수 있지만 Python 입장에서는 그냥 하나의 문자열이다.

ML 설정을 생각해보자.

```
config = {
    "epochs": 10,
    "batch_size": 32,
    "learning_rate": 0.001,
    "hidden_sizes": [512, 256, 128]
}
```

이런 **구조화된 데이터**를 파일로 저장하고 다시 불러오고 싶을 수 있다.

JSON은 이런 데이터를 표현하기 위한 대표적인 형식이다.

`config.json`:

```
{
    "epochs": 10,
    "batch_size": 32,
    "learning_rate": 0.001,
    "hidden_sizes": [512, 256, 128]
}
```

Python Dictionary와 상당히 비슷하게 생겼다.

하지만 중요한 차이가 존재한다.

> JSON 자체는 Python dictionary가 아니다.

그냥 JSON이라는 데이터 형식이다.

1.5.30 JSON 파일 → Python 객체

Python에서는 표준 라이브러리 `json`을 사용한다.

```
import json

with open("config.json", "r", encoding="utf-8") as f:
    config = json.load(f)
```

그러면:

```
print(type(config))
```

결과:

```
<class 'dict'>
```

그리고:

```
print(config['learning_rate'])
```

```
0.01
```

처럼 사용할 수 있다.

흐름을 보면:

```
config.json
    ↓
file object f
    ↓
json.load(f)
    ↓
Python 객체
    ↓
config
    ↓
dict
```

즉 `json.load()`는 JSON 형식의 데이터를 읽어서 적절한 Python 객체로 변환해준다.

1.5.31 JSON과 Python 타입은 서로 대응한다

대표적으로:

JSON	Python
object	dict
array	list
string	str
number	int/float
true	True
false	False
null	None

그래서 JSON:

```
{
    "use_gpu": true,
    "dropout": null,
    "layers": [512, 256]
}
```

을 읽으면 Python에서는 대략:

```
{
    "use_gpu": True,
    "dropout": None,
    "layers": [512, 256]
}
```

가 된다.

1.5.32 반대로 Python 객체 → JSON 파일

이번에는:

```
config = {
    "epochs": 20,
    "batch_size": 64,
    "learning_rate": 0.001
}
```

를 저장해보자.

```
import json
```

```
with open("config.json", "w", encoding="utf-8") as f:
    json.dump(config, f)
```

방향이 반대다.

```
Python dict
    ↓
json.dump()
    ↓
 JSON 형식
    ↓
config.json
```

그래서 이름도 꽤 직관적이다.

load
파일에서 Python으로 불러옴

dump
Python 객체를 파일로 내보냄

1.5.33 load()와 loads()는 다르다

여기서 Python에서 상당히 헷갈리는 이름이 등장한다.

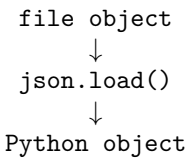
```
json.load(...)
json.loads(...)
```

s 하나 차이인데 의미가 다르다.

json.load()

파일 객체에서 읽는다.

```
with open("config.json") as f:
    config = json.load(f)
```

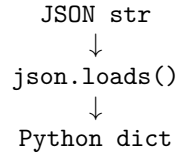


json.loads()

이미 가지고 있는 **JSON 문자열**을 Python 객체로 변환한다.

```
text = '{"epochs": 10, "batch_size": 32}'
config = json.loads(text)
```

여기서는 파일이 없다.

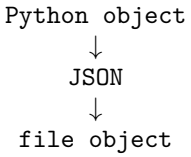


여기서 loads의 s는 **string**이라고 기억하면 편하다.

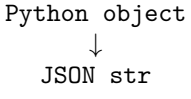
1.5.34 반대쪽에도 dump()와 dumps()가 있다

```
json.dump(...)
json.dumps(...)
```

dump():



dumps():



예를 들어:

```
config = {
    "epochs": 10,
    "batch_size": 32
}

text = json.dumps(config)
print(type(text))
```

결과:

```
<class 'str'>
```

즉 전체를 하나로 정리하면:

	파일	문자열
읽기	json.load() ↓ Python object	json.loads() ↓ Python object
쓰기	json.dump() ↑ Python object	json.dumps() ↑ Python object

1.5.35 ML/DL에서는 어디에 쓰일까?

예를 들어 학습 설정을:

```
{
    "model_name": "distilbert-base-uncased",
    "epochs": 3,
    "batch_size": 16,
    "learning_rate": 0.00002
}
```

로 저장하고:

```
import json
```

```
with open("config.json", encoding="utf-8") as f:
    config = json.load(f)

model_name = config["model_name"]
epochs = config["epochs"]
```

처럼 사용할 수 있다.

또 학습 결과를:

```
metrics = {
    "accuracy": 0.91,
    "f1": 0.89,
    "loss": 0.21
}

with open("metric.json", "w", encoding="utf-8") as f:
    json.dump(metrics, f, indent=4)
```

처럼 저장할 수도 있다.

indent=4를 사용하면 사람이 읽기 좋게 들여쓰기해서 저장해준다.

1.5.36 그런데 여기서 argparse와 연결된다

우리가 이전에 배운:

```
python train.py \
    --epochs 10 \
    --batch-size 32 \
    --learning-rate 0.001
```

도 결국 **프로그램 외부에서 설정을 넣는 방법**이었다.

JSON도 비슷한 목적에 사용할 수 있다.

```
python train.py --config config.json
```

그리고:

```
import argparse
import json
from pathlib import Path
```

```python
parser = argparse.Argumentparser()
parser.add_argument(
    "--config",
    type=Path,
    required=True
)

args = parser.parse_args()

with args.config.open(encoding='utf-8') as f:
    config = json.load(f)

print(config['learning_rate'])
```

이제 지금까지 따로 배웠던 것들이:

Terminal
↓
argparse
↓
Path
↓
file object
↓
JSON
↓
dict
↓
ML training configuration

으로 하나로 연결된다.

이게 앞에서:

argparse → pathlib → filesystem → File I/O

를 순서대로 공부한 이유이기도 하다.

다음은 CSV를 Python 표준 csv 모듈 관점에서 짧게 보고,

왜 ML/Data Science에서는 보통 그것보다

pandas.read_csv()를 쓰는지 연결해서 볼 것이다.

1.5.37 CSV는 사실 텍스트 파일이다

예를 들어 `scores.csv`:

```
name,math,english
kim,90,85
lee,80,92
park,95,88
```

CSV는 **Comma-Separated Values**이다.

특별한 Python 객체가 저장되어 있는 게 아니라

기본적으로 이런 **텍스트 데이터**이다.

그래서 원한다면:

```
with open("scores.csv", "r", encoding="utf-8") as f:
    print(f.read())
```

로도 읽을 수 있다.

다만 우리가 원하는 건 보통 문자열 전체가 아니라:

name	math	english
kim	90	85
lee	80	92
park	95	88

같은 **표 구조**이다.

그래서 CSV 형식을 해석하는 도구가 필요하다.

1.5.38 Python에는 표준 csv모듈이 있다

```
import csv

with open("scores.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)

    for row in reader:
        print(row)
```

결과는:

```
['name', 'math', 'english']
['kim', '90', '85']
['lee', '80', '92']
['park', '95', '88']
```

여기서 중요한 부분이 하나 보인다

```
'90'
```

이지:

```
90
```

이 아니다.

CSV 자체는 텍스트이기 때문에 `csv.reader()`가 기본적으로 읽어온 필드들도 문자열로 다뤄진다.

필요하면 우리가:

```
math = int(row[1])
```

처럼 변환할 수 있다.

1.5.39 `csv.reader(f)`와 `iterator`

여기서 배운 내용이 연결된다.

```
reader = csv.reader(f)
```

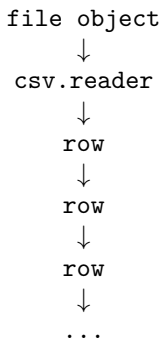
`reader`도 순차적으로 row를 제공하는 iterable/iterator 계열 객체다.

그래서:

```
for row in reader:
    ...
```

가 자연스럽게 가능하다.

흐름은:



즉 Python의 iterator모델이 여기서도 계속 나타난다.

1.5.40 그런데 ML/Data Science에서는 Pandas를 훨씬 많이 쓴다

```
import pandas as pd

df = pd.read_csv("scores.csv")
```

가 등장한다.

이 경우 Pandas가 상당히 많은 일을 대신 처리한다.



그래서:

```
df.info()
df.describe()
df["math"].mean()
```

같은 데이터 분석 작업으로 바로 넘어갈 수 있다.

1.5.41 CSV에서 실제로 알아둘 옵션

ML/Data Science 기준으로는 이 정도면 충분하다.

```
df = pd.read_csv(
    "data.csv",
    encoding='utf-8'
)
```

한글 파일이면 상황에 따라:

```
df = pd.read_csv(
    "data.csv",
    encoding='cp949'
)
```

를 만날 수 있다.

쉼표가 아닌 다른 구분자를 사용하는 파일도 있다.

예를 들어:

```
name|math|english
kim|90|85
lee|80|92
```

이면:

```
df = pd.read_csv(
    "data.csv",
    sep="|"
)
```

처럼 delimiter를 지정할 수 있다.

`sep` = separator라고 보면 된다.

1.5.42 DataFrame → CSV도 가능하다

반대 방향:

```
df.to_csv("result.csv")
```

그런데 Pandas에서 흔히:

```
df.to_csv(
    "result.csv",
    index=False,
    encoding="utf-8"
)
```

를 많이 본다.

`index=False`는 DataFrame의 index를 별도 CSV 열로 저장하지 않겠다는 뜻이다.

예를 들어:

```
    name score
0   kim    90
1   lee    80
```

를 그냥 저장하면 index까지 포함될 수 있는데,

```
index = False
```

로 하면:

```
name,score
kim,90
lee,80
```

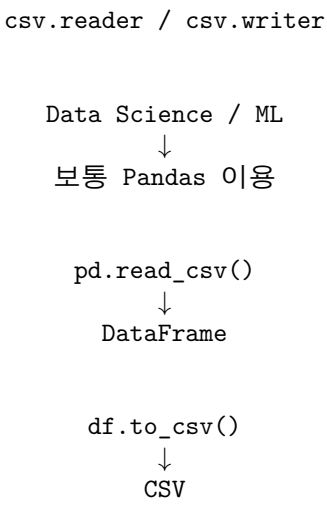
처럼 실제 데이터 열만 저장할 수 있다.

1.5.43 그러면 CSV는 이 정도면 끝이다

기억할 것은:

CSV
↓
텍스트 기반의 표 형식

Python 기본 기능
↓



이 정도면 충분하다.

1.5.44 이제 Binary I/O

Text mode와 Binary mode

지금까지:

```
open("data.txt", "r")
```

라고 썼다.

사실 기본적으로 text mode라는 의미가 포함되어 있다.

명시하면:

```
open("data.txt", "rt")
```

라고도 할 수 있다.

t = text

반면:

```
open("image.jpg", "rb")
```

에서는:

```
r = read
b = binary
```

즉 **binary read**이다.

1.5.45 가장 중요한 차이는 반환되는 Python 객체

Text:

```
with open("data.txt", "r", encoding="utf-8") as f:
    data = f.read()

print(type(data))
```

```
str
```

Binary:

```
with open("image.jpg", "rb") as f:
    data = f.read()

print(type(data))
```

```
bytes
```

즉:

```
text mode

file bytes
    ↓
decoding
    ↓
Python str


binary mode

file bytes
    ↓
그대로
    ↓
Python bytes
```

이게 핵심이다.

1.5.46 Binary에서는 encoding을 지정하지 않는다

예를 들어:

```
open("image.jpg", "rb", encoding="utf-8")
```

처럼 생각하면 **안된다.**

Binary mode에서는:

bytes
↓
그대로 사용

하기 때문이다.

encoding은:

bytes ↔ str

변환 규칙이다.

Binary mode는 그 변환 자체를 하지 않는다.

1.5.47 그런데 ML/DL에서는 보통 직접 안 한다

예를 들어 NumPy array를 저장한다고 하자.

우리가 직접:

```
with open("array.npy", "wb") as f:
    ...
```

하면서 NumPy binary format을 구현하지 않는다.

그냥:

```
import numpy as np

np.save("array.npy", arr)
```

하고:

```
arr = np.load("array.npy")
```

한다.

PyTorch도:

```
torch.save(...)
torch.load(...)
```

이미지도:

```
Image.open(...)
```

Pandas Parquet도:

```
pd.read_parquet(...)
```

처럼 사용한다.

즉:

저수준

```
open(..., 'rb')
open(..., 'wb')
      ↓
bytes 직접 처리
```

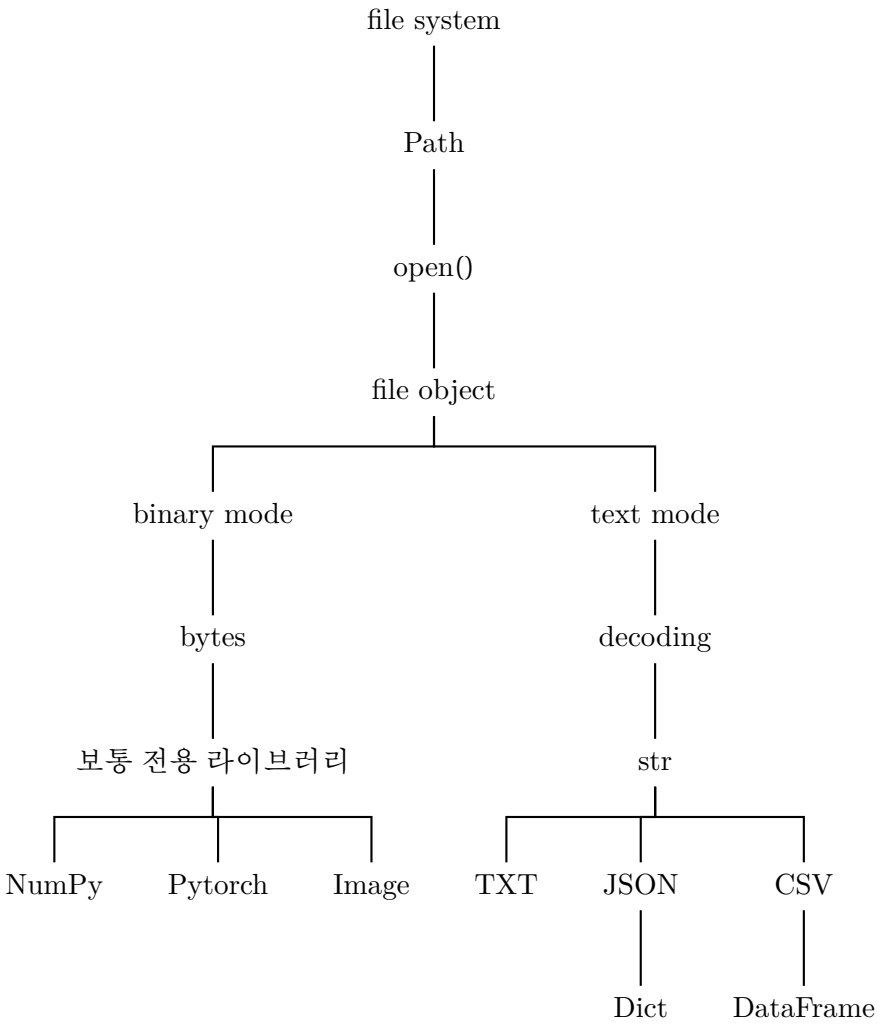
고수준

```
  np.load()
 torch.load()
 Image.open()
pd.rad_parquet()
      ↓
라이브러리가 binary format 처리
```

그렇기에 ML/DL 하면서 `'rb'`를 거의 못 본것이 이상한 것은 아니다.

1.5.48 File I/O 전체가 이제 하나로 연결된다

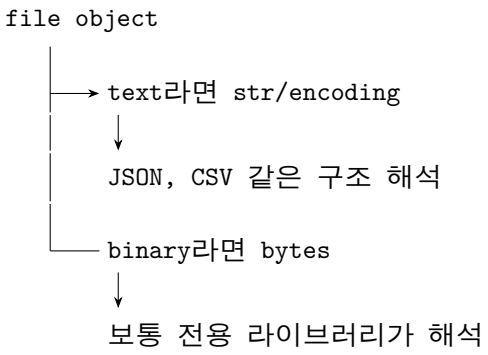
전체 구조를 보면:



이정도면 File I/O를 거의 완료했다고 봐도 된다.

특히 중요한 건 API를 암기하는 것이 아니라:

`Path`
↓



이라는 전체 레이어를 이해하는 것이다.

> 번외로, iterator에 대한 한 문장 설명을 추가하자면,
>
> Iterator는 for 문을 돌릴 수 없는 객체가 아니라,
>
> for 문이 실제로 값을 하나씩 꺼내기 위해 사용하는 객체다.

1.6 URL vs Local Path

1.6.1 먼저 둘의 차이부터 이해하자

지금까지 우리가 다루었던 경로는 대부분 이런 형태였다.

```
from pathlib import Path

path = Path("data/train.csv")
```

또는:

```
path = Path("/Users/user/project/data/train.csv")
```

이것들은 Local Path(로컬 파일 경로)이다.

반면 다음은 URL 이다:

```
https://example.com/data/train.csv
```

둘 다 파일의 위치를 나타내는 것처럼 보인다.

하지만 Python에서 접근하는 방식은 완전히 다르다.

구분	Local Path	URL
목적	파일 시스템의 위치 지정	리소스의 위치 및 접근 방법 지정
예시	`data/train.csv`	`https://example.com/data/train.csv`
접근 방식	파일 시스템	네트워크(HTTP 등)
대표 도구	`pathlib`, `open()`	`requests`, `urllib`
접근 대상	로컬 또는 마운트된 파일 시스템	웹 서버 등의 리소스

여기서 중요한 것은 **URL은 단순히 인터넷에 있는 파일의 경로만을 의미하지 않는다는 것**

URL은 `Uniform Resource Locator`의 약자로,

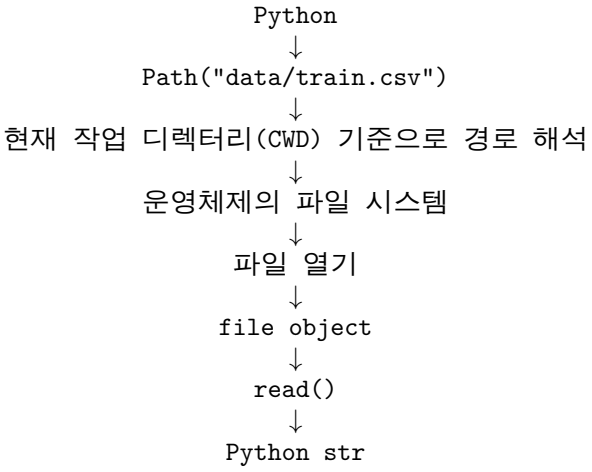
리소스에 접근하기 위한 위치와 방법을 나타낸다.

1.6.2 Local Path는 파일 시스템에서 해석된다

```python
from pathlib import Path

path = Path("data/train.csv")

with path.open("r", encoding="utf-8") as f:
    data = f.read()
```



예를 들어 CWD가:

```
/Users/user/my_project
```

라면:

```
Path("data/train.csv")
```

는 다음 파일을 가리키는 상대 경로로 해석된다.

```
/Users/user/my_project/data/train.csv
```

여기서 중요한 것은 **open()이 파일 시스템을 대상으로 작동한다는 것**이다.

1.6.3 URL은 네트워크 요청을 통해 접근한다

이번에는:

```
https://example.com/data/train.csv
```

를 보자.

이 URL을 분해하면 다음과 같다.

```
https:/example.com/data/train.csv
├── https → scheme
├── example.com → host
└── /data/train.csv → path
```

각각의 의미는 다음과 같다.

```
1. Scheme
    https
    어떤 방식으로 리소스에 접근할지 나타낸다.
    여기서 HTTPS 프로토콜을 사용한다는 뜻이다.


2. Host
    example.com
    어느 서버에 접근할지 나타내는 호스트 이름이다.
```

3. Path
   /data/train.csv
   서버에 요청할 리소스의 경로이다.
   여기서 주의할 점은,
   URL의 Path는 로컬 파일시스템의 path와 동일한 개념이 아니다.

서버가 이 경로를 실제 파일에 연결할 수도 있지만,
데이터베이스 조회나 프로그램 실행 결과에 연결할 수도 있다.

예를 들어:

```
https://api.example.com/users/123
```

은 서버의 users/123이라는 실제 파일을 의미하는 것이 아닐 수 있다.

서버가 사용자 123의 정보를 조회해서 JSON을 반환하는 API일 수도 있다.

1.6.4 그래서 open()으로 URL을 직접 열 수 없다

다음 코드를 보자.

```
with open("https://example.com/data/train.csv") as f:
    data = f.read()
```

이 코드는 일반적으로 실패한다.

왜일까?

open()은 전달된 문자열을 **파일 시스템 경로**로 해석하기 때문이다.

즉 Python은 이것을 네트워크 주소라고 판단해서 HTTP 요청을 보내지 않는다.

반면 HTTP 요청을 보내려면 다음과 같은 라이브러리를 사용할 수 있다.

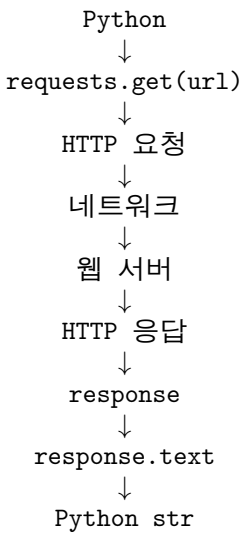
```
import requests

url = "https://example.com/data/train.csv"

response = requests.get(url, timeout=10)
response.raise_for_status()
```

```
data = response.text
```

전체 흐름은 다음과 같다.



여기서 `requests.get()`은 HTTP GET 요청을 보내는 함수다.

`response`는 서버에서 돌아온 HTTP 응답을 표현하는 객체이다.

`response.text`는 응답 본문을 문자로 디코딩해서 제공한다.

즉, encoding/decoding 개념과 연결된다.

1.6.5 그런데 Pandas에서는 URL을 바로 사용할 수 있다

우리가 지금까지 배운:

```
pd.read_csv('data/train.csv')
```

는 로컬 파일을 읽는 코드이다.

그런데 Pandas는 다음과 같은 코드도 지원한다.

```
import pandas as pd

url = "https://example.com/data/train.csv"
```

```
df = pd.read_csv(url)
```

단, 실제로 해당 URL에서 CSV 데이터를 제공해야 정상적으로 작동한다.

그렇다면 왜 Pandas에서는 URL을 직접 사용할 수 있을까?

Pandas가 URL을 인식하고 내부에서 필요한 데이터 접근 및 파싱 작업을 처리하기 때문이다.

개념적으로:

```
pd.read_csv("data/train.csv")
        ↓
   Local Path
        ↓
  파일 시스템
        ↓
      CSV
        ↓
   DataFrame
```

반면:

```
pd.read_csv("https://...")
        ↓
       URL
        ↓
   HTTP 요청
        ↓
  CSV 데이터
        ↓
   DataFrame
```

즉 사용자에게는 동일한 `read_csv()` 인터페이스를 제공하지만,

내부적으로 데이터를 가져오는 방법은 달라질 수 있다.

이것을 **추상화(Abstraction)**의 사례로 볼 수 있다.

1.6.6 Hugging Face에서도 비슷한 일이 일어난다

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained(
    "distilbert-base-uncased"
)
```

여기서:

```
"distilbert-base-uncased"
```

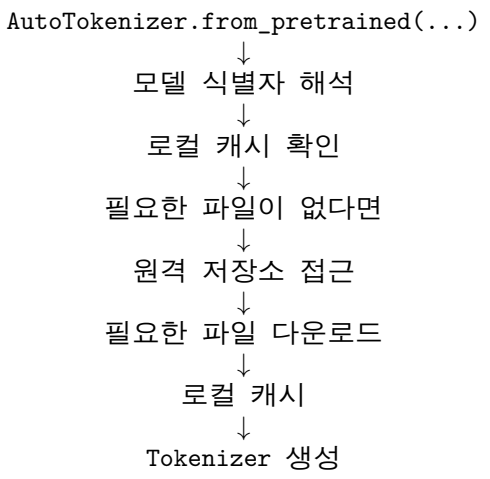
는 일반적인 URL도 아니고, 파일 경로라고 명시한 것도 아니다.

이것은 Hugging Face 모델을 식별하는 이름으로 사용할 수 있다.

라이브러리는 이 식별자를 바탕으로 필요한 파일을 찾고,

로컬 캐시에 없으면 원격 저장소에서 다운로드할 수 있다.

개념적으로:



물론 로컬 디렉터리를 직접 지정하는 것도 가능하다.

```
AutoTokenizer.from_pretrained(
    "./my_tokenizer"
)
```

이 경우에는 해당 로컬 디렉터리에서 필요한 파일을 찾도록 사용할 수 있다.

따라서 `from_pretrained()` 같은 함수는 단순한 파일 읽기 함수보다

더 높은 수준의 인터페이스를 제공하는 것이다.

1.6.7 URL의 데이터도 결국 메모리나 파일로 가져올 수 있다

예를 들어 웹에서 JSON을 가져온다고 하자.

```
import requests

url = "https://api.example.com/config"
response = requests.get(url, timeout=10)
response.raise_for_status()
config = response.json()
```

여기서:

```
response.json()
```

은 HTTP 응답 본문의 JSON 데이터를 Python 객체로 변환한다.

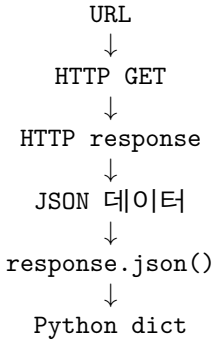
앞에서 배운:

```
json.loads(text)
```

과 비슷한 역할을 한다고 이해하면 된다.

다만 `response.json()`은 `requests`가 제공하는 메서드다.

전체 흐름은:



이다.

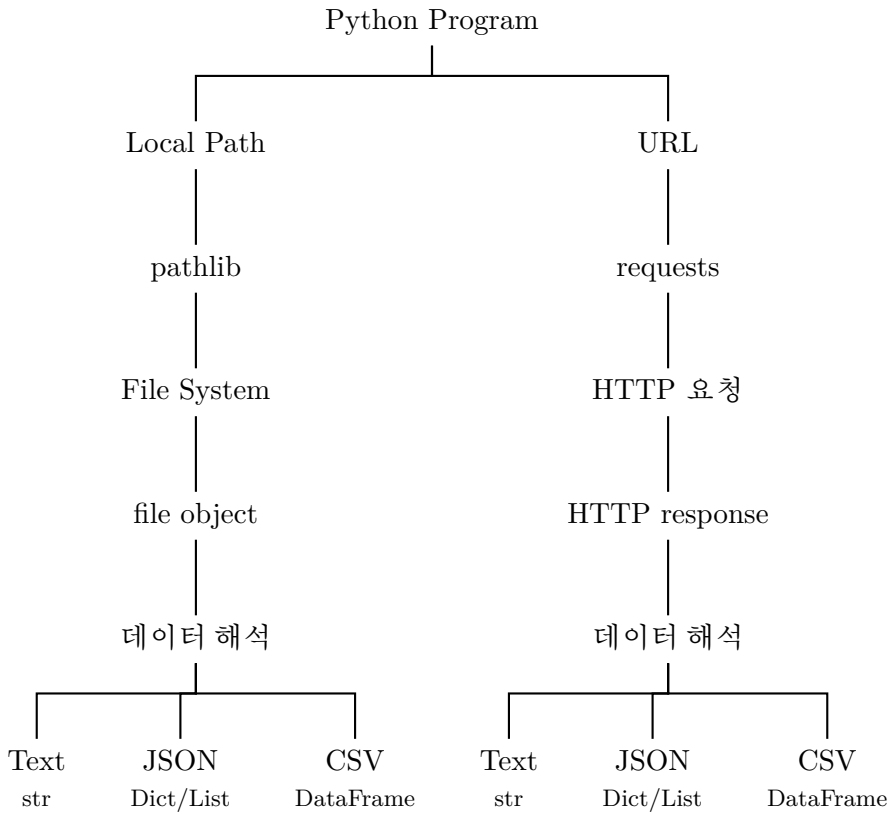
JSON 최상위 구조에 따라 list 등 다른 Python 객체가 반환될 수 있다.

여기서 중요한 점은 **네트워크에서 데이터를 가져온다고 해서 반드시 로컬 파일로 저장해야 하는 것은 아니라는 것**이다.

응답 데이터를 바로 메모리에서 처리할 수도 있다.

1.6.8 지금까지 배운 개념을 통합해보자

우리가 지금까지 학습한 흐름을 연결하면 다음과 같다.



이 그림에서 한 가지를 이해하면 된다.

데이터가 어디에 있는지와 데이터를 어떤 형식으로 해석하는지는 서로 다른 문제다.

예를 들어 CSV는 로컬 파일에서도 읽을 수 있고 HTTP 서버에서도 가져올 수 있다.

하지만 어느 쪽에서 가져왔든 CSV 데이터를 DataFrame으로 변환하는 작업 자체는 별개의 단계이다.

이 구분은 앞으로 데이터 파이프라인을 설계할 때도 중요하다.

1.6.9 핵심 정리

기억할 것은 4 가지이다.

1. Local Path는 파일 시스템에서 해석되는 경로이며,
   pathlib 와 open() 으로 다룬다.

2. URL은 리소스의 위치와 접근 방법을 나타내며,
   HTTP URL은 requests 같은 도구로 접근할 수 있다.

3. pd.read_csv()처럼 고수준 라이브러리는
   로컬 파일과 URL을 모두 지원할 수 있다.

4. 데이터에 접근하는 단계와 JSON·CSV 등의 형식을 해석하는 단계는 구분된다.

1.6.10 HTTP 요청과 응답

HTTP는 클라이언트와 서버가 요청(request)과 응답(response)을 주고받기 위한 통신 규칙이다.

예를 들어 Python에서 공공데이터를 요청한다고 생각해보자.

Python 프로그램 (Client)
↓ HTTP Request
공공데이터 서버
↓ HTTP Response
Python 프로그램

이때 Python 프로그램이 클라이언트다.

공공데이터를 제공하는 쪽이 서버이다.

핵심은 **HTTP 통신에서는 클라이언트가 요청을 보내고 서버가 응답한다**는 구조다.

1.6.11 HTTP Request는 어떻게 구성될까?

HTTP 요청을 개념적으로 분해하면 다음과 같다.



하나씩 보자.

1.6.12 Method - 무엇을 하려는가?

HTTP Method는 서버에 요청하는 작업의 성격을 나타낸다.

대표적인 두 가지가 있다.

GET

서버에서 데이터를 가져오는 데 사용한다.

```
response = request.get(
    "https://api.example.com/data"
)
```

예를 들어:

- 공공데이터 조회
- 특정 금융상품 정보 조회
- 날씨 정보 조회
- 모델 메타데이터 조회

등에 사용할 수 있다.

POST

서버에 데이터를 전달해서 처리하도록 요청할 때 주로 사용한다.

```
response = requests.post(
    "https://api.example.com/predict",
    json={"income": 5000, "debt": 1000}
)
```

이 경우 서버에 데이터를 보내서 예측 결과를 요청하는 형태다.

여기서 한 가지 중요한 점은 **GET은 조회, POST는 생성이라고 외우면 안 된다는 것**이다.

POST는 새로운 데이터를 생성하는 데도 사용하지만, 데이터를 전달해서 계산이나 예측을 수행하도록 요청하는 데도 사용한다.

예를 들어 ML 추론 API가 대표적이다.

GET
클라이언트 → 데이터 조회 요청 → 서버

POST
클라이언트 → 입력 데이터 전달 → 서버에서 처리

1.6.13 Headers - 요청에 대한 부가 정보

Header는 HTTP 요청이나 응답에 대한 메타데이터를 전달한다.

예를 들어:

```
headers = {
    "Accept": "application/json"
}

response = requests.get(
    "https://api.example.com/data",
    headers=headers
)
```

여기서:

```
Accept: application/json
```

은 클라이언트가 JSON 형식의 응답을 받을 수 있고 선호한다는 의미이다.

다만 서버가 반드시 JSON을 반환해야 한다는 뜻은 아니다.

다른 예시로:

```
headers = {
    "Authorization": "Bearer YOUR_API_TOKEN"
}
```

같은 인증 정보도 Header에 넣을 수 있다.

이 부분은 Hugging Face나 외부 API를 사용할 때 자주 등장한다.

API 토큰은 실제 프로젝트에서 코드에 직접 적지 않고 환경변수 등으로 관리하는 것이 좋다.

1.6.14 Body - 서버에 전달할 데이터

이번에는 POST 요청을 보자.

```
payload = {
    "income": 5000,
    "debt": 1000
}

response = requests.post(
    "https://api.example.com/predict",
    json=payload
)
```

여기서:

```
json=payload
```

는 Python dictionary를 JSON 형식으로 직렬화하여

HTTP 요청의 Body에 넣도록 `requests`에 지시하는 것이다.

또한 일반적으로:

```
Content-Type: application/json
```

이라는 Header도 자동으로 설정한다.

전체 구조를 보면:

```
HTTP Request
├── Method: POST
├── URL: /predict
├── Headers
│   └── Content-Type: application/json
└── Body
    └── {"income": 5000, "debt": 1000}
```

여기서 JSON이 다시 등장한다.

우리는 이미:

```
json.dumps(payload)
```

가 Python 객체를 JSON 문자열로 변환한다는 것을 배웠다.

`requests.post(...,json=payload)`는 이런 JSON 직렬화 과정을 내부적으로 처리해주는 것이다.

1.6.15 HTTP REsponse는 무엇인가?

클라이언트가 요청을 보내면 서버는 응답을 반환한다.

개념적으로:

```
HTTP Response
├── Status Code
│   └── 200 / 404 / 500 ...
├── Headers
│   └── Content-Type 등
└── Body
    └── 실제 응답 데이터
```

여기서 가장 먼저 살펴볼 것은 Status Code이다.

Status Code - 요청 처리 결과

HTTP 상태 코드는 서버가 요청을 어떻게 처리했는지 알려주는 숫자다.

상태코드	의미
200	OK - 요청성공
201	Created - 리소스 생성 성공
400	Bad Request - 잘못된 요청
401	Unauthorized - 인증 필요 또는 인증 실패
403	Forbidden - 접근 거부
404	Not Found - 요청한 리소스를 찾을 수 없음
429	Too Many Requests - 요청 횟수 제한
500	Internal Server Error - 서버 내부 오류
503	Service Unavailable - 서버가 일시적으로 요청을 처리할 수 없음

개별 코드를 전부 암기할 필요는 없다.

대신 다음 분류를 이해하면 된다.

2xx → 성공

3xx → 리다이렉션

4xx → 클라이언트 측 요청 문제

5xx → 서버 측 처리 문제

실제로는 인증, 권한, 요청 제한 등 여러 상황이 있으므로 4xx를 무조건 클라이언트 코드의 버그라고 생각하면 안 된다.

1.6.16 requests.Response객체 이해하기

이제 다음 코드를 보자.

```
import requests
```

```
response = requests.get(
    "https://api.example.com/data",
    timeout=10
)
```

여기서 response는 단순한 문자열이 아니다.

requests.Response 객체이다.

이 객체는 서버에서 받은 응답 정보를 가지고 있다.

```
response.status_code
response.headers
response.text
response.content
response.json()
```

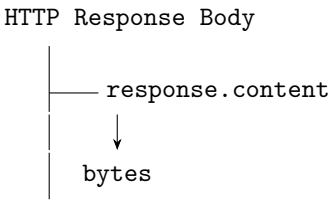
각각 의미가 다르다.

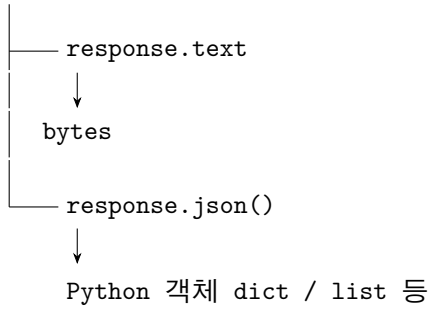
코드	반환 내용
response.status_code	HTTP 상태 코드(int)
response.headers	응답 Header 정보
response.text	응답 Body를 문자열로 디코딩한 결과(str)
response.content	응답 Body의 바이트 데이터(bytes)
response.json()	JSON Body를 Python 객체로 파싱한 결과

특히 다음 세 가지는 구분해야 한다.

```
response.text
response.content
response.json()
```

앞에서 배운 File I/O와 연결하면:





단, `response.json()`은 응답 본문이 유효한 JSON이어야 정상적으로 동작한다.

JSON이 아닌 HTML이나 일반 텍스트를 반환했다면 파싱 오류가 발생할 수 있다.

1.6.17 `raise_for_status()`는 왜 필요할까?

다음 코드를 보자.

```
response = requests.get(url, timeout=10)
response.raise_for_status()
```

이 메서드는 HTTP 응답이 4xx 또는 5xx 오류 상태라면 예외를 발생시킨다.

예를 들어 서버가 404를 반환했다면:

```
response.raise_for_status()
```

에서 `HTTPError`가 발생한다.

반면 정상적인 200 응답이라면 다음 코드로 진행할 수 있다.

```
data = response.json()
```

여기서 중요한 차이가 있다.

request.get()이 HTTP 응답을 받았다는 사실과 요청이 성공했다는 사실은 다르다.

서버가 404 오류 페이지를 반환했더라도 HTTP 응답 자체는 수신했을 수 있다.

또한 네트워크 연결 자체가 실패하면 Response를 받기 전에 예외가 발생할 수도 있다.

그래서 기본적인 API 호출에서는:

```
response = requests.get(url, timeout=10)
response.raise_for_status()
```

를 함께 사용하는 것이 좋다.

1.6.18 공공데이터 API는 어떻게 호출할까?

예를 들어 공공데이터 API가 다음과 같은 주소를 제공한다고 가정하자.

```
https://api.example.com/financial-products
```

그리고 API 문서에 다음 파라미터가 정의되어 있다고 가정해보자.

```
    page = 페이지 번호

    per_page = 페이지당 데이터 수
```

그러면 Python에서:

```
import requests

url = "https://api.example.com/financial-products"
params = {
    "page": 1,
    "per_page": 100
}

response = requests.get(
    url,
    params=params,
    timeout=10
)

response.raise_for_status()
data = response.json()
```

처럼 요청할 수 있다.

여기서 새롭게 등장한 것은:

```
params = params
```

이다.

이건 URL에 **Query Parameters**를 추가하는 기능이다.

실제 요청 URL은 개념적으로 다음과 같이 구성된다.

```
https://api.example.com/financial-products?page=1&per_page=100
```

구조를 분해하면:

```
https:/api.example.com/financial-products
└── URL Path

?page=1
└── page = 1

&per_page=100
└── per_page = 100
```

즉 GET 요청에서는 필요한 조회 조건을 URL의 Query Parameters로 전달하는 경우가 많다.

하지만 모든 GET 요청이 반드시 Query Parameters를 사용해야 하는 것은 아니다.

1.6.19 Query Parameters와 JSON Body의 차이

둘을 비교해보자.

GET 요청

```
requests.get(
    url,
    params={'page':1}
)
```

```
GET /financial-products?page=1
```

POST 요청

```
requests.post(
    url,
    json={"income": 5000}
)
```

```
POST /predict

Body:
{"income": 5000}
```

여기에서 기억할 핵심은:

- params= 는 URL의 Query Parameters를 구성한다.
- json= 은 JSON 형식의 요청 Body를 구성한다.

다만 Query Parameters는 POST에서도 사용할 수 있고,

POST Body가 반드시 JSON이어야 하는 것도 아니다

실제 사용 방식은 API 문서에서 정한다.

1.6.20 공공데이터 API → Pandas 연결

공공데이터 API가 다음과 같은 JSON을 반환한다고 가정하자.

```
{
    "data": [
        {
            "product_name": "상품A",
            "interest_rate": 3.5
        },
        {
            "product_name": "상품B",
            "interest_rate": 3.8
        }
    ]
}
```

Python에서는:

```
import requests
```

```
import pandas as pd

response = requests.get(url, timeout=10)
response.raise_for_status()

result = response.json()

records = result["data"]

df = pd.DataFrame(records)
```

처럼 처리할 수 있다.

전체 흐름을 보면:

```
공공 데이터 API
   ↓ HTTP GET
HTTP Response
   ↓ response.json()
Python dict
   ↓ result['data']
Python list[dict]
   ↓ pd.DataFrame()
DataFrame
   ↓
```

이렇게 Pandas와 연결된다.

실제 공공데이터 API에서는 인증키, 페이지네이션, 응답 형식 등이 서비스마다 다르기에

반드시 해당 API 문서를 확인해야 한다.

1.6.21 다운로드와 파일 저장도 연결해보자

마지막으로 API에서 받은 데이터를 파일로 저장하는 경우도 살펴보자.

```
import requests
from pathlib import Path

url = "https://example.com/data/train.csv"
```

```
response = requests.get(url, timeout=10)
response.raise_for_status()

path = Path("data/train.csv")

path.parent.mkdir(
    parents=True,
    exist_ok=True
)

path.write_bytes(response.content)
```

여기서:

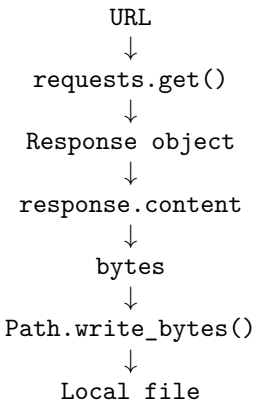
```
response.content
```

는 bytes 객체이고,

```
path.write_bytes(...)
```

는 해당 바이트 데이터를 파일에 기록하는 메서드다.

이 코드의 전체 흐름은:



가 된다.

이후:

```
df = pd.read_csv(path)
```

로 읽으면 된다.

참고로 큰 파일은 `response.content`로 한 번에 메모리에 올리기보다

`stream=True`를 사용해서 나누어 저장하는 것이 적절할 수 있다.

1.7 Environment Variables (환경변수)

이번에는 Python 실행 환경에서 중요한 개념인 **환경변수**를 살펴본다.

앞에서 다음 3 가지를 배웠다.

1. `sys.argv` → 프로그램 실행 시 전달하는 인수
2. `sys.path` → Python의 모듈 검색 경로
3. `sys.executable` → 현재 실행 중인 Python 인터프리터

그런데 프로그램이 실행될 때 외부에서 정보를 전달하는 방법이 하나 더 있다.

바로 환경변수(Environment Variable)이다.

1.7.1 환경변수란 무엇인가?

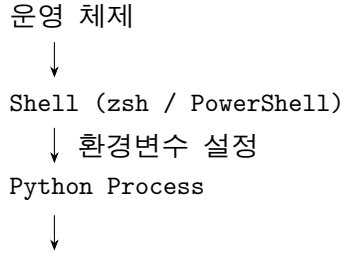
환경변수는 운영체제가 프로세스에 제공하는 **이름과 값으로 구성된 설정 정보**이다.

예를 들어:

```
MODEL_NAME=distilbert-base-uncased
BATCH_SIZE=32
DATA_DIR=/Users/user/project/data
```

이런 형태로 구성된다.

중요한 것은 환경변수가 Python만의 기능이 아니라는 것이다.



Python 프로그램은 실행될 때 전달받은 환경변수들을 읽어서 사용할 수 있다.

1.7.2 Python에서 환경변수 읽기

Python에서는 os 모듈을 사용한다.

```
import os

value = os.environ.get("MODEL_NAME")

print(value)
```

여기서:

```
os.environ
```

은 현재 프로세스의 환경변수에 접근할 수 있는 매핑 객체이다.

딕셔너리와 비슷하게 사용할 수 있다.

```
os.environ["MODEL_NAME"]
```

또는:

```
os.environ.get("MODEL_NAME")
```

으로 값을 가져올 수 있다.

다만 차이가 있다.

```
os.environ["MODEL_NAME"]
```

은 해당 환경변수가 없으면 KeyError가 발생한다.

반면:

```
os.environ.get("MODEL_NAME")
```

은 없으면 기본적으로 None을 반환한다.

딕셔너리의 접근 방식과 비슷하다.

1.7.3 환경변수의 값은 문자열이다

여기서 sys.argv와 공통점이 등장한다.

환경변수도 기본적으로 **문자열 형태로 다룬다**는 것이다.

예를 들어:

```
BATCH_SIZE=32
```

라는 환경변수가 있다고 하자.

```
import os

batch_size = os.environ("BATCH_SIZE")

print(batch_size)
print(type(batch_size))
```

결과:

```
32
<class 'str'>
```

즉 숫자처럼 보이더라도 실제로는 문자열이다.

그래서 ML 학습에 사용하려면:

```
batch_size = int(os.environ.get("BATCH_SIZE", "32"))
```

처럼 변환해야 한다.

여기서 "32"는 환경변수가 없을 때 사용하는 기본값이다.

1.7.4 macOS에서 환경변수 설정하기

터미널에서:

```
export MODEL_NAME=distilbert-base-uncased
```

를 실행해보자.

이 명령은 현재 셸 환경에서 MODEL_NAME이라는 변수를 설정하고,

이후 실행하는 자식 프로세스에 전달할 수 있도록 한다.

그다음:

```
python train.py
```

를 실행하면 Python 프로세스에서:

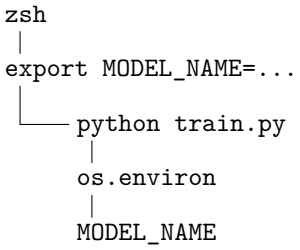
```
import os

print(os.environ.get("MODEL_NAME"))
```

을 통해 해당 값을 읽을 수 있다.

```
distilbert-base-uncased
```

여기서 중요한 것은 export가 **Python 코드가 아니라 셸 명령**이라는 것이다.



이 구조다.

1.7.5 환경변수는 어디까지 유지될까?

이 부분이 중요하다.

터미널에서:

```
export BATCH_SIZE=32
```

를 실행했다고 하자.

그러면 해당 셸과 그 셸에서 실행하는 자식 프로세스에 적용된다.

하지만 터미널을 종료하면 일반적으로 그 셸에서 설정한 환경변수도 사라진다.

즉:

```
Terminal A
 |
export BATCH_SIZE=32
 |
 └── python train.py
       |
       └── BATCH_SIZE=32
```

반면 별도로 실행한:

```
Terminal B
 |
 └── python train.py
       |
       └── BATCH_SIZE가 없을 수 있음
```

이 되는 것이다.

환경변수는 컴퓨터 전체에 자동으로 영구 저장되는 설정이 아니다.

필요하다면 셸 설정 파일 등을 이용해서 새로운 셸을 시작할 때 설정하도록 만들 수 있다.

1.7.6 그렇다면 argparse와 환경변수는 무엇이 다를까?

앞에서 우리는:

```
python train.py --batch-size 32
```

를 배웠다.

이건 커맨드라인 인수이다.

환경변수는:

```
export BATCH_SIZE=32
python train.py
```

처럼 사용할 수 있다.

둘 다 외부에서 설정을 전달하지만 목적이 조금 다르다

구분	argparse	환경변수
전달방식	명령줄 인수	프로세스 환경
Python 접근	`arg.batch_size`	`os.environ.get()`
대표 용도	실행별 학습 설정	실행 환경 및 외부 서비스 설정
값의 기본 형태	문자열, `type`으로 변환 가능	문자열
설정 범위	해당 프로그램 호출	현재 프로세스 및 자식 프로세스 환경

예를 들어 ML 학습에서는:

```
python train.py \
    --epochs 10 \
    --batch-size 32 \
    --learning-rate 0.001
```

처럼 실험마다 달라지는 설정을 argparse로 전달하는 것이 자연스럽다.

그렇다면 어떤 상황에서 환경변수를 사용하는 것이 적절할까?

대표적인 사례는 세 가지다.

용도	환경변수 예시	설명
인증 정보	`HF_TOKEN`	Hugging Face 접근 토큰
데이터베이스 연결	`DATABASE_URL`	PostgreSQL 연결 정보
실행 환경 설정	`APP_ENV`	개발,테스트,운영 환경 구분

여기서 중요한 차이는 다음과 같다.

argparse는 특정 프로그램 실행에 필요한 인수를 명시적으로 전달할 때 적합

하고,

환경변수는 프로그램이 실행되는 환경에 따라 달라지는 설정을 제공할 때 유용하다.

물론 반드시 이렇게 구분해야 하는 것은 아니다.

예를 들어 `BATCH_SIZE`는 `argparse`로 전달해도 되고 환경변수로 전달해도 된다.

다만 프로그램을 설계할 때 설정의 성격에 따라 적절한 방법을 선택하는 것이다.

예시: Hugging Face 토큰

Hugging Face 모델을 다운로드하면서 인증 문제를 겪었다고 해보자.

다음과 같이 토큰을 코드에 직접 작성할 수도 있다.

```
token = "hf_xxxxxxxxxxxxxx"
```

하지만 이런 방식은 권장하지 않는다.

코드를 GitHub에 올릴 때 토큰까지 공개될 위험이 있기 때문이다.

대신 환경변수를 사용할 수 있다.

macOS 터미널에서:

```
export HF_TOKEN="실제_토큰값"
```

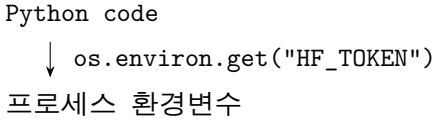
Python에서는:

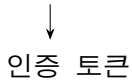
```
import os

token = os.environ.get("HF_TOKEN")
```

이렇게 접근한다.

여기서 중요한 점은 **Python 코드와 인증 정보를 분리했다는 것**이다.





이렇게 하면 Python 소스코드에 실제 토큰을 기록할 필요가 없다.

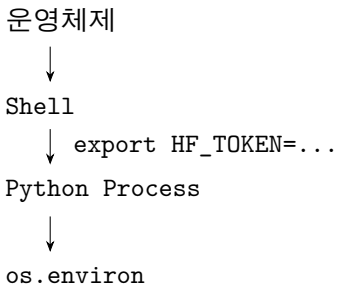
단, **환경변수가 토큰을 완벽하게 보호하는 보안 저장소는 아니다.**

셸 기록이나 프로세스 환경 노출 등의 위험은 여전히 존재하므로 실제 운영 환경에서는 비밀 관리 도구를 사용하기도 한다.

1.7.7 환경변수는 프로세스 단위로 관리된다

이 부분은 조금 더 정확하게 이해할 필요가 있다.

앞에서 배운 환경변수 구조를 다시 보자.



여기서 핵심은 각 프로세스가 자신의 환경변수 집합을 가진다는 것이다.

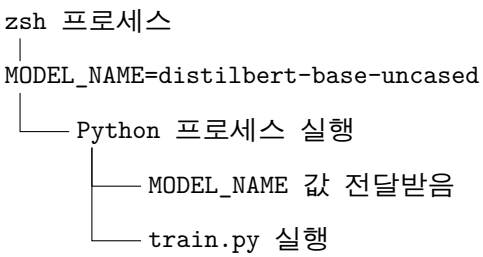
그리고 일반적으로 새로운 프로세스가 생성될 때 부모 프로세스의 환경변수를 전달받을 수 있다.

예를 들어:

```
export MODEL_NAME=distilbert-base-uncased
python train.py
```

라고 실행했다고 하자.

이때 구조는 다음과 같다.



이제 Python 코드에서:

```
import os

print(os.environ["MODEL_NAME"])
```

을 실행하면 값을 읽을 수 있다.

그렇다면 Python에서 환경변수를 변경하면 어떻게 될까?

다음 코드를 보자.

```
import os

os.environ["MODEL_NAME"] = "bert-base-uncased"

print(os.environ["MODEL_NAME"])
```

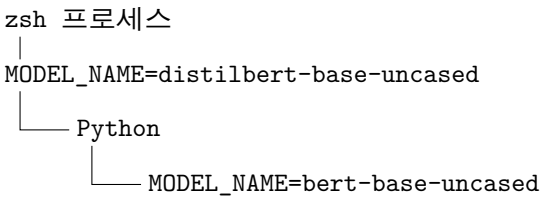
출력:

```
bert-base-uncased
```

이렇게 Python에서도 환경변수를 변경할 수 있다.

하지만 여기서 중요한 점이 있다.

Python 프로세스에서 환경변수를 변경했다고 해서 부모 프로세스의 zsh의 환경변수까지 변경되지는 않는다.



즉 환경변수는 하나의 전역 딕셔너리를 모든 프로세스가 공유하는 구조가 아니다.

이 점은 이전에 배운 **객체의 참조와 재바인딩**을 떠올리면 이해하기 쉽다.

다만 환경변수의 전달은 Python 객체의 참조 공유와는 다르다는 점도 기억해야 한다.

새로운 프로세스는 환경변수의 값을 전달받는 것이지,

부모 프로세스의 `os.environ` 객체를 공유하는 것이 아니다.

1.7.8 환경변수와 PATH, sys.path는 무엇이 다른가?

이전에 Python 실행 환경과 import를 공부할 때 다음을 배웠다.

```
import sys

print(sys.path)
```

그런데 운영체제에도 PATH라는 환경변수가 있다.

이름이 비슷해서 혼동하기 쉽다.

1. **PATH 환경변수**

macOS 터미널에서:

```
echo PATH
```

를 실행해보면 다음과 비슷한 결과가 나올 수 있다.

```
/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin
```

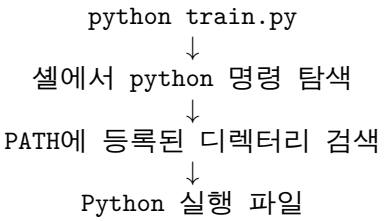
이것은 셸이 실행할 프로그램을 찾을 때 사용하는 경로 목록이다.

예를 들어:

```
python train.py
```

를 실행하면 셸은 python이라는 명령에 해당하는 실행 파일을 찾아야 한다.

이때 PATH 환경변수가 사용될 수 있다.



정확히는 셸의 명령 탐색에는 별칭, 함수, 내장 명령 등도 관여하지만,

외부 실행 파일을 찾는 데 PATH가 중요한 역할을 한다고 이해하면 된다.

2. **sys.path**

반면:

```
import sys

print(sys.path)
```

는 Python이 모듈을 import할 때 참조하는 검색 경로 목록이다.

예를 들어:

```
import pandas
```

를 실행하면 Python은 sys.path에 포함된 위치 등을 바탕으로 Pandas 패키지를 찾는다.

3. **둘의 차이**

구분	**PATH**	**sys.path**
소속	운영체제의 환경변수	Python 런타임의 속성
목적	실행 파일 검색	Python 모듈, 패키지 검색
접근	os.environ["PATH"]	sys.path
주요 대상	python, git 등	numpy, pandas 등

여기서 한 가지를 더 추가하자.

4. **PYTHONPATH**

Python에는 PYTHONPATH라는 환경변수도 있다.

```
export PYTHONPATH="/Users/user/my_project"
```

이 환경변수는 Python의 모듈 검색 경로를 구성할 때 추가 경로를 제공할 수 있다.

관계를 정리하면:

```
PATH
→ 운영체제가 실행 파일을 찾는 데 사용


PYTHONPATH
→ Python 모듈 검색 경로에 추가 경로 제공


sys.path
→ 실행 중인 Python이 실제로 사용하는 모듈 검색 경로 목록
```

즉 PYTHONPATH와 sys.path는 관련이 있지만 같은 것은 아니다.

그리고 일반적인 프로젝트에서는 PYTHONPATH를 무분별하게 설정하기보다는

패키지 구조를 올바르게 구성하고 python -m 등을 사용하는 것이 좋다.

1.7.9 Conda 환경과 환경변수의 관계

이번에는 Conda 환경과 연결해보자.

예를 들어 macOS에서:

```
conda activate transformer
```

를 실행했다고 하자.

그러면 터미널 프롬프트가:

```
(transformer) user@MacBook ~ %
```

처럼 변경된다.

그런데 단순히 화면에 (transformer)라는 문자열만 표시하는 것은 아니다.

Conda는 활성화 과정에서 현재 셸의 환경변수 등을 변경한다.

대표적으로:

```
PATH
CONDA_PREFIX
CONDA_DEFAULT_ENV
```

같은 환경변수가 관련된다.

Python에서 확인해보자.

```
import os
import sys

print(os.environ.get("CONDA_DEFAULT_ENV"))
print(os.environ.get("CONDA_PREFIX"))
print(sys.executable)
```

예상 결과는 다음과 비슷하다.

```
transformer
/Users/user/miniconda3/envs/transformer
/Users/user/miniconda3/envs/transformer/bin/python
```

설치 위치는 컴퓨터마다 다를 수 있다.

여기서 중요한 것은 세 값의 의미이다.

`CONDA_DEFAULT_ENV`
→ 현재 활성화된 Conda 환경의 이름을 나타낸다.

`CONDA_PREFIX`
→ 활성화된 Conda 환경의 서릿 경로를 나타낸다.

`sys.executable`
→ 현재 실행 중인 Python 인터프리터의 실제 경로를 나타낸다.

이 세 가지는 관련이 있지만 동일한 개념은 아니다.

특히 VS Code나 Jupyter에서는 Conda 환경의 활성화 상태와 실제 Python 커널이 서로 다를 수 있다.

따라서 실제로 어떤 Python이 실행되고 있는지 확인하려면:

```
import sys

print(sys.executable)
```

을 확인하는 것이 가장 직접적이다.

1.7.10 .env파일은 무엇인가?

실제 Python 프로젝트에서는 환경변수를 매번 터미널에 입력하는 것이 번거로울 수 있다.

그래서 .env파일을 사용하기도 한다.

예를 들어 프로젝트 구조가 다음과 같다고 하자.

```
my_project/
├── .env
├── train.py
└── requirements.txt
```

.env파일 내용:

```
MODEL_NAME=distilbert-base-uncased
BATCH_SIZE=32
HF_TOKEN=your_token_here
```

그런데 여기서 중요한 사실이 있다.

.env파일은 Python이 자동으로 읽어주는 특별한 파일이 아니다.

단순히 환경변수 설정을 기록해둔 텍스트 파일이다.

이 파일을 Python 프로세스의 환경변수로 반영하려면 별도의 처리가 필요하다.

대표적으로 python-dotenv 라이브러리를 사용할 수 있다.

설치:

```
pip install python-dotenv
```

사용:

```
import os
from dotenv import load_dotenv

load_dotenv()

model_name = os.environ.get("MODEL_NAME")
batch_size = int(os.environ.get("BATCH_SIZE", "32"))

print(model_name)
print(batch_size)
```

여기서:

```
load_dotenv()
```

는 .env파일을 찾아 읽고 그 값을 프로세스 환경변수에 반영하는 역할을 한다.

기본 설정에서는 이미 존재하는 환경변수 값을 덮어쓰지 않는다.

주의할 점

.env파일에 실제 API 토큰이나 비밀번호가 들어 있다면

Github에 올리지 않도록 주의해야 한다.

일반적으로 .gitignore에:

```
.env
```

를 추가한다.

다만 .gitignore는 아직 추적되지 않은 파일을 Git이 무시하도록 하는 설정이므로,

이미 커밋한 비밀정보까지 자동으로 제거하지는 않는다.

1.7.11 argparse + 환경변수 + pathlib 실전 예제

이제 지금까지 배운 내용을 하나의 코드로 연결해보자.

상황은 다음과 같다.

- 모델 이름은 환경변수로 관리한다.

- 배치 크기는 CLI 인수로 전달한다.

- 결과 저장 디렉터리도 CLI 인수로 전달한다.

```
import os
import argparse
from pathlib import Path

# 1. CLI 인수 정의
parser = argparse.ArgumentParser()

parser.add_argument(
    "--batch-size",
    type=int,
    default=32
)

parser.add_argument(
    "--save-dir",
    type=Path,
    default=Path("results/run1")
)

args = parser.parse_args()

# 2. 환경변수 읽기
model_name = os.environ.get(
    "MODEL_NAME",
    "distilbert-base-uncased"
)

# 3. 저장 디렉터리 생성
args.save_dir.mkdir(
    parents=True,
    exist_ok=True
)

# 4. 설정 출력
print("Model:", model_name)
print("Batch size:", args.batch_size)
print("Save directory:", args.save_dir)
```

macOS 터미널에서:

```
export MODEL_NAME=bert-base-uncased

python train.py \
    --batch-size 16 \
```

```
    --save-dir results/experiment1
```

라고 실행하면:

```
Model: bert-base-uncased
Batch size: 16
Save directory: results/experiment1
```

이 출력된다.

1.7.12 핵심 정리

환경변수는 프로그램 외부에서 실행 환경에 필요한 설정을 전달하는 방법이다.

이번 장에서 특히 중요한 것은 다음 다섯 가지다.

1. `os.environ`은 현재 Python 프로세스의
   환경변수에 접근하는 매핑 객체다.

2. 환경변수는 기본적으로 문자열이며,
   숫자로 사용하려면 형 변환이 필요하다.

3. 부모 프로세스에서 자식프로세스로 환경변수가 전달될 수 있지만,
   자식의 변경이 부모에게 자동으로 반영되지는 않는다.

4. `PATH`, `PYTHONPATH`, `sys.path`는 서로 다른 개념이다.

5. `.env`는 환경변수 설정을 기록한 파일이며,
   Python이 자동으로 읽는 것은 아니다.

1.7.13 번외. 바인딩과 매핑

이 내용은 변수가 객체를 참조하는 바인딩의 개념과,

키-밸류 구조의 매핑의 개념에 대한 설명으로 건너뛰어도 좋다.

1. **Binding - 이름과 객체의 연결**

```
x = 10
x = 20
```

Python에서는 이름 x가 어떤 객체를 참조하는지가 핵심이다.

x → int(10)

재바인딩 후

x → int(20)

즉, **binding**은 이름과 객체 사이의 연결이고,

rebinding은 기존 이름을 다른 객체에 연결하는 작업이다.

2. **Mapping - 키와 값의 대응 관계**

```
config = {"epochs": 10}
config["epochs"] = 20
```

여기에는 두 관계가 동시에 존재한다.

1. Binding
   config → dict 객체

2. Mapping
   dict 객체 내부
   "epochs" → 10

두 번째 대입 이후에는:

config → 동일한 객체

dict 객체 내부

"epochs" → 20

이 되는 것이다.

여기서 중요한 점은 **mapping은 키를 통해 값을 찾는 추상적인 대응 관계**고,

mutation은 객체의 상태를 변경하는 행위라는 것이다.

따라서 mapping과 mutation은 서로 대립하는 개념이 아니다.

- mapping : 객체가 어떤 방식으로 키와 값을 대응시키는가?
- mutation : 기존 객체의 상태가 변경되는가?
- rebinding : 이름이 참조하는 객체가 변경되는가?

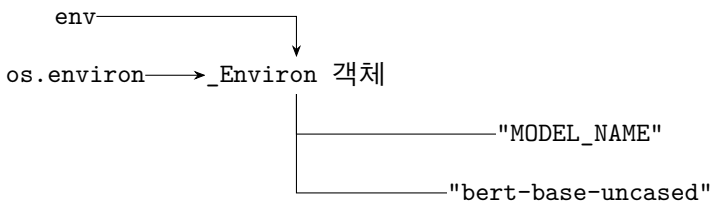
서로 다른 질문에 답하는 개념이다.

os.environ에 적용하면:

```
import os

env = os.environ
env["MODEL_NAME"] = "bert-base-uncased"
```

이 코드에서는 다음과 같은 일이 일어난다.



이후:

```
env["MODEL_NAME"] = "distilbert-base-uncased"
```

를 실행하면:

- env 라는 이름은 여전히 같은 _Environ 객체를 참조한다.
- _Environ 객체 내부의 "MODEL_NAME" 항목이 변경된다.
- 그 변경이 현재 프로세스의 환경변수에도 반영된다.

즉 매핑 객체의 mutation이 발생한 것이지,

env라는 이름의 rebinding이 발생한 것은 아니다.

참고로 매핑 객체가 항상 mutable인 것은 아니다.

읽기 전용 매핑도 존재하므로,

mapping이라는 자료구조의 성질과

mutation이 가능한지 여부도 별개이다.

최종 정리

내용을 수식처럼 표현하면 다음과 같다.

Binding : Name → Object

Mapping : Key → Value

Rebinding : Name → Another Object

Mutation : Object State → Modified State

결국 os.environ을 매핑 객체라고 부른 이유는

키와 값의 대응 관계를 제공하기 때문이고,

os.environ["MODEL_NAME"] = ...을 mutation이라고 설명한 이유는

기존 객체의 상태를 변경하기 때문이다.

1.8 프로세스와 외부 프로그램

1.8.1 프로그램(Program)과 프로세스(Process)

가장 먼저 구분해야 할 개념은 프로그램과 프로세스이다.

다음 파일을 생각해 보자.

```
# train.py
print("Training started")
```

이 파일은 디스크에 저장된 Python 소스 코드이다.

아직 실행하지 않았으므로 train.py 자체는 프로세스가 아니다.

이제 터미널에서 실행해 보자.

```
python train.py
```

이 명령이 실행되는 과정을 단순화하면 다음과 같다.

```
Terminal
   ↓
Shell
   ↓ python train.py
Operating System
   ↓ Python 실행 요청
Python Process
   ↓ Python Interpreter
train.py 실행
```

여기서 중요한 점은 다음과 같다.

- 프로그램(Program) : 실행 가능한 명령이나 코드가 저장된 것
- 프로세스(Process) : 운영체제에서 실행 중인 프로그램의 인스턴스

따라서 train.py는 소스 파일이고,

이를 실행하는 Python 인터프리터가 프로세스로 동작하는 것이다.

정확하게 말하면

python train.py는 Python 인터프리터 프로그램을 실행하고,

그 인터프리터가 train.py를 읽어서 실행하는 구조이다.

같은 프로그램을 두 번 실행하면?

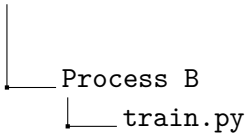
터미널을 두 개 열어서 각각 실행한다고 생각해 보자.

```
python train.py
```

```
python train.py
```

그러면 일반적으로 두 개의 독립적인 Python 프로세스가 생성된다.

```
Python Interpreter
└── Process A
    └── train.py
```



두 프로세스는 같은 소스 파일을 실행하지만,

서로 다른 실행 상태를 가지고 있다.

예를 들어 각각 독립적인 변수와 Python 객체를 생성할 수 있다.

```
# train.py

epochs = 10
epochs += 1

print(epochs)
```

두 프로세스에서 실행해도 각각 11이 출력된다.

왜냐하면 두 프로세스의 epochs가

동일한 Python 객체를 공유하는 것이 아니기 때문이다.

물론 프로세스끼리 데이터를 주고받는 방법도 있다.

하지만 이를 위해서는 별도의 통신 또는 공유 메커니즘이 필요하다.

1.8.2 프로세스에는 무엇이 존재할까?

프로세스를 단순히 실행 중인 코드라고 생각하면 중요한 부분을 놓칠 수 있다.

운영체제는 프로세스를 관리하기 위해 여러 정보를 유지한다.

대표적으로 다음과 같은 것들이 있다.

구성 요소	설명
PID	프로세스를 식별하는 번호
Memory	프로세스가 사용하는 메모리 공간
Environment Variables	프로세스가 전달받은 환경변수
CWD	현재 작업 디렉터리
Arguments	실행할 때 전달받은 인수
stdin	표준 입력
stdout	표준 출력
stderr	표준 오류 출력
Exit Code	프로세스의 종료 상태

익숙한 개념들이 보인다.

우리는 이미 다음을 배웠다.

```
import os
import sys
from pathlib import Path

print(os.environ)
print(Path.cwd())
print(sys.argv)
print(sys.executable)
```

이 코드들이 서로 무관한 기능을 제공하는 것이 아니다.

현재 실행 중인

Python 프로세스의 실행 환경과 상태를 서로 다른 관점에서 조회하는 것

이다.

sys.executable은 현재 Python 인터프리터의 실행 파일 경로를 나타내고,

sys.argv는 Python 프로그램에 전달된 인수를 나타낸다는 차이가 있다.

기존에 배웠던 개념들이 하나의 프로세스라는 단위로 연결되기 시작한다.

1.8.3 os와 sys의 차이

지금까지 os와 sys를 여러 번 사용했다.

그런데 둘의 역할이 비슷해 보여서 오해할 수 있다.

> sys - Python 실행 환경과 인터프리터
>
> sys는 Python 인터프리터와 실행 환경에 관련된 기능을 제공한다.

```
import sys

print(sys.executable)
print(sys.version)
print(sys.platform)
print(sys.argv)
print(sys.path)
```

각각 다음을 의미한다.

속성	의미
sys.executable	현재 인터프리터 실행 파일 경로
sys.version	Python 버전 정보
sys.platform	실행 플랫폼 식별
sys.argv	커맨드라인 인수
sys.path	모듈 탐색 경로

> os - 운영체제와 상호작용
>
> os는 운영체제가 제공하는 기능과 상호작용하기 위한 인터페이스다.

```
import os

print(os.getcwd())
print(os.environ)
print(os.getpid())
```

함수, 속성	의미
os.getcwd()	현재 작업 디렉터리
os.environ	현재 프로세스의 환경변수
os.getpid()	현재 프로세스의 PID

예를 들어 다음 코드를 실행해 보자.

```python
import os

print(os.getpid())
```

출력은:

```
42817
```

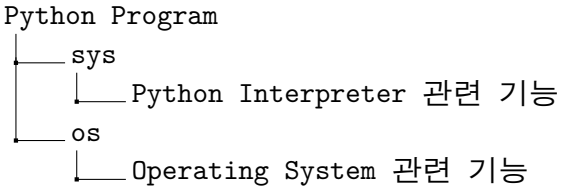
와 같을 수 있다.

여기서 42817은 운영체제가 해당 프로세스에 부여한 PID이다.

같은 코드를 새 Python 프로세스에서 실행하면 일반적으로 다른 PID가 출력된다.

그렇다면 왜 os와 sys가 분리되어 있을까?

역할을 구분하기 위해서이다.



물론 두 모듈의 기능이 완전히 배타적인 것은 아니다.

예를 들어 둘 다 프로세스 종료와 관련된 기능을 제공한다.

따라서 이렇게 이해하는 것이 좋다.

sys는 Python 인터프리터 관점

os는 운영체제 인터페이스 관점

에 초점을 맞춘 모듈이다.

1.8.4 부모 프로세스와 자식 프로세스

여기서부터는 이전에 배운 환경변수와 직접 연결된다.

터미널에서 다음을 실행했다고 생각해 보자.

```
python train.py
```

일반적인 터미널 환경에서는 Shell이 Python 프로세스를 실행한다.

```
Shell Process
   ↓ python train.py
Python Process
   │
train.py
```

이때 **Shell을 부모 프로세스(Parent Process)**,

Python을 **자식 프로세스(Child Process)**라고 부를 수 있다.

자식 프로세스는 생성될 때 부모가 설정해 전달한 환경변수 등을 받는다.

예를 들어 macOS에서 다음과 같이 실행하면:

```
export MODEL_NAME=bert-base-uncased

python train.py
```

Python 코드에서 다음을 읽을 수 있다.

```
import os

print(os.environ.get("MODEL_NAME"))
```

출력:

```
bert-base-uncased
```

이전에 배웠던 환경변수 전달이 실제로는 이러한 프로세스 생성 과정에서 이루어진다.

단, 환경변수가 부모 프로세스와 자식 프로세스 사이에서 자동으로 실시간 동기화되는 것은 아니다.

자식 프로세스는 생성될 때 전달받은 환경을 바탕으로 자신의 환경변수를 관리한다.

1.8.5 Python에서 다른 프로그램을 실행할 수 있을까?

가능하다.

지금까지는 Shell에서 Python을 실행했다.

```
Shell
└── Python
```

그런데 Python에서도 새로운 프로세스를 실행할 수 있다.

```
Shell
└── Python
    └── External Program
```

이때 대표적으로 사용하는 모듈이 바로 `subprocess`이다.

예를 들어:

```
import subprocess

subprocess.run(["python", "--version"])
```

과 같은 코드가 있다.

이 코드는 Python 프로그램 안에서 외부 명령을 실행한다.

여기서 `subprocess.run()`에 전달한 리스트를 살펴보자.

```
["python", "--version"]
```

- `"python"` : 실행할 프로그램
- `"--version"` : 해당 프로그램에 전달할 인수

즉, 터미널에서 실행하는:

```
python --version
```

과 비슷한 작업을 Python 코드 안에서 수행하는 것이다.

다만 중요한 차이가 있다.

`subprocess.run()`은 기본적으로 Shell을 거치지 않고 프로그램을 직접 실행한다.

그리고 `"python"`이라는 이름은 실행 환경의 프로그램 검색 규칙에 따라 해석되므로,

반드시 현재 Python 프로세스와 같은 인터프리터를 실행한다고 보장할 수 없다.

현재 사용 중인 Python 인터프리터를 정확하게 실행하고 싶다면 다음처럼 작성하는 것이 좋다.

```
import subprocess
import sys

subprocess.run([sys.executable, "--version"])
```

이렇게 하면 `sys.executable`이 가리키는 인터프리터를 사용하게 된다.

특히 Conda 환경을 사용한다면 중요한 부분이다.

1.8.6 `subprocess.run()`의 기본 실행 흐름

다음 코드를 보자:

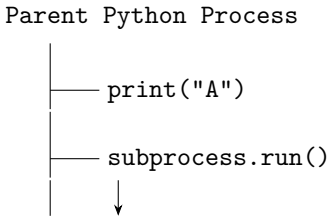
```
import subprocess
import sys

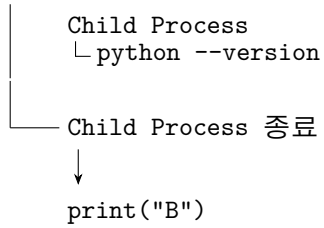
print("A")

subprocess.run([sys.executable, "--version"])

print("B")
```

실행 흐름은 다음과 같다.





일반적인 subprocess.run()은 자식 프로세스가 종료될 때까지 기다린다.

따라서 출력 순서는 다음과 같다.

```
A
Python 3.x.x
B
```

단, Jupyter 등에서 실행하면 출력 캡쳐 방식에 따라 화면에 표시되는 순서는 다르게 보일 수 있다.

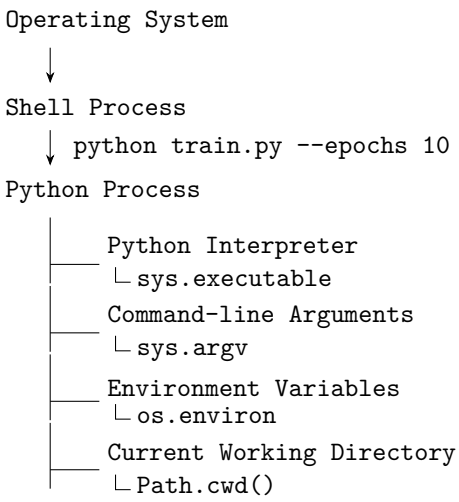
여기서 가장 중요한 개념은 다음이다.

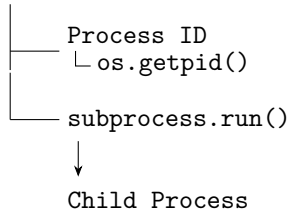
> subprocess.run()은 외부 프로그램을 실행하고,
>
> 해당 프로그램이 종료될 때까지 기다린 뒤 결과를 반환한다.

반대로 자식 프로세스를 실행한 뒤 종료를 기다리지 않고

다른 작업을 진행하고 싶다면 subprocess.Popen() 등을 사용할 수 있다.

여기까지의 핵심 연결





이 구조가 8장의 출발점이다.

앞으로 배울 내용

1. subprocess.run()의 반환값과 CompletedProcess
2. stdin, stdout, stderr의 의미
3. returncode와 exit code
4. sys.exit()와 os._exit()의 차이
5. Shell의 파이프 |와 리다이렉션 >
6. subprocess.run()에서 출력 캡처하기
7. shell=True의 의미와 주의점
8. subprocess.Popen()과 비동기적 프로세스 실행
9. ML/DL 프로젝트에서 외부 학습 스크립트 실행하기

1.8.7 subprocess.run()과 CompletedProcess

1. subprocess.run()은 무엇을 반환할까?

먼저 다음 코드를 살펴보자:

```
import subprocess
import sys

result = subprocess.run(
    [sys.executable, "--version"]
)

print(result)
```

예를 들어 다음과 같은 결과가 나올 수 있다:

```
Python 3.12.7
CompletedProcess(args=['.../python', '--version'], returncode=0)
```

여기서 두 가지가 출력되었다.

첫 번째는 자식 프로세스가 출력한 Python 버전이다.

```
Python 3.12.7
```

두 번째는 `subprocess.run()`이 반환한 객체를 출력한 결과다.

```
CompletedProcess(args=['.../python', '--version'], returncode=0)
```

즉, `subprocess.run()`은 단순히 외부 프로그램을 실행하는 것으로 끝나지 않는다.

외부 프로그램의 실행이 완료되면,

실행 결과에 관한 정보를 담은 `CompletedProcess` 객체를 반환한다.

이 구조를 이전에 배운 함수의 관점에서 생각해 보자.

```
result = subprocess.run(...)
```

여기서:

- `subprocess.run` 은 함수 객체를 참조한다.
- `subprocess.run(...)`은 함수를 호출한다.
- 함수가 반환한 `CompletedProcess` 객체가 `result` 라는 이름에 바인딩된다.

`Path` 객체나 `requests.Response` 객체와 비슷하게 이해하면 된다.

2. `CompletedProcess` 객체의 주요 속성

`CompletedProcess` 객체에는 대표적으로 다음 속성이 있다.

속성	의미
`args`	실행한 프로그램과 인수
`returncode`	자식 프로세스의 종료 상태
`stdout`	자식 프로세스의 표준 출력 (캡쳐하도록 설정한 경우)
`stderr`	자식 프로세스의 표준 오류 출력 (캡쳐하도록 설정한 경우)

예제를 보자:

```
import subprocess
import sys

result = subprocess.run(
    [sys.executable, "--version"],
    capture_output=True,
    text=True
)

print("args:", result.args)
print("returncode:", result.returncode)
print("stdout:", result.stdout)
print("stderr:", result.stderr)
```

예상 출력은 다음과 같다:

```
args: ['/path/to/python', '--version']
returncode: 0
stdout: Python 3.12.7
stderr:
```

여기서 새로운 인수가 등장했다.

```
(
    capture_output=True
    text=True
)
```

이 두 인수의 의미를 이해하려면

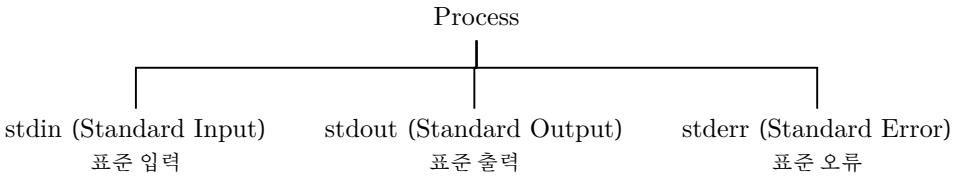
먼저 운영체제의 **표준 입출력(Standard I/O)** 개념을 알아야 한다.

1.8.8 표준 입력과 표준 출력

프로세스는 외부와 데이터를 주고받을 수 있다.

그런데 프로세스마다 데이터를 입력받고 출력하는 방식을 완전히 다르게 정의한다면, 프로그램을 연결하기가 매우 어려워진다.

그래서 운영체제는 일반적으로 다음과 같은 표준 입출력 통로를 제공한다.



각각을 자세히 살펴본다.

```
stdin - Standard Input
```

프로세스가 데이터를 입력받는 통로이다.

가장 익숙한 예시는 `input()` 함수이다.

```
name = input("이름을 입력하세요: ")
print(name)
```

터미널에서 실행하면 다음과 같이 동작한다.

```
이름을 입력하세요: Kim
Kim
```

이때 일반적으로 키보드에서 입력한 데이터가 터미널을 통해 프로세스 `stdin`으로 전달된다.

다만 `stdin`이 반드시 키보드여야 하는 것은 아니다.

파일이나 다른 프로세스에서 데이터를 전달받을 수도 있다.

이것이 뒤에서 배울 파이프와 연결된다.

```
stdout - Standard Output
```

프로세스가 일반적인 실행 결과를 출력하는 통로이다.

```
print("Training started")
```

우리가 사용하는 print()는 기본적으로 sys.stdout에 데이터를 출력한다.

```
import sys

print("Hello")

sys.stdout.write("World\n")
```

결과:

```
Hello
World
```

두 코드 모두 표준 출력으로 데이터를 보낸다는 점에서 비슷하다.

여기서 중요한 사실은 다음과 같다.

stdout은 터미널 화면 자체가 아니라 출력 데이터를 전달하는 통로이다.

보통 터미널에서 실행하면 stdout이 터미널에 연결되어 있어서 화면에 출력되는 것이다.

다른 프로그램이나 파일에 연결할 수도 있다.

```
stderr - Standard Error
```

오류 메시지나 진단 정보를 출력하는 별도의 통로이다.

예를 들어 다음 코드를 보자.

```
import sys

print("Training started")

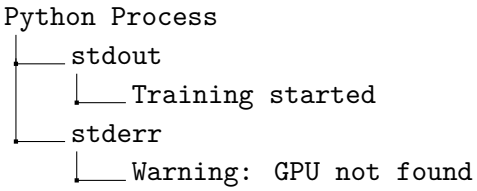
print(
    "Warning: GPU not found",
    file=sys.stderr
)
```

일반적인 터미널에서는 다음과 같이 출력될 수 있다.

```
Training started
Warning: GPU not found
```

화면만 보면 둘 다 동일한 방식으로 출력된 것처럼 보인다.

하지만 내부적으로는 서로 다른 통로를 사용했다.



왜 굳이 분리할까?

예를 들어 ML 모델의 예측 결과를 다른 프로그램에 전달한다고 생각해 보자.

```
Prediction: 0.85
Warning: GPU not found
```

두 종류의 출력이 섞이면 다른 프로그램이 예측 결과를 해석하기 어려울 수 있다.

그래서 정상적인 결과와 오류·진단 정보를 분리할 수 있도록 한 것이다.

단, 모든 프로그램이 이 관례를 엄격하게 지키는 것은 아니다.

stderr에 출력되었다고 반드시 실행 실패를 의미하지는 않는다.

1.8.9 capture_output=True의 의미

이제 다시 subprocess.run()으로 돌아가 보자.

다음 두 코드를 비교해 보자.

첫 번째 코드:

```
import subprocess
import sys

result = subprocess.run(
    [sys.executable, "--version"]
)

print(result.stdout)
```

예상 결과:

```
Python 3.12.7
None
```

왜 None이 출력될까?

자식 프로세스가 버전 정보를 출력하기는 했지만,

부모 Python 프로세스가 그 출력을 별도로 수집하지 않았기 때문이다.

기본적으로 자식 프로세스의 출력은 부모가 연결해 준 표준 출력 대상으로 전달된다.

따라서 터미널에서는 출력이 보이지만, result.stdout에 저장되는 것은 아니다.

두 번째 코드:

```
import subprocess
import sys

result = subprocess.run(
    [sys.executable, "--version"],
    capture_output=True
)

print(result.stdout)
```

예상 결과:

```
b'Python 3.12.7\n'
```

이번에는 stdout이 None이 아니라 데이터로 반환되었다.

왜냐하면:

```
(
    capture_output=True
)
```

를 지정했기 때문이다.

이 옵션을 사용하면 subprocess.run()이

자식 프로세스의 stdout과 stderr를 캡처해 결과 객체에 저장한다.

그런데 출력에 b가 붙어 있다.

```
b'Python 3.12.7\n'
```

이것은 bytes 객체이다.

운영체제의 표준 입출력에서 전달되는 데이터는 기본적으로 바이트 단위이기 때문에,

subprocess도 기본적으로 캡처 결과를 bytes로 반환한다.

따라서 다음과 같이 디코딩할 수 있다.

```
print(result.stdout.decode("utf-8"))
```

하지만 매번 직접 디코딩하는 것은 번거롭다.

그래서 text=True를 제공한다.

1.8.10 text=True와 문자열 변환

다음 코드를 보자.

```
import subprocess
import sys

result = subprocess.run(
    [sys.executable, "--version"],
    capture_output=True,
    text=True
)

print(type(result.stdout))
print(result.stdout)
```

결과:

```
<class 'str'>
Python 3.12.7
```

text=True를 지정하면 subprocess가 입출력을 텍스트 모드로 처리한다.

캡처한 바이트를 문자열로 디코딩해서 반환한다.

파일 입출력	subprocess
open("data.txt", "rb")	기본 캡처 모드
open("data.txt", "r", encoding='utf-8')	text=True, encoding='utf-8'
bytes 반환	기본적으로 bytes 반환
str 반환	text=True이면 str 반환

다음처럼 인코딩을 명시할 수도 있다.

```
result = subprocess.run(
    [sys.executable, "--version"],
    capture_output=True,
    text=True,
    encoding="utf-8"
)
```

여기서 기억할 것은 단순하다.

```
    - capture_output=True : 자식 프로세스의 출력을 수집한다.

    - text=True : 입출력 데이터를 문자열로 처리한다.
```

두 옵션은 서로 다른 역할을 수행한다.

참고로 capture_output=True 없이 text=True만 지정해도

result.stdout이 자동으로 채워지는 것은 아니다.

1.8.11 종료 코드(Exit Code)와 returncode

이제 프로세스의 실행이 성공했는지 실패했는지 판단하는 방법을 알아보자.

외부 프로그램이 실행을 마치면 운영체제에 종료 상태를 전달할 수 있다.

이를 종료 코드(Exit Code)라고 부른다.

일반적으로 다음과 같은 관례를 사용한다.

종료 코드	의미
0	정상 종료
0이 아닌 값	오류 또는 특정 종료 상태

여기서 주의할 점은 종료 코드의 구체적인 의미가 프로그램마다 다를 수 있다는 것이다.

예를 들어 어떤 프로그램은 1을 일반적인 오류에 사용하고,

다른 프로그램은 특정 조건을 나타내는 데 사용할 수도 있다.

예제 1 - 정상 종료

```
import subprocess
import sys

result = subprocess.run(
    [sys.executable, "-c", "print('Hello')"],
    capture_output=True,
    text=True
)

print(result.stdout)
print(result.returncode)
```

결과:

```
Hello

0
```

여기서 새로운 인수 -c가 등장했다.

```
python -c "print('Hello')"
```

-c는 Python 인터프리터에게 뒤에 전달된 문자열을

Python 코드로 실행하라는 옵션이다.

즉, 별도의 .py 파일을 만들지 않고 간단한 Python 코드를 실행할 수 있다.

예제 2 - 비정상 종료

이번에는 의도적으로 오류를 발생시켜 보자.

```
import subprocess
import sys

result = subprocess.run(
    [
        sys.executable,
        "-c",
        "raise ValueError('Invalid value')"
    ],
    capture_output=True,
    text=True
)

print("returncode:", result.returncode)
print("stdout:", result.stdout)
print("stderr:", result.stderr)
```

출력은 대략 다음과 같다:

```
returncode: 1
stdout:
stderr: Traceback (most recent call last):
...
ValueError: Invalid value
```

여기서 중요한 부분이 있다.

자식 프로세스에서 `ValueError`가 발생했지만,

부모 Python 프로그램에서 `subprocess.run()`이 자동적으로 예외를 발생시키지 않았다.

대신 자식 프로세스의 종료 상태를 다음 속성에 저장했다.

```
result.returncode
```

즉, 자식 프로세스의 실패와

부모 Python 프로세스의 예외 발생은 서로 다른 개념이다.

이 차이를 이해하는 것이 매우 중요하다.

1.8.12 check=True와 CalledProcessError

그렇다면 자식 프로세스가 실패했을 때 부모 프로그램에서도 예외가 발생하도록 할 수 있을까?

가능하다.

```
import subprocess
import sys

result = subprocess.run(
    [
        sys.executable,
        "-c",
        "raise ValueError('Invalid value')"
    ],
    capture_output=True,
    text=True,
    check=True
)
```

여기서 추가된 옵션은 다음과 같다.

```
(
    check=True
)
```

이 옵션을 지정하면 자식 프로세스가 0이 아닌 종료 코드를 반환했을 때

subprocess.CalledProcessError 예외가 발생한다.

따라서 다음과 같이 처리할 수 있다.

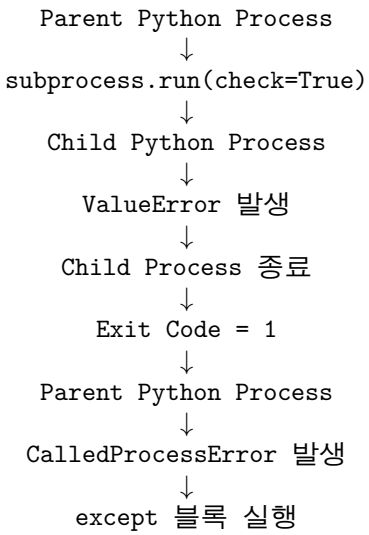
```
import subprocess
import sys

try:
    result = subprocess.run(
        [
            sys.executable,
            "-c",
            "raise ValueError('Invalid value')"
        ],
        capture_output=True,
        text=True,
        check=True
    )
```

```
except subprocess.CalledProcessError as e:
    print("실행 실패")
    print("종료 코드:", e.returncode)
    print("오류 메시지:", e.stderr)
```

여기서 예외 객체 e는 자식 프로세스의 실패에 관한 정보를 가지고 있다.

실행 흐름을 정리하면 다음과 같다.



중요한 점은 부모에서 발생한 예외가 자식의 ValueError 객체 자체는 아니다.

부모가 받은 것은

자식 프로세스의 비정상 종료를 나타내는 CalledProcessError다.

이 구조는 ML/DL 프로젝트에서 여러 학습 스크립트를 자동으로 실행할 때 특히 유용하다.

1.8.13 sys.exit()과 프로세스 종료

이제 프로세스의 종료 상태를 직접 지정하는 방법을 알아보자.

우리는 이미 sys 모듈을 배웠는데, sys에는 다음과 같은 함수도 있다.

```
sys.exit()
```

이 함수는 현재 Python 프로그램의 실행을 종료하도록 요청하는 역할을 한다.

예를 들어 다음 코드를 보자:

```
# train.py

import sys

print("Training started")

sys.exit(0)

print("Training finished")
```

실행 결과:

```
Training started
```

`sys.exit(0)` 이후의 코드는 실행되지 않는다.

여기서 `0`은 정상 종료를 나타내는 종료 코드로 사용된다.

이번에는 다음처럼 작성해 보자:

```
# train.py

import sys

print("Training started")

sys.exit(1)
```

이 프로그램을 `subprocess.run()`으로 실행하면 다음과 같다:

```
import subprocess
import sys

result = subprocess.run(
    [sys.executable, "train.py"]
)

print(result.returncode)
```

결과:

```
Training started
```

```
1
```

train.py가 sys.exit(1)로 종료되었기 때문에

부모 프로그램은 returncode에서 1을 확인할 수 있다.

> sys.exit()은 내부적으로 무엇을 할까?

사실 sys.exit()은 내부적으로 SystemExit 예외를 발생시킨다.

```
sys.exit(1)
```

는 개념적으로 다음과 유사하다:

```
raise SystemExit(1)
```

따라서 일반적인 예외처럼 처리될 여지가 있고, finally 블록도 실행된다.

```
import sys

try:
    print("A")
    sys.exit(1)

finally:
    print("B")
```

결과:

```
A
B
```

그 후 SystemExit이 처리되지 않은 채 최상위까지 전달되면 Python 인터프리터가 종료된다.

즉, sys.exit()은 운영체제에 프로세스를 즉시 강제 종료하라는 명령과는 다르다.

이 차이 때문에 일반적인 Python 프로그램에서는 sys.exit()을 사용하고,

os._exit()은 특별한 상황에서만 사용한다.

os._exit()은 Python의 일반적인 정리 절차를 거치지 않고 프로세스를

즉시 종료하기 때문에 보통의 학습, 응용 코드에서 사용할 필요는 없다.

1.8.14 실제 ML 프로젝트에서 활용하기

이제 지금까지 배운 내용을 실제 프로젝트에 연결해 보자.

다음과 같은 프로젝트가 있다고 가정하자.

```
ml-project/
├── train.py
├── run_experiment.py
├── data/
│   └── train.csv
└── results/
```

train.py는 학습을 수행하는 스크립트다.

```python
# train.py

import argparse
import sys
from pathlib import Path

parser = argparse.ArgumentParser()

parser.add_argument(
    "--data",
    type=Path,
    required=True
)

parser.add_argument(
    "--epochs",
    type=int,
    default=10
)

args = parser.parse_args()

if not args.data.is_file():
    print(
        f"Data file not found: {args.data}",
        file=sys.stderr
    )

    sys.exit(1)

print(f"Data: {args.data}")
```

```
print(f"Epochs: {args.epochs}")

print("Training completed")
```

이 코드는 지금까지 배운 개념들로 구성되어 있다.

- `argparse` : 커맨드 라인 인수 처리
- `Path` : 파일 경로 표현
- `is_file()` : 파일 존재 여부 확인
- `sys.stderr` : 오류 메시지 출력
- `sys.exit(1)` : 오류 상태로 종료

이번에는 `run_experiment.py`를 작성해 보자:

```
# run_experiment.py

import subprocess
import sys
from pathlib import Path

project_root = Path(__file__).resolve().parent

train_script = project_root / "train.py"
data_path = project_root / "data" / "train.csv"

command = [
    sys.executable,
    str(train_script),
    "--data",
    str(data_path),
    "--epochs",
    "10"
]

result = subprocess.run(
    command,
    capture_output=True,
    text=True,
    encoding="utf-8"
)

print("Exit code:", result.returncode)
print("Standard output:")
```

```
print(result.stdout)

print("Standard error:")
print(result.stderr)
```

여기서 눈여겨볼 부분은 다음이다.

```
command = [
    sys.executable,
    str(train_script),
    "--data",
    str(data_path),
    "--epochs",
    "10"
]
```

이전에 배운 커맨드라인 인수가 등장했다.

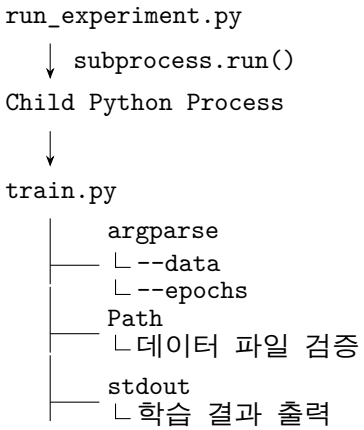
subprocess.run()의 리스트는 실행 프로그램과 각 인수를 구분해서 전달하는 역할을 한다.

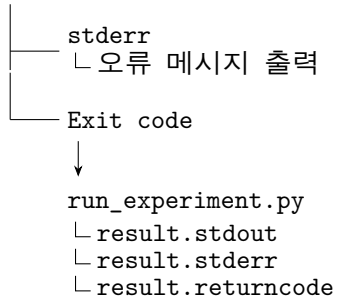
또한 "10"이 문자열인 이유도 이해할 수 있다.

커맨드라인 인수는 문자열로 전달되고, train.py의 argparse가 다음 설정을 통해 정수로 변환한다.

```
parser.add_argument(
    "--epochs",
    type=int
)
```

전체 흐름을 보면 다음과 같다.





이것이 Python으로 학습 실행을 자동화하는 가장 기본적인 구조이다.

물론 실제 모델 학습에서는 `subprocess.run()` 대신

하나의 Python 프로세스 안에서 학습 함수를 직접 호출하는 방식도 사용할 수 있다.

별도 프로세스를 실행하는 것이 항상 더 좋은 것은 아니고,

실행 환경 격리나 독립적인 스크립트 실행이 필요할 때 유용하다.

1.8.15 이번 학습 내용 정리

지금까지 살펴본 내용을 표로 정리해 보자.

개념	역할
`subprocess.run()`	외부 프로그램을 실행하고 종료를 기다림
`CompletedProcess`	실행 결과 정보를 담은 객체
`args`	실행한 명령과 인수
`stdin`	표준 입력
`stdout`	표준 출력
`stderr`	표준 오류 출력
`capture_output=True`	자식 프로세스의 출력을 캡처
`text=True`	입출력을 문자열로 처리
`returncode`	자식 프로세스의 종료 상태
`check=True`	비정상 종료 시 `CalledProcessError` 발생
`sys.exit()`	`SystemExit`를 발생시켜 프로그램 종료 요청
`sys.executable`	현재 Python 인터프리터의 실행 파일 경로

이번 장에서 가장 중요한 세 가지

첫 번째, 프로그램의 출력과 함수의 반환값은 서로 다르다

```
result = subprocess.run(...)
```

여기서 자식 프로세스가 출력하는 내용과

`subprocess.run()`이 반환하는 `CompletedProcess` 객체는 별개의 개념이다.

두 번째, 자식 프로세스의 실패와 부모 프로세스의 예외는 서로 다르다.

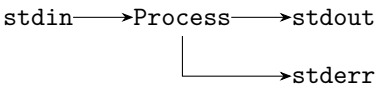
```
result.returncode
```

는 자식 프로세스의 종료 상태를 나타내고,

```
(
    check=True
)
```

는 그 상태를 부모 Python 프로그램의 예외 처리와 연결해주는 옵션이다.

세 번째, 표준 입출력은 터미널 화면이 아니라 데이터 통로다.



이 구조를 이해해야 서로 다른 프로그램을 연결하는 파이프와 리다이렉션을 이해할 수 있다.

다음 학습에서는 Shell의 파이프와 리다이렉션,

다음과 같은 터미널에서 사용하는 기호들의 의미를 살펴본다.

```
python train.py > train.log
python train.py 2> error.log
python train.py > train.log 2>&1
python generate.py | python analyze.py
```

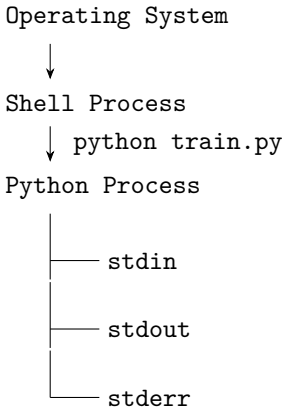
여기서 `>`, `2>`, `2>&1`, `|`는

Python 문법이 아니라 Shell이 제공하는 기능이다.

이 부분까지 이해하면 Shell, Python, 운영체제, 프로세스, 파일 입출력이 어떻게 연결되는지 거의 완성된다.

1.8.16 Shell과 표준 입출력의 관계

먼저 지금가지 배운 프로세스 구조를 다시 살펴보자



여기서 중요한 질문이 하나 있다.

Python 프로세스가 `print()`를 실행하면 왜 터미널 화면에 문자열이 출력

될까?

우리는 지금까지 다음과 같이 이해했다.

```
print("Hello")
```

print()는 기본적으로 sys.stdout에 문자열을 출력한다.

그런데 stdout 자체가 터미널 화면을 의미하는 것은 아니라고 하였다.

그렇다면 다음 두 개념을 구분해야 한다.

1. 표준 출력(stdout)
   프로세스가 데이터를 내보내는 논리적 통로

2. 터미널(Terminal)
   해당 데이터를 화면에 표시할 수 있는 입출력 환경

일반적인 터미널 실행에서는 Shell이 자식 프로세스를 생성할 때 표준 입출력이 터미널에 연결되도록 한다.

```
Python Process──────────────────→Terminal

                  stdout
```

따라서 Python이 stdout에 데이터를 쓰면 터미널에 표시된다.

하지만 Shell은 이 연결을 변경할 수도 있다.

예를 들어 stdout을 터미널이 아니라 파일에 연결할 수 있다.

이것을 리다이렉션(Redirection)이라고 한다.

1.8.17 파일 디스크립터(File Descriptor)

리다이렉션을 이해하기 전에 새로운 개념 하나를 알아야 한다.

바로 파일 디스크립터(File Descriptor, FD)이다.

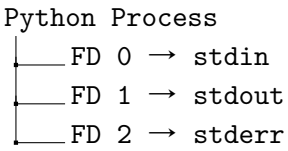
운영체제는 프로세스가 열어 놓은 파일이나 입출력 자원에 접근할 수 있도록 식별자를 제공한다.

POSIX 계열 운영체제(macOS/Linux)에서는 이러한 식별자로 정수를 사용한다.

대표적인 표준 파일 디스크립터는 다음과 같다.

파일 디스크립터	이름	의미
0	stdin	표준 입력
1	stdout	표준 출력
2	stderr	표준 오류 출력

이를 구조로 표현하면 다음과 같다.



여기서 한 가지 주의해야 한다.

0, 1, 2는 Python에서 정의한 숫자가 아니다.

POSIX 계열 운영체제에서 사용하는 표준 파일 디스크립터 번호다.

Python은 이러한 운영체제 기능을 활용하는 것이지,

Windows에서도 표준 입출력 스트림이라는 개념은 존재하지만,

내부적인 핸들 관리 방식은 POSIX와 차이가 있다.

Python에서 확인하기

macOS의 일반적인 터미널 환경에서 다음 코드를 실행해 보자.

```python
import sys

print(sys.stdin.fileno())
print(sys.stdout.fileno())
print(sys.stderr.fileno())
```

일반적으로 다음 결과가 나온다.

```
0
1
2
```

`fileno()`는 해당 스트림과 연결된 운영체제 수준의 파일 디스크립터 번호를 반환하는 메서드다.

다만 Jupyter Notebook이나 일부 IDE에서는 표준 스트림을 별도의 객체로 대체하기 때문에 `fileno()`가 지원되지 않을 수 있다.

여기서 핵심은 다음이다.

> `Shell`은 파일 디스크립터의 연결 대상을 변경하여
>
> 프로세스의 입출력 방향을 제어할 수 있다.

1.8.18 출력 리다이렉션 >

이제 가장 기본적인 리다이렉션부터 살펴보자.

다음 Python 파일이 있다고 가정하자.

```
# train.py

print("Training started")
print("Epoch 1 completed")
print("Epoch 2 completed")
```

터미널에서 다음과 같이 실행하면:

```
python train.py
```

결과가 화면에 출력된다:

```
Training started
Epoch 1 completed
Epoch 2 completed
```

그런데 다음처럼 실행하면 어떻게 될까?

```
python train.py > train.log
```

이번에는 정상 출력이 터미널에 표시되지 않고 `train.log` 파일에 저장된다.

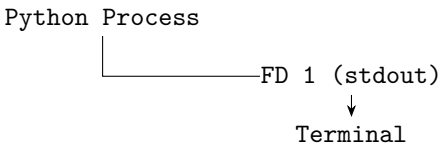
파일 내용은 다음과 같다:

```
Training started
Epoch 1 completed
Epoch 2 completed
```

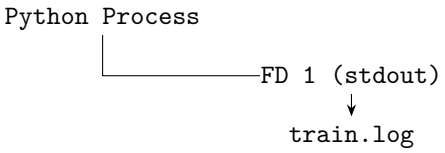
이것이 출력 리다이렉션이다.

내부 구조

일반적인 실행:



리다이렉션을 사용한 실행:



여기서 중요한 사실이 있다.

Python 코드 자체는 변경되지 않았다.

```python
print("Training started")
```

이 코드는 여전히 표준 출력에 데이터를 보내고 있다.

다만 Shell이 프로세스를 실행할 때 `stdout`의 연결 대상을 파일로 변경했기 때문에 파일에 저장되는 것이다.

즉, 출력 목적지를 결정하는 주체가 Python 코드가 아니라 Shell인 경우이다.

>는 기존 파일을 어떻게 처리할까?

다음 명령을 두 번 실행해 보자.

```
python train.py > train.log
```

```
python train.py > train.log
```

일반적으로 두 번째 명령을 실행하면 기존 `train.log`의 내용이 지워지고 새로운 내용으로 대체된다.

왜냐하면 `>`는 기본적으로 파일을 생성하거나 기존 파일을 잘라내고(truncate) 출력을 기록하기 때문이다.

이전에 배운 다음 코드와 비슷한 성질이다.

```
with open("train.log", "w") as f:
    f.write("Training started\n")
```

다만 Shell 리다이렉션과 Python의 `open()`은 서로 다른 계층에서 동작한다는 차이가 있다.

1.8.19 추가 출력 리다이렉션 >>

기존 파일 내용을 유지하면서 새로운 내용을 추가하고 싶다면 어떻게 해야 할까?

이때는 `>>`를 사용한다.

```
python train.py >> train.log
```

두 연산자를 비교하면 다음과 같다.

연산자	동작	Python 파일 모드와 비교
`>`	파일을 새로 쓰거나 기존 내용을 잘라내고 기록	`"w"`
`> >`	기존 파일 끝에 내용을 추가	`"a"`

예를 들어 다음과 같이 실행한다고 생각해 보자.

```
python train.py > train.log
python train.py >> train.log
```

그러면 `train.log`에는 두 번의 실행 결과가 순서대로 기록된다.

```
Training started
Epoch 1 completed
```

```
Epoch 2 completed
Training started
Epoch 1 completed
Epoch 2 completed
```

ML/DL 프로젝트에서 학습 로그를 누적할 때 사용할 수 있다.

하지만 여러 실험을 하나의 로그 파일에 계속 추가하면

어느 실험의 결과인지 구분하기 어려워질 수 있으므로,

실제 프로젝트에서는 실행별 로그 파일을 분리하는 방법도 자주 사용한다.

1.8.20 오류 출력 리다이렉션 2>

이번에는 stderr를 살펴보자

다음 코드를 작성했다고 가정하자:

```python
# train.py

import sys

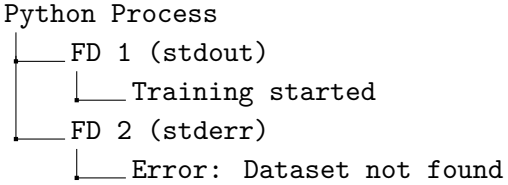
print("Training started")

print(
    "Error: Dataset not found",
    file=sys.stderr
)
```

일반적으로 실행하면 두 메시지가 모두 터미널에 표시된다.

```
Training started
Error: Dataset not found
```

하지만 내부적으로는 서로 다른 통로를 사용하고 있다.



그렇다면 다음 명령을 실행해 보자.

```
python train.py > train.log
```

어떻게 될까?

stdout만 파일로 연결했기 때문에 train.log에는 다음 내용만 저장된다.

```
Training started
```

반면 오류 메시지는 여전히 터미널에 표시된다.

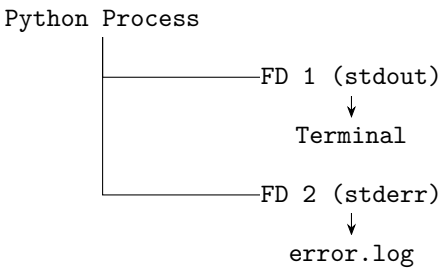
```
Error: Dataset not found
```

이번에는 다음 명령을 실행해 보자.

```
python train.py 2> error.log
```

여기서 2는 stderr의 파일 디스크립터 번호다.

따라서 다음과 같이 동작한다.



stdout은 터미널에 표시되고, stderr는 error.log에 저장된다.

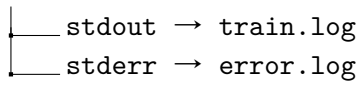
> 두 출력을 서로 다른 파일에 저장하기

다음처럼 작성할 수도 있다.

```
python train.py > train.log 2> error.log
```

그러면 다음과 같이 분리된다.

```
    Python Process
```



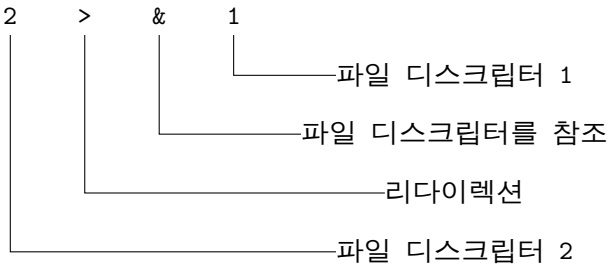
이 방식은 프로그램의 정상 실행 결과와 오류 메시지를 구분해 저장하고 싶을 때 유용하다.

1.8.21 2>&1의 의미

이제 처음보면 이해하기 어려운 표현이 등장한다

```
python train.py > train.log 2>&1
```

이것을 단순히 암기하기 보다는 기호를 나누어서 이해해 보자.



즉, 2>&1은 FD 2를 현재 FD 1이 가리키는 출력 대상으로 연결하라는 의미다.

여기서 중요한 단어는 '현재'다.

실행 순서

다음 명령을 보자.

```
python trian.py > train.log 2>&1
```

Shell은 리다이렉션을 왼쪽에서 오른쪽 순서로 처리한다.

먼저:

```
> train.log
```

를 처리하면 다음과 같다.

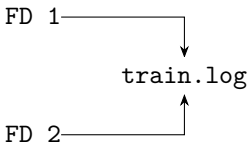
```
FD 1 → train.log
```

```
FD 2 → Terminal
```

그다음:

```
2>&1
```

을 처리하면 FD 2도 현재 FD 1이 연결된 대상으로 향하게 된다.



따라서 정상 출력과 오류 출력이 모두 `train.log`에 저장된다.

순서를 바꾸면 어떻게 될까?

이번에는 다음 명령을 살펴보자.

```
python train.py 2>&1 > train.log
```

비슷해 보이지만 결과가 다르다.

첫 번째 단계:

```
2>&1
```

이 시점에서 FD 1은 터미널에 연결되어 있다.

따라서 FD 2도 터미널에 연결된다.

```
FD 1 → Terminal
FD 2 → Terminal
```

두 번째 단계:

이제 FD 1만 파일에 연결된다.

```
FD 1 \ra train.log
FD 2 \ra Terminal
```

따라서 최종 결과는 다음과 같다.

명령	stdout	stderr
`> train.log 2>&1`	파일	파일
`2>&1 > train.log`	파일	터미널

이것이 Shell 리다이렉션에서 연산자 순서가 중요한 이유다.

특히 `2>&1`은 두 스트림을 영구적으로 하나의 스트림으로 합친다는 뜻이 아니라, FD 2의 출력 대상을 그 시점의 FD 1과 동일하게 설정한다는 의미로 이해하는 것이 정확하다.

1.8.22 입력 리다이렉션 <

지금까지는 프로세스의 출력을 다른 곳으로 보냈다.

이번에는 반대로 프로세스에 데이터를 입력하는 방법을 알아보자.

다음 파일을 작성해 보자.

```
# analyze.py

import sys

data = sys.stdin.read()

print("Received:")
print(data)
```

이 프로그램은 `stdin`으로 전달된 데이터를 모두 읽는다.

일반적으로 터미널에서 실행하면 입력을 기다리게 된다.

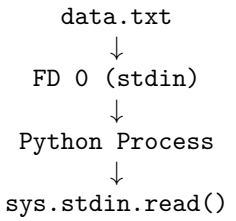
```
python analyze.py
```

이때 키보드로 문자열을 입력할 수 있다.

그런데 다음과 같이 실행하면 어떨까?

```
python analyze.py < data.txt
```

이 명령은 `data.txt`를 표준 입력에 연결한다.



따라서 Python 코드를 수정하지 않아도 파일의 내용을 입력으로 사용할 수 있다.

> Path.read_text()와 무엇이 다를까?

다음 두 코드를 비교해 보자.

방법 A - 파일 경로를 직접 사용

```
from pathlib import Path

data = Path("data.txt").read_text()
```

이 경우 Python 코드가 특정 파일을 직접 연다.

방법 B - 표준 입력 사용

```
import sys

data = sys.stdin.read()
```

이 경우 Python은 입력 데이터가 어디에서 오는지 알 필요가 없다.

Shell이 파일을 연결할 수도 있고, 사용자가 키보드로 입력할 수도 있고, 다른 프로세스가 데이터를 전달할 수도 있다.

이것이 표준 입력을 사용하는 중요한 장점이다.

입력 데이터의 출처와 데이터를 처리하는 프로그램을 분리할 수 있다는 것이다.

1.8.23 파이프(Pipe) |

이제 이번 학습에서 가장 중요한 개념으로 넘어간다.

다음 명령을 살펴보자.

```
python generate.py | python analyze.py
```

여기서 |는 Shell의 파이프 연산자다.

파이프는 일반적으로 앞 프로세스의 표준 출력을 뒤 프로세스의 표준 입력에 연결한다.

구조는 다음과 같다:

```
Python Process A
generate.py
   ↓ stdout
PIPE
   ↓ stdin
Python Process B
analyze.py
```

여기서 중요한 점은 파일을 중간에 저장하지 않아도 된다는 것이다.

프로세스 A가 출력한 데이터를 프로세스 B가 입력받을 수 있다.

예제 1. 데이터 생성

먼저 `generate.py`를 작성해 보자:

```
# generate.py

for i in range(1, 6):
    print(i)
```

실행하면 다음과 같이 출력된다.

```
1
2
3
4
5
```

예제 2. 데이터 분석

이번에는 `analyze.py`를 작성해 보자:

```
# analyze.py
```

```
import sys

numbers = []

for line in sys.stdin:
    numbers.append(int(line.strip()))

print("Sum:", sum(numbers))
```

이 코드도 새로운 내용은 거의 없다.

파일 객체의 반복 처리와 동일한 방식으로 sys.stdin을 사용하고 있다.

```
for line in sys.stdin:
```

sys.stdin은 파일과 유사한 인터페이스를 제공하기 때문에 줄 단위 반복이 가능하다.

그리고 다음 코드로 개행 문자 등을 제거한다.

```
line.strip()
```

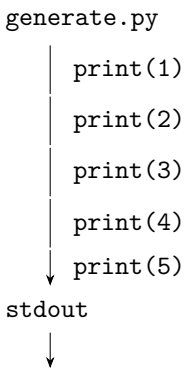
이제 두 프로그램을 연결해 보자:

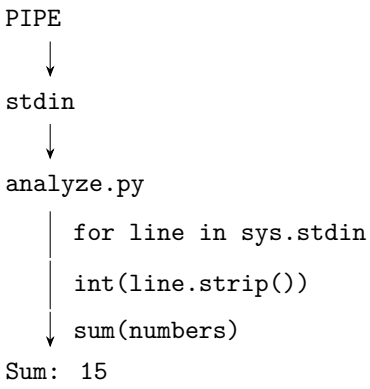
```
python generate.py | python analyze.py
```

출력:

```
Sum: 15
```

전체 흐름을 표현하면 다음과 같다:





이 구조에서 두 프로그램은 서로의 내부 구현을 알 필요가 없다.

`generate.py`는 데이터를 출력하기만 하면 되고,

`analyze.py`는 입력된 데이터를 처리하기만 하면 된다.

즉, 표준 입출력이라는 공통 인터페이스를 통해 서로 다른 프로그램을 조합할 수 있는 것이다.

1.8.24 파이프와 함수 호출의 차이

여기서 Python의 함수 호출과 비교해 보자.

우리가 일반적으로 작성하는 코드는 다음과 같다.

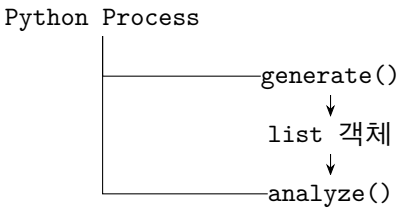
```
def generate():
    return [1, 2, 3, 4, 5]


def analyze(numbers):
    return sum(numbers)

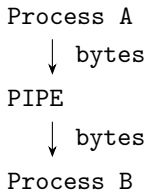

data = generate()
result = analyze(data)

print(result)
```

이 경우에는 하나의 Python 프로세스 안에서 함수들이 데이터를 전달한다.



반면 파이프는 서로 다른 프로세스 사이에서 데이터를 전달한다.



여기에 핵심적인 차이가 있다.

함수 호출은 같은 프로세스 안에서 Python 객체를 전달할 수 있지만,

일반적인 운영체제 파이프는 바이트 데이터를 전달한다.

따라서 파이프를 통해 리스트나 딕셔너리 같은 Python 객체 자체가 그대로 전달되는 것은 아니다.

예를 들어 다음 코드를 보자:

```
print([1, 2, 3])
```

이 코드는 리스트 객체를 `stdout`으로 전달하는 것이 아니다.

리스트의 문자열 표현을 출력하는 것이다.

파이프를 통해 전달되는 것은 그 문자열을 인코딩한 바이트 데이터다.

그래서 실제 데이터 처리 파이프라인에서는 JSON, CSV< JSON Lines 같은 직렬화 형식을 사용하기도 한다.

1.8.25 Shell 파이프와 Python `subprocess.PIPE`

여기까지 이해했다면 이전에 배운 `subprocess.run()`과 연결할 수 있다.

다음 Shell 명령을 생각해 보자:

```
python generate.py | python analyze.py
```

Python에서도 유사한 작업을 구현할 수 있다.

먼저 단순한 방법을 보자:

```
import subprocess
import sys

generated = subprocess.run(
    [sys.executable, "generate.py"],
    capture_output=True,
    text=True,
    check=True
)

analyzed = subprocess.run(
    [sys.executable, "analyze.py"],
    input=generated.stdout,
    capture_output=True,
    text=True,
    check=True
)

print(analyzed.stdout)
```

결과:

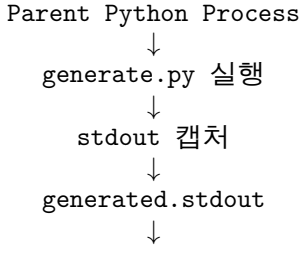
```
Sum: 15
```

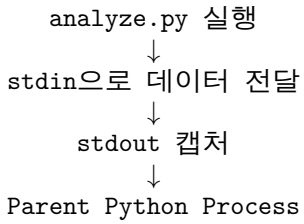
여기서 새로운 인수는 다음과 같다:

```
input = generated.stdout
```

이 인수는 자식 프로세스의 표준 입력으로 데이터를 전달한다.

실행 흐름은 다음과 같다:





하지만 여기에는 Shell 파이프와 차이가 있다.

이 코드는 첫 번째 프로세스가 완전히 종료된 다음, 캡처한 출력을 두 번째 프로세스에 전달한다.

반면 Shell 파이프는 일반적으로 두 프로세스를 동시에 실행하고, 한쪽에서 생성한 데이터를 다른쪽에서 읽을 수 있도록 연결한다.

따라서 위의 `subprocess.run()` 두 번은 파이프와 비슷한 데이터 전달을 구현한 것이지만, 실제로 두 프로세스를 스트리밍 방식으로 연결한 것은 아니다.

실제 운영체제 파이프를 직접 연결하려면 `subprocess.Popen()`과

`subprocess.PIPE` 등을 사용할 수 있다.

1.8.26 |와 >를 함께 사용하기

파이프와 리다이렉션을 결합할 수도 있다.

```
python generate.py | python analyze.py > result.txt
```

이 명령을 해석해 보자.

첫 번째 단계:

```
python generate.py
```

숫자 데이터를 생성한다.

두 번째 단계:

```
| python analyze.py
```

생성된 데이터를 분석 프로그램의 표준 입력으로 전달한다.

세 번째 단계:

```
> result.txt
```

분석 프로그램의 표준 출력을 파일에 저장한다.

전체 구조는 다음과 같다:

```
generate.py
   ↓ stdout
PIPE
   ↓ stdin
analyze.py
   ↓ stdout
result.txt
```

결과적으로 result.txt에는 다음 내용이 저장된다.

```
Sum: 15
```

이런 방식으로 여러 프로그램을 조합해 하나의 데이터 처리 흐름을 구성할 수 있다.

1.8.27 파이프와 종료 코드의 관계

마지막으로 주의할 부분이 하나 있다.

다음 명령을 보자.

```
python generate.py | python analyze.py
```

여기에는 두 개의 프로세스가 존재한다.

따라서 각 프로세스는 독립적인 종료 코드를 가질 수 있다.

```
generate.py
 └── Exit Code = 1

analyze.py
 └── Exit Code = 0
```

그런데 Shell에서 파이프라인 전체의 종료 코드를 확인하면 어떻게 될까?

일반적인 Bash와 zsh에서는 기본적으로 마지막 명령의 종료 코드가 파이프라인의 종료 상태가 된다.

따라서 앞의 `generate.py`가 실패하더라도 마지막 `analyze.py`가 정상 종료하면 파이프라인 전체가 성공한 것처럼 보일 수 있다.

이 문제를 방지하기 위해 Bash와 zsh에서는 다음 옵션을 사용할 수 있다.

```
set -o pipefail
```

이 옵션을 활성화하면 파이프라인에서 실패한 명령이 있을 때 그 실패를 파이프라인 종료 상태에 반영할 수 있다.

정확하게는 Bash와 zsh에서 `pipefail`을 사용하면 파이프라인의 종료 상태가 0이 아닌 종료 코드를 반환한 가장 오른쪽 명령의 종료 코드가 되고, 모든 명령이 성공하면 0이 된다.

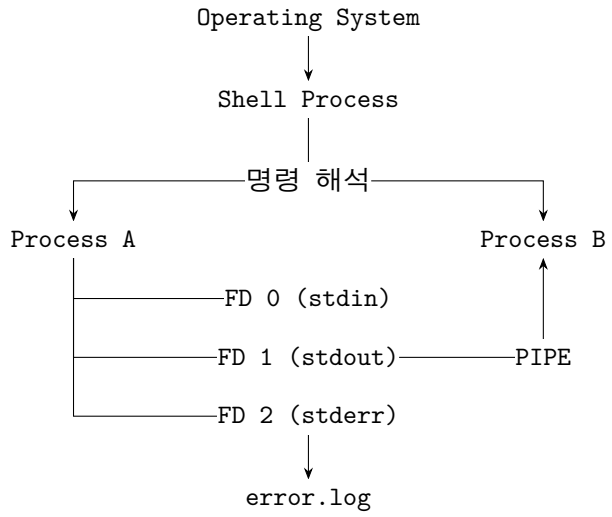
ML/DL 자동화 스크립트에서 파이프라인을 사용할 때 유용한 설정이다.

1.8.28 이번 학습 내용 정리

이번에 배운 핵심 개념들을 정리하면 다음과 같다.

개념	의미
File Descriptor	프로세스의 열린 입출력 자원을 식별하는 번호
0	stdin
1	stdout
2	stderr
>	stdout을 파일에 기록
>>	stdout을 파일 끝에 추가
2>	stderr를 파일에 기록
2>&1	stderr의 출력 대상을 현재 stdout의 출력 대상과 동일하게 설정
<	파일을 stdin에 연결
|	앞 명령의 stdout을 뒤 명령의 stdin에 연결
subprocess.PIPE	Python에서 자식 프로세스의 입출력 파이프를 구성할 때 사용
pipefail	파이프라인의 실패 상태를 더 정확하게 반영하는 Shell 옵션

지금 까지 배운 내용을 하나로 연결하면:



이제 Shell이 단순히 명령을 입력받는 프로그램이 아니라, 프로세스를 실행하고 각 프로세스의 입출력 통로를 연결하는 역할도 수행한다는 것을 이해할 수 있을 것이다.

그리고 Python의 `print()`, `input()`, `sys.stdin`, `sys.stdout`, `sys.stderr`가

왜 존재하는지도 더 명확해졌을 것이다.

다음 학습에서는 subprocess.run()과 subprocess.Popen()의 차이를 살펴볼 것이다.

1.8.29 subprocess.run()과 subprocess.Popen()의 차이

먼저 다음 코드를 보자.

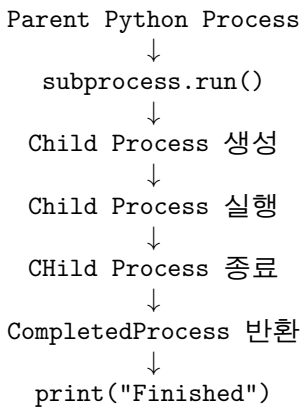
```
import subprocess
import sys

result = subprocess.run(
    [sys.executable, "-c", "print('Hello')"]
)

print("Finished")
```

이 코드는 자식 프로세스가 종료된 다음 Finshed를 출력한다.

실행 흐름은 다음과 같다:



그렇다면 자식 프로세스가 실행되는 동안 부모 프로세스에서 다른 작업을 하고 싶다면 어떻게 해야 할까?

이때 사용하는 것이 subprocess.Popen()이다.

```
import subprocess
import sys
```

```
process = subprocess.Popen(
    [sys.executable, "-c", "print('Hello')"]
)

print("Parent is running")
```

Popen()은 자식 프로세스를 생성한 뒤,

그 프로세스가 종료될 때까지 기다리지 않고 Popen 객체를 반환한다.

따라서 부모 프로세스는 자식 프로세스가 실행 중인 상태에서도 다음 코드를 실행할 수 있다.

다만 두 프로세스의 출력 순서는 실행 스케줄링에 따라 달라질 수 있다.

두 함수의 차이

구분	**subprocess.run()**	**subprocess.Popen()**
주요 목적	외부 프로그램 실행 후 결과 확인	실행 중인 프로세스 직접 관리
기본 대기 동작	종료까지 기다림	생성 직후 반환
반환 객체	`CompletedProcess`	`Popen`
종료 코드	`result.returncode`	`process.returncode`
실행 중 상태 확인	직접 지원하지 않음	`poll()`
종료 대기	내부적으로 수행	`wait()`
입출력 통신	`input, capture_output` 등	`communicate()`
프로세스 종료 요청	직접 제어에 제한적	`terminate(), kill()`

여기서 중요한 사실이 하나 있다.

subprocess.run()은 내부적으로 Popen()을 이용해 구현된 고수준 함수다.

즉, 두 함수가 완전히 별개의 프로세스 실행 메커니즘을 사용하는 것은 아니다.

run()은 자주 사용하는 실행·대기·결과 수집 절차를 편리하게 묶어 놓은 인터페이스라고 이해하면 된다.

1.8.30 Popen 객체와 프로세스의 관계

이전 학습에서 Python의 이름과 객체 사이의 바인딩을 배웠다.

그 관점에서 다음 코드를 살펴보자.

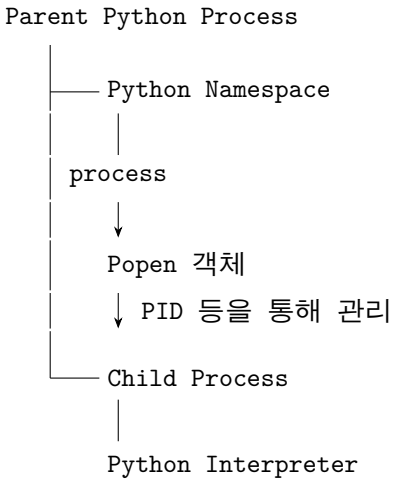
```
process = subprocess.Popen(
    [sys.executable, "-c", "print('Hello')"]
)
```

여기서 두 가지가 발생한다.

첫 번째는 운영체제 수준의 자식 프로세스 생성이다.

두 번째는 그 자식 프로세스를 관리하기 위한 Python의 `Popen` 객체 생성이다.

이를 구조로 표현하면 다음과 같다:



여기서 반드시 구분해야 할 점이 있다.

`Popen` 객체 자체가 자식 프로세스인 것은 아니다.

`Popen` 객체는 부모 Python 프로세스의 메모리에 존재하는 Python 객체다.

반면 자식 프로세스는 운영체제가 별도로 관리하는 실행 단위이다.

따라서 다음 두 개념은 서로 다르다.

```
process
```

이는 `Popen` 객체를 참조하는 Python 이름이다.

```
process.pid
```

이는 자식 프로세스의 PID를 나타내는 속성이다.

예를 들어 다음 코드를 실행해 보자.

```
import os
import subprocess
import sys

print("Parent PID:", os.getpid())

process = subprocess.Popen(
    [sys.executable, "-c", "import os; print(os.getpid())"]
)

print("Child PID:", process.pid)

process.wait()
```

예상 출력은 다음과 비슷하다.

```
Parent PID: 42001
Child PID: 42002
42002
```

출력 순서는 달라질 수 있지만, 중요한 것은 부모와 자식의 PID가 서로 다르다는 사실이다.

또한 자식 프로세스가 종료되더라도 부모에 있는 Popen 객체가 자동으로 사라지는 것은 아니다.

Python 객체의 생명주기와 운영체제 프로세스의 생명주기는 별개의 개념이기 때문이다.

1.8.31 poll() - 프로세스가 종료되었는지 확인하기

Popen()은 자식 프로세스의 종료를 기다리지 않고 반환한다고 했다.

그렇다면 자식 프로세스가 아직 실행 중인지 어떻게 확인할까?

이때 사용하는 메서드가 poll()이다.

```
process.poll()
```

반환값은 다음과 같다.

반환값	의미
None	아직 종료되지 않음
정수	프로세스가 종료되었으며 해당 값은 종료 코드

예제를 보자.

```
import subprocess
import sys
import time

process = subprocess.Popen(
    [
        sys.executable,
        "-c",
        "import time; time.sleep(3)"
    ]
)

print("Immediately:", process.poll())

time.sleep(4)

print("After 4 seconds:", process.poll())
```

예상 출력:

```
Immediately: None
After 4 seconds: 0
```

여기서 `time.sleep(3)`은 자식 프로세스에서 3초 동안 실행을 지연시키는 코드다.

부모 프로세스에서는 4초를 기다렸으므로 일반적으로 자식 프로세스가 종료된 상태가 된다.

하지만 시스템 스케줄링 등에 따라 정확한 실행 시점은 달라질 수 있다.

`poll()`은 대기 함수일까?

아니다.

이 부분이 중요하다.

`poll()`은 현재 종료 상태를 확인하고 즉시 반환하는 메서드다.

자식 프로세스가 종료될 때까지 기다리지 않는다.

따라서 다음과 같은 코드를 작성할 수 있다.

```
while process.poll() is None:
    print("Still running...")
    time.sleep(1)
```

이 코드는 자식 프로세스가 실행 중인 동안 주기적으로 상태를 확인한다.

이러한 방식을 `polling`이라고 부른다.

다만 너무 짧은 간격으로 반복 확인하면 불필요하게 CPU 자원을 사용할 수 있다.

1.8.32 `wait()` - 프로세스 종료까지 기다리기

이번에는 `wait()`을 살펴보자.

```
process.wait()
```

`wait()`은 자식 프로세스가 종료될 때까지 기다린 뒤 종료 코드를 반환한다.

예제를 보자.

```
import subprocess
import sys

process = subprocess.Popen(
    [
        sys.executable,
        "-c",
        "import time; time.sleep(3)"
    ]
)

print("Before wait")

returncode = process.wait()

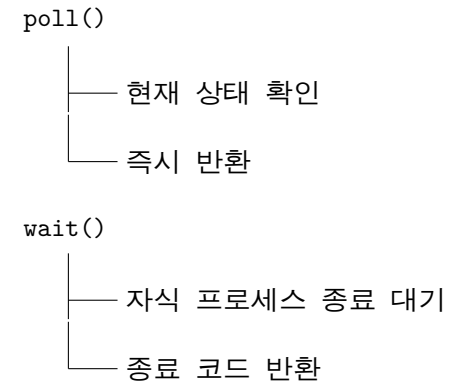
print("After wait")
print("Return code:", returncode)
```

실행 결과:

```
Before wait
After wait
Return code: 0
```

여기서 After wait은 자식 프로세스가 종료된 뒤에 출력된다.

poll()과 wait()의 차이



이렇게 구분하면 된다.

그리고 wait()가 반환된 이후에는 다음 속성에서도 종료 코드를 확인할 수 있다.

```
process.returncode
```

단 Popen 객체의 returncode는 프로세스가 종료되었다는 사실을

poll(), wait() 등의 방법으로 확인하기 전에는 None일 수 있다.

1.8.33 communicate() - 자식 프로세스와 데이터 주고받기

이제 중요한 메서드인 communicate()를 살펴보자.

우리는 앞에서 subprocess.run()을 사용할 때 다음과 같이 작성하였다.

```
result = subprocess.run(
    command,
    capture_output=True,
    text=True
)
```

Popen()에서는 다음과 같이 작성할 수 있다.

```
import subprocess
import sys

process = subprocess.Popen(
    [
        sys.executable,
        "-c",
        "print('Hello from child')"
    ],
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    text=True
)

stdout, stderr = process.communicate()

print("stdout:", stdout)
print("stderr:", stderr)
print("returncode:", process.returncode)
```

예상 출력:

```
stdout: Hello from child

stderr:
returncode: 0
```

여기에 새로운 코드가 등장했다.

```
stdout=subprocess.PIPE
stderr=subprocess.PIPE
```

이는 자식 프로세스의 표준 출력과 표준 오류 출력을 파이프로 연결하겠다는 의미다.

그리고 다음 코드로 데이터를 수집한다.

```
stdout, stderr = process.communicate()
```

communicate()는 자식 프로세스와 표준 입출력을 통해 통신하고, 프로세스 종료까지 기다린다.

반환값은 두 개의 요소로 구성된 튜플이다.

(stdout, stderr)

따라서 튜플 언패킹을 통해 각각의 결과를 변수에 바인딩할 수 있다.

communicate() 와 wait() 의 차이

wait()는 기본적으로 프로세스 종료를 기다리는 역할이다.

반면 communicate()는 파이프를 통해 데이터를 주고받으면서 종료까지 기다리는 역할을 수행한다.

메서드	주요 역할
poll()	종료 상태 확인
wait()	종료 대기
communicate()	표준 입출력 통신 및 종료 대기

여기서 한 가지 중요한 주의사항이 있다.

stdout=PIPE 또는 stderr=PIPE로 설정한 상태에서 자식 프로세스가 많은 데이터를 출력한다면,

단순히 wait()만 호출하는 것은 위험할 수 있다.

왜 그럴까?

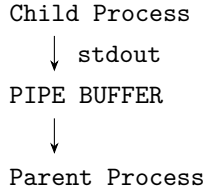
1.8.34 파이프 버퍼와 교착 상태(Deadlock)

이 부분은 운영체제의 프로세스 모델을 이해하는 데 중요한 내용이다.

앞에서 파이프는 두 프로세스 사이에서 바이트 데이터를 전달하는 통로라고 하였다.

그런데 파이프에는 일반적으로 유한한 크기의 버퍼(buffer)가 존재한다.

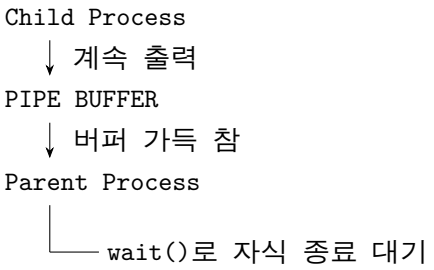
다음 구조를 생각해 보자.



자식 프로세스가 데이터를 출력하면 파이프 버퍼에 데이터가 쌓인다.

부모 프로세스가 그 데이터를 읽으면 버퍼에 공간이 생긴다.

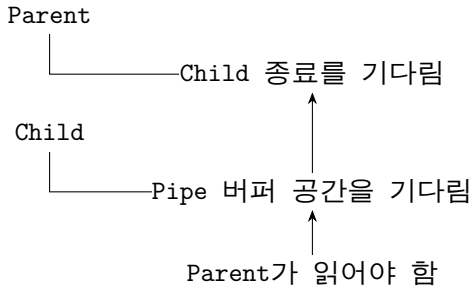
그런데 부모가 데이터를 읽지 않는다면 어떻게 될까?



버퍼가 가득 차면 자식 프로세스는 추가 데이터를 쓰기 위해 기다려야 할 수 있다.

반면 부모 프로세스는 자식 프로세스가 종료되기를 기다리고 있다.

결과적으로 다음과 같은 상태가 발생할 수 있다.



이것이 교착 상태의 한 예시이다.

따라서 파이프를 이용해 자식 프로세스의 출력을 수집할 때는

일반적으로 communicate()를 사용하는 것이 안전하다.

communicate()는 파이프에서 데이터를 읽으면서 자식 프로세스의 종료를 기다리도록 설계되어 있다.

다만 communicate()는 수집한 출력을 메모리에 보관하므로,

매우 큰 출력에는 적합하지 않을 수 있다.

이 점은 ML/DL 프로젝트의 로그 처리에서 다시 연결할 것이다.

1.8.35 Popen()으로 두 프로세스를 실제로 연결하기

이제 앞에서 배운 Shell 파이프를 Python으로 구현해 보자.

Shell에서는 다음과 같이 작성했다.

```
python generate.py | python analyze.py
```

이를 Popen()으로 구현하면 다음과 같다.

```
import subprocess
import sys

producer = subprocess.Popen(
    [sys.executable, "generate.py"],
    stdout=subprocess.PIPE
)

consumer = subprocess.Popen(
    [sys.executable, "analyze.py"],
    stdin=producer.stdout,
    stdout=subprocess.PIPE,
    text=True
)

producer.stdout.close()

output, _ = consumer.communicate()

producer_returncode = producer.wait()

print(output)
print("Producer:", producer_returncode)
print("Consumer:", consumer.returncode)
```

여기서 가장 중요한 부분은 다음이다.

```
stdout=subprocess.PIPE
```

첫 번째 프로세스의 표준 출력을 파이프로 연결한다.

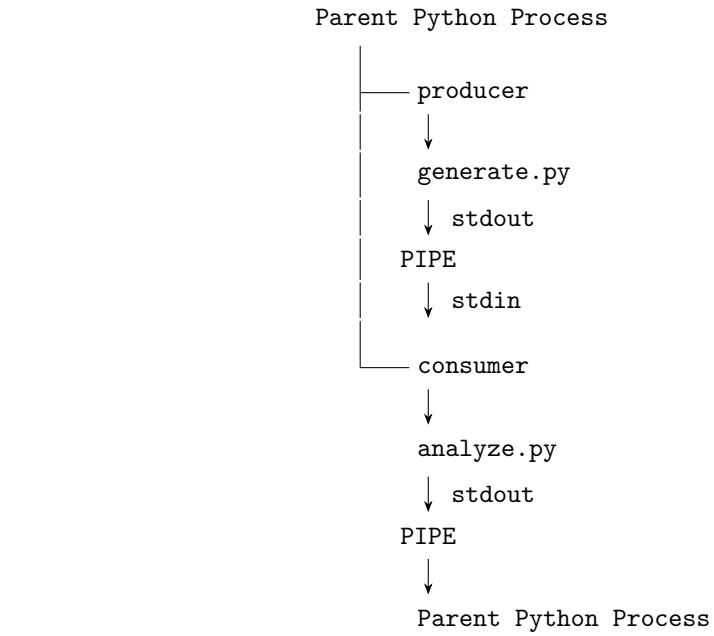
그리고 다음 코드에서

```
stdin=producer.stdout
```

두 번째 프로세스의 표준 입력을 첫 번째 프로세스의 표준 출력 파이프에 연

결한다.

전체 구조는 다음과 같다.



여기서 두 프로세스는 도잇에 실행될 수 있다.

첫 번째 프로세스가 데이터를 생성하면 두 번째 프로세스가 이를 읽어 처리할 수 있다.

> producer.stdout.close()는 왜 필요할까?

다음 코드가 눈에 보인다.

```
producer.stdout.close()
```

부모 프로세스가 가지고 있는 첫 번째 파이프의 읽기 쪽 핸들을 닫는 작업이다.

두 번째 프로세스가 이미 해당 파이프를 입력으로 전달받았으므로,

부모가 그 읽기 핸들을 계속 유지할 필요가 없다.

이렇게 하면 두 번째 프로세스가 조기에 종료되어 더 이상 파이프를 읽지 않을 때,

첫 번째 프로세스가 적절하게 파이프 단절을 감지하는 데 도움이 된다.

POOSIX 환경에서는 이 경우 생산자에게 SIGPIPE가 전달되거나 쓰기 작업이 실패할 수 있다.

중요한 것은 close()가 생산자 프로세스를 직접 종료시키는 명령은 아니라는 점이다.

파이프의 한쪽 끝에 대한 부모 프로세스의 참조를 닫는 것이다.

1.8.36 timeout - 실행 시간 제한하기

외부 프로그램을 실행할 때는 예상보다 오래 걸리는 상황이 발생할 수 있다.

예를 들어 모델 학습이 특정 단계에서 멈추거나, 외부 프로그램이 응답하지 않는 상황을 생각해 보자.

이때 사용할 수 있는 것이 timeout이다.

subprocess.run()에서 사용하기

```
import subprocess
import sys

try:
    result = subprocess.run(
        [
            sys.executable,
            "-c",
            "import time; time.sleep(10)"
        ],
        timeout=3
    )

except subprocess.TimeoutExpired:
    print("Execution timed out")
```

이 코드는 자식 프로세스가 지정된 제한 시간 안에 종료되지 않으면

TImeoutExpired 예외를 발생시킨다.

subprocess.run()은 시간 초과 시 자식 프로세스를 종료시키고

종료 처리를 기다린 뒤 예외를 발생시키도록 구현되어 있다.

Popen()에서 사용하기

Popen()에서는 조금 다르다.

```
import subprocess
import sys

process = subprocess.Popen(
    [
        sys.executable,
        "-c",
        "import time; time.sleep(10)"
    ]
)

try:
    process.wait(timeout=3)

except subprocess.TimeoutExpired:
    print("Timeout")

    process.terminate()
    process.wait()
```

여기서 중요한 차이가 있다.

Popen.wait(timeout=...)에서 시간 초과가 발생해도

자식 프로세스가 자동으로 종료되지는 않는다.

따라서 필요한 경우 직접 종료 요청을 해야 한다.

```
process.terminate()
```

그리고 종료를 기다린다.

```
process.wait()
```

이 차이는 실무에서 반드시 기억해야 한다.

1.8.37 terminate()와 kill()

프로세스가 실행 중일 때 종료를 요청하는 방법도 알아보자.

대표적으로 다음 두 메서드가 있다.

process.terminate()

process.kill()

둘 다 프로세스 종료에 사용하지만 운영체제에 따라 동작 방식이 다르다

macOS/Linux

POSIX 환경에서는 일반적으로 다음과 같이 동작한다.

메서드	동작
terminate()	SIGTERM 신호 전달
kill()	SIGKILL 신호 전달

SIGTERM은 프로세스에 종료를 요청하는 신호다.

프로세스가 이를 처리할 수 있기 때 ㅐ문에 종료 전에 일부 정리 작업을 수행할 수도 있다.

반면 SIGKILL은 프로세스가 일반적인 방식으로 처리하거나 무시할 수 없는 강제 종료 신호다.

따라서 가능하다면 정상적인 종료 절차를 먼저 사용하는 것이 좋다.

Windows

Windows에서는 내부적인 프로세스 종료 방식이 다르다.

특히 Python의 Popen.terminate()와 Popen.kill()은

Windows에서 모두 강제 종료에 가까운 동작을 수행한다.

따라서 macOS/Linux의 SIGTERM과 SIGKILL 구분을

Windows에 그대로 적용해서는 안 된다.

terminate()를 호출하면 즉시 종료될까?

반드시 그렇지는 않다.

process.terminate()

는 종료를 요청하는 메서드다.

따라서 이후에

```
process.wait()
```

를 호출해 실제 종료를 확인하는 것이 일반적이다.

또한 `terminate()`와 `kill()`은 기본적으로

해당 `Popen` 객체가 관리하는 프로세스를 대상으로 하며,

그 프로세스가 추가로 생성한 모든 자식 프로세스까지 자동으로 종료한다고 보장하지는 않는다.

1.8.38 ML/DL 프로젝트에서 프로세스 제어하기

이제 지금까지 배운 내용을 실제 프로젝트에 연결해 보자.

다음과 같은 구조가 있다고 가정하자.

```
my-project/
├── train.py
├── run_experiment.py
├── data/
│   └── trian.csv
└── logs/
```

이번에는 학습 스크립트를 실행하면서 로그를 파일에 저장하고,

실행 상태를 확인하는 구조를 만들어 보자.

1. 학습 스크립트

```
# train.py

import argparse
import time

parser = argparse.ArgumentParser()

parser.add_argument(
    "--epochs",
```

```
    type=int,
    default=5
)

args = parser.parse_args()

for epoch in range(1, args.epochs + 1):
    print(f"Epoch {epoch}/{args.epochs}", flush=True)

    time.sleep(1)

print("Training completed", flush=True)
```

여기서 `flush=True`가 새롭게 등장했다.

```
print(..., flush=True)
```

일반적으로 표준 출력은 버퍼링될 수 있다.

특히 출력 대상이 터미널이 아니라 파일이나 파이프일 때는 출력이 즉시 전달되지 않을 수 있다.

`flush=True`는 `print()` 호출 후 해당 출력 스트림의 버퍼를 비우도록 요청하는 옵션이다.

따라서 학습 로그를 가능한 한 즉시 확인하고 싶을 때 유용하다.

2. 실험 실행 스크립트

```
# run_experiment.py

import subprocess
import sys
import time
from pathlib import Path

project_root = Path(__file__).resolve().parent

train_script = project_root / "train.py"

log_dir = project_root / "logs"
log_dir.mkdir(parents=True, exist_ok=True)

log_path = log_dir / "train.log"

command = [
    sys.executable,
    str(train_script),
```

```
    "--epochs",
    "5"
]

with log_path.open("w", encoding="utf-8") as log_file:

    process = subprocess.Popen(
        command,
        cwd=project_root,
        stdout=log_file,
        stderr=subprocess.STDOUT
    )

    print("Training PID:", process.pid)

    while process.poll() is None:
        print("Training is running...")
        time.sleep(1)

    print("Training finished")
    print("Exit code:", process.returncode)
```

이 코드를 하나씩 분석해 보자.

```
cwd=project_root
```

`cwd=project_root`

이 옵션은 자식 프로세스의 현재 작업 디렉터리(CWD)를 지정한다.

예전에 배운 `Path.cwd()`와 연결되는 부분이다.

부모 프로세스의 CWD를 변경하는 것은 아니다.

```
stdout=log_file
```

`stdout=log_file`

자식 프로세스의 표준 출력을 파일에 연결한다.

Shell에서 다음 명령을 실행하는 것과 비슷하다.

```
python train.py > train.log
```

```
stderr=subprocess.STDOUT
```

```
stderr=subprocess.STDOUT
```

자식 프로세스의 표준 오류 출력을 표준 출력과 동일한 대상으로 연결한다.

Shell의 다음 표현과 유사한 의미이다.

```
python train.py > train.log 2>&1
```

따라서 정상 출력과 오류 출력이 모두 `train.log`에 저장된다.

```
process.poll()
```

```
while process.poll() is None:
```

자식 프로세스가 아직 실행 중인지 확인한다.

실행 중이면 부모 프로세스에서 상태 메시지를 출력한다.

로그 파일이 닫히는 시점

```
with log_path.open("w", encoding="utf-8") as log_file:
```

`with` 블록이 끝나면 부모가 연 로그 파일 객체가 닫힌다.

여기서는 반복문이 자식 프로세스의 종료를 확인 한 뒤 `with` 블록을 빠져나가므로,

로그 파일을 너무 일찍 닫는 문제를 피할 수 있다.

1.8.39 `subprocess.run()`과 `Popen()` 중 무엇을 선택할까?

실무에서는 대부분의 단순한 외부 프로그램 실행에 `subprocess.run()`을 먼저 고려하면 된다.

예를 들어 다음처럼 실행 결과만 필요하다면:

```
result = subprocess.run(
    command,
    capture_output=True,
    text=True,
    check=True
)
```

이 방식이 간결하고 관리하기 쉽다.

반면 다음과 같은 상황에서는 Popen()이 적합할 수 있다.

- 외부 프로그램을 실행한 뒤 부모가 다른 작업을 계속해야 하는 경우
- 실행 중인 프로세스의 상태를 주기적으로 확인해야 하는 경우
- 여러 프로세스를 동시에 실행해야 하는 경우
- 프로세스의 표준 입출력을 직접 연결해야 하는 경우
- 실행 중인 프로세스에 종료 신호를 보내야 하는 경우

다만 Popen()이 더 저수준으 인터페이스이므로,

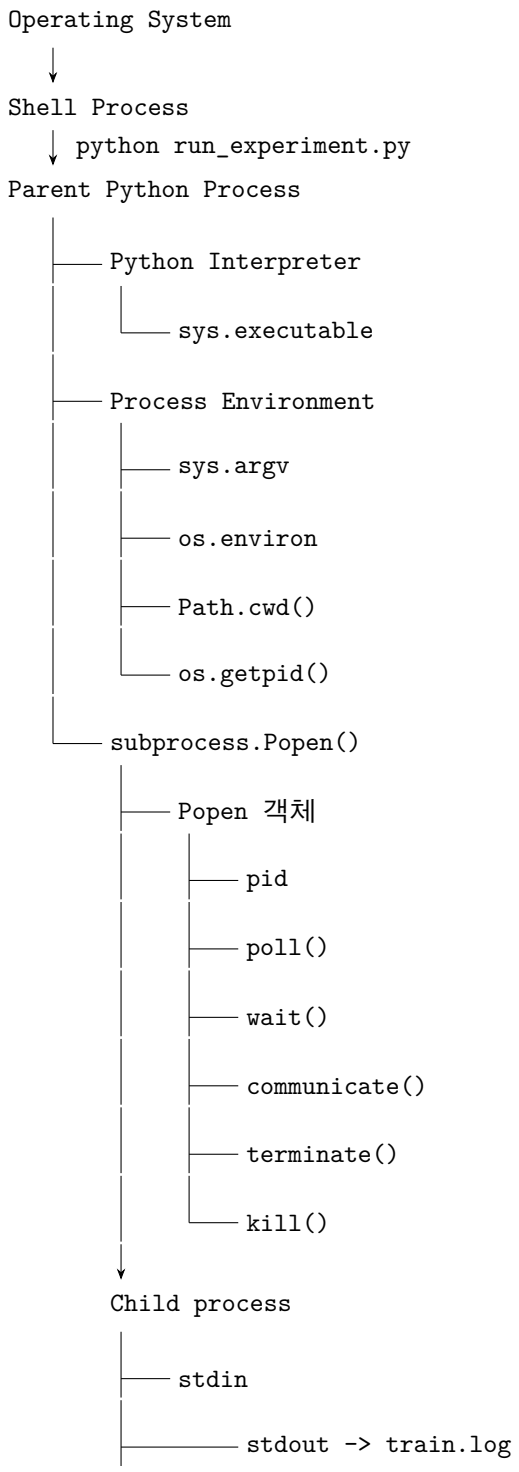
자식 프로세스의 종료 확인과 자원 정리를 개발자가 더 신경 써야 한다.

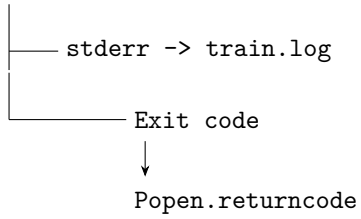
1.8.40 이번 학습 내용 정리

이번에 배운 핵심 개념은 다음과 같다.

개념	의미
Popen()	자식 프로세스를 생성하고 관리 객체를 반환
Popen 객체	자식 프로세스를 관리하는 Python 객체
pid	자식 프로세스의 PID
poll()	종료 상태를 확인하고 즉시 반환
wait()	자식 프로세스 종료까지 대기
communicate()	표준 입출력 통신과 종료 대기
subprocess.PIPE	자식 프로세스의 표준 입출력을 파이프로 연결
subprocess.STDOUT	stderr를 stdout과 동일한 대상으로 연결
timeout	실행 시간 제한
terminate()	자식 프로세스에 종료 요청
kill()	자식 프로세스 강제 종료
cwd	자식 프로세스의 작업 디렉터리 지정

전체 학습 구조는 다음과 같다





여기서 가장 중요한 것은 Python 객체와 운영체제 프로세스를 구분하는 것이다.

Popen 객체는 부모 Python 프로세스에 존재하는 객체이고,

자식 프로세스는 운영체제가 별도로 관리하는 실행 단뒤이다.

그리고 poll(), wait(), communicate()는

그 자식 프로세스의 상태와 입출력을 관리하기 위한 인터페이스다.

이것으로 8장 프로세스와 외부 프로그램의 핵심 이론 학습은 완료했다.

물론 프로세스 간 통신(IPC), 시그널 처리, 프로세스 그룹, 멀티프로세싱 등

더 깊이 공부할 수 있는 내용은 남아 있다.

하지만 현재 목표인 Python 프로젝트 실행 환경을

이해하는 데 필요한 기본 범위는 충분히 다루었다.

이제 다음은 마지막 단계인 9장 — ML/DL 프로젝트에서 실제로 합치기다.

여기서는 지금까지 배운 개념을 개별적으로 설명하는 것이 아니라,

하나의 프로젝트를 구성하면서 연결할 것이다.

예를 들어 다음 명령을 실행한다고 생각해 보자.

```
conda activate transformer

python -m src.train \
    --data ./data/train.csv \
    --epochs 10 \
    --batch-size 32 \
    --output ./checkpoints/model.pt
```

이 명령을 중심으로 다음을 통합할 것이다.

- Conda 환경과 Python 인터프리터 선택
- 프로젝트 루트와 CWD
- python -m과 패키지 import
- argparse와 Path
- 환경변수와 .env
- 데이터 파일 읽기
- 모델 체크포인트 저장
- 로그 디렉터리 관리
- Hugging Face 모델 다운로드와 캐시
- GPU/CPU 장치 선택
- 학습 프로세스 실행과 종료 코드 관리

1.9 ML/DL 프로젝트 실행 구조 통합

1.9.1 지금까지 배운 내용이 왜 필요했을까?

먼저 우리가 최종적으로 실행하려는 명령을 살펴보자.

```
conda activate transformer

python -m src.train \
    --data ./data/train.csv \
    --epochs 10 \
    --batch-size 32 \
    --output ./checkpoints/model.pt
```

이 명령에는 지금까지 학습한 거의 모든 개념이 포함되어 있다.

명령 요소	관련 학습 내용
conda activate transformer	2장: Python 실행 환경
python	2장: Python 인터프리터
-m src.train	4장: 모듈과 패키지
--data	3장: 커맨드라인 인수
./data/train.csv	1장: 상대 경로와 CWD
--epochs 10	3장: argparse와 타입 변환
--batch-size 32	3장: 커맨드라인 옵션
--output	3장: 실행 인수
./checkpoints/model.pt	5장: 파일 입출력

그리고 명령에는 직접 드러나지 않지만, 내부적으로 다음 개념들도 사용하게 된다.

```
import os
import sys
from pathlib import Path

sys.executable
sys.argv
sys.path

os.environ
os.getcwd()

Path.cwd()
Path(__file__)
```

여기에 이전에 배운 subprocess까지 연결하면,

여러 학습 실험을 자동으로 실행하는 구조도 만들 수 있다.

즉, 9장의 목표는 개별 Python 기능을 하나의 프로젝트 실행 시스템으로 구성하는 것이다.

1.9.2 프로젝트의 기본 디렉터리 구조

우선 다음과 같은 프로젝트를 가정하자.

```
transformer-project/
├── src/
│   ├── __init__.py
│   ├── train.py
│   ├── config.py
│   ├── dataset.py
│   └── model.py
├── data/
│   └── train.csv
├── checkpoints/
├── logs/
├── configs/
│   └── train.json
├── scripts/
│   └── run_experiments.py
├── .env
├── .gitignore
├── environment.yml
└── README.md
```

각 디렉터리와 파일의 역할을 살펴보자.

구성 요소	역할
src/	Python 소스 코드
src/train.py	모델 학습 진입점
src/config.py	학습 설정 관리
src/dataset.py	데이터 로딩과 전처리
src/model.py	모델 정의
data/	학습 데이터
checkpoints/	모델 가중치 및 체크포인트
logs/	학습 로그
configs/	학습 설정 파일
scripts/	실험 실행 및 자동화
.env	로컬 환경변수 설정
.gitignore	Git 추적 제외 규칙
environment.yml	Conda 환경 정의
README.md	프로젝트 실행 방법

여기서 중요한 것은 파일의 종류가 아니라 책임을 기준으로 디렉터리를 분리했다는 것이다.

예를 들어 모델의 구조를 변경하려면 `model.py`,

데이터 전처리를 변경하려면 `dataset.py`,

학습 실행 옵션을 변경하려면 `train.py` 또는 설정 파일을 수정하는 식이다.

물론 실제 프로젝트의 규모에 따라 이 구조는 단순해질 수도 있고 복잡해질 수도 있다.

이번 학습에서는 이 구조를 기준으로 진행한다.

1.9.3 프로젝트 루트(Project Root)와 CWD

이번 장에서 가장 먼저 확실히 구분해야 할 개념이 있다.

바로 **프로젝트 루트와 현재 작업 디렉터리**이다.

우리는 `Path.cwd()`와 `Path(__file__)`의 차이를 배웠다.

그런데 실제 프로젝트에서는 이 차이가 매우 중요해진다.

프로젝트 루트란?

프로젝트 루트는 프로젝트의 최상위 디렉터리를 의미한다.

앞의 예제에서는 다음 디렉터리가 프로젝트 루트다.

```
transformer-project/
```

여기에 `src`, `data`, `checkpoints` 등이 포함되어 있다.

하지만 주의해야 할 점이 있다.

프로젝트 루트는 Python이나 운영체제가 자동으로 지정하는 특별한 디렉터리가 아니다.

개발자가 프로젝트의 구조를 설계하면서 정하는 논리적인 기준 디렉터리다.

반면 CWD는 운영체제가 각 프로세스에 대해 관리하는 실행 환경의 일부이다.

따라서 다음 두 개념은 구분해야 한다.

개념	의미
Project Root	개발자가 정의한 프로젝트 기준 디렉터리
CWD	현재 프로세스의 작업 디렉터리

두 경로가 같을 수도 있지만 반드시 같아야 하는 것은 아니다.

> 같은 코드를 다른 위치에서 실행하면?

프로젝트 구조가 다음과 같다고 가정하자.

```
/Users/user/Desktop/transformer-project/
├── src/
│   ├── __init__.py
│   └── train.py
├── data/
└── train.csv
```

train.py에 다음 코드를 작성해 보자.

```
from pathlib import Path

print("CWD:", Path.cwd())
print("File:", Path(__file__).resolve())
```

먼저 프로젝트 루트에서 실행해 보자.

```
cd /Users/user/Desktop/transformer-project

python -m src.train
```

결과:

```
CWD: /Users/user/Desktop/transformer-project
File: /Users/user/Desktop/transformer-project/src/train.py
```

이번에는 상위 디렉터리에서 실행해 보자.

```
cd /Users/user/Desktop

python transformer-project/src/train.py
```

결과:

```
CWD: /Users/user/Desktop
File: /Users/user/Desktop/transformer-project/src/train.py
```

여기서 무엇이 달라졌을까?

`Path.cwd()`는 변경되었지만, `Path(__file__).resolve()`는 동일한 파일을 가리키고 있다.

즉,

```
Path.cwd()
```

는 현재 프로세스가 어디에서 실행되고 있는지를 나타내고,

```
Path(__file__).reslove()
```

는 현재 소스 파일의 경로를 나타내는 것이다.

단, `__file__`은 일반적인 Python 소스 모듈에서는 사용할 수 있지만

Jupyter Notebook이나 대화형 인터프리터에서는 정의되지 않을 수 있다.

1.9.4 상대 경로는 무엇을 기준으로 해석될까?

다음 코드를 보자.

```
from pathlib import Path

data_path = Path("./data/train.csv")

print(data_path)
```

이 코드에서 `data_path`는 상대 경로를 표현하는 `Path` 객체다.

그런데 실제 파일에 접근할 때는 이 경로를 어디를 기준으로 해석할까?

일반적인 파일 접근에서는 **현재 작업 디렉터리(CWD)**를 기준으로 해석한다.

예를 들어 프로젝트 루트에서 실행하면

```
CWD = /Users/user/Desktop/transformer-project
```

다음 경로는

```
Path("./data/train.csv")
```

실질적으로 다음 파일을 가리킨다.

```
/Users/user/Desktop/transformer-project/data/train.csv
```

하지만 상위 디렉터리에서 실행하면

```
CWD = /Users/user/Desktop
```

동일한 상대 경로가 다음 파일을 가리키게 된다.

```
/Users/user/Desktop/data/train.csv
```

따라서 파일을 찾지 못하는 문제가 발생할 수 있다.

이것이 프로젝트 실행 위치에 따라 FileNotFoundError가 발생하는 대표적인 이유 중 하나다.

해결 방법 : 프로젝트 루트 기준 경로

프로젝트 루트를 코드에서 명시적으로 계산할 수도 있다.

```
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parent.parent

DATA_PATH = PROJECT_ROOT / "data" / "train.csv"

print(DATA_PATH)
```

여기서 경로를 단계별로 살펴보자.

```
Path(__file__).resolve()
```

결과:

```
/Users/user/Desktop/transformer-project/src/train.py
```

다음으로

```
Path(__file__).resolve().parent
```

결과:

```
/Users/user/Desktop/transformer-project/src
```

그리고

```
Path(__file__).resolve().parent.parent
```

결과:

```
/Users/user/Desktop/transformer-project
```

따라서 PROJECT_ROOT를 얻을 수 있다.

이제 다음 코드로 데이터를 참조하면

```
DATA_PATH = PROJECT_ROOT / "data" / "train.csv"
```

현재 작업 디렉터리와 무관하게 동일한 프로젝트 파일을 가리킬 수 있다.

다만 이 방식은 train.py가 프로젝트 루트 바로 아래의 src/에 위치한다는 구조적 가정에 의존한다.

프로젝트 구조가 바뀌면 parent.parent도 적절히 변경해야 한다.

1.9.5 CLI로 전달한 상대 경로는 어떻게 처리할까?

여기서 이전에 배운 argparse와 연결해 보자.

다음 코드를 작성했다고 가정하자.

```
import argparse
from pathlib import Path

parser = argparse.ArgumentParser()

parser.add_argument(
    "--data",
    type=Path,
    required=True
```

```
)

args = parser.parse_args()

print(args.data)
```

터미널에서 다음과 같이 실행하면

```
python -m src.train --data ./data/train.csv
```

`args.data`에는 어떤 객체가 저장될까?

```
Path("data/train.csv")
```

와 같은 상대 경로를 표현하는 `Path` 객체가 저장된다.

여기서 중요한 점이 있다.

`argparse`의 `type=Path`는 문자열을 `Path` 객체로 반환할 뿐,

자동으로 프로젝트 루트 기준의 절대 경로를 만들어 주지는 않는다.

따라서 다음 코드는

```
args.data.is_file()
```

현재 작업 디렉터리를 기준으로 파일 존재 여부를 확인하게 된다.

CLI 경로를 처리하는 두 가지 설계

실제 프로젝트에서는 크게 두 가지 방식이 있다.

방식 A - 사용자의 CWD를 기준으로 해석

```
data_path = args.data.resolve()
```

이 방식은 사용자가 터미널에서 전달한 상대 경로를 현재 작업 디렉터리 기준으로 해석한다.

예를 들어 사용자가

```
python -m src.train --data ./data/train.csv
```

를 실행했다면, 사용자가 명령을 실행한 위치를 기준으로 파일을 찾는 것이다.

일반적인 CLI 프로그램에서는 자연스로운 방식이다.

방식 B - 프로젝트 루트를 기준으로 해석

```
data_path = args.data

if not data_path.is_absolute():
    data_path = PROJECT_ROOT / data_path
```

이 방식은 상대 경로를 프로젝트 루트 기준으로 해석한다.

프로젝트 내부 데이터와 설정 파일을 일관된 위치에서 관리하려는 경우 유용하다.

하지만 사용자가 다른 디렉터리에 있는 파일을 상대 경로로 전달했을 때 예상과 다르게 동작할 수도 있다.

따라서 어느 방식이 절대적으로 옳다고 할 수는 없다.

중요한 것은 CLI가 받은 상대 경로를 어떤 기준으로 해석할지 명시적으로 설계하는 것이다.

이번 프로젝트에서는 학습 명령을 프로젝트 루트에서 실행하는 것을 기본 규칙으로 삼고, CLI 경로는 CWD 기준으로 처리한다.

어이렇게 하면 일반적인 Python CLI의 동작 방식과도 일치한다.

1.9.6 `python train.py`와 `python -m src.train`

이번에는 이전에 배웠던 import와 패키지 구조를 연결해 보자.

다음 프로젝트가 있다고 가정하자.

```
transformer-project/
└── src/
    ├── __init__.py
    ├── train.py
    ├── dataset.py
    └── model.py
```

`train.py`에서 같은 패키지에 있는 `dataset.py`를 가져오고 싶다.

```
from .dataset import load_dataset
```

여기서 .은 현재 패키지를 기준으로 하는 상대 import를 의미한다.

그런데 다음처럼 실행하면 문제가 발생할 수 있다.

```
python src/train.py
```

왜 그럴까?

Python은 `train.py`를 패키지 내부의 모듈이 아니라 직접 실행한 스크립트로 취급하기 때문이다.

이 경우 일반적으로

```
__name__="__main__"
```

이고,

```
__package__ is None
```

이므로 상대 import를 해석할 패키지 정보가 없다.

반면 다음과 같이 실행하면

```
python -m src.train
```

Python은 `src.train`을 패키지 구조 안에서 찾아 실행한다.

이때 실행되는 모듈의 정보는 일반적으로 다음과 같다.

```
__name__=="__main__"
```

```
__package__=="src"
```

따라서 다음 상대 import가 가능해진다.

```
from .dataset import load_dataset
```

이를 정리하면 다음과 같다.

실행 방식	`__name__`	`__package__`	상대 import
`python src/train.py`	`"__main__"`	일반적으로 `None`	일반적으로 실패
`python -m src.train`	`"__main__"`	`"src"`	가능

물론 `python -m src.train`이 성공하려면

`src` 패키지를 Python이 모듈 검색 경로에서 찾을 수 있어야 한다.

이번 예제에서는 프로젝트 루트에서 실행하기 때문에 해당 조건을 만족한다.

왜 프로젝트에서는 `-m` 실행을 권장할까?

핵심은 소스 파일의 물리적 경로가 아니라 패키지 구조를 기준으로 실행할 수 있기 때문이다.

프로젝트가 커질수록 모듈 사이의 import 관계가 복잡해지는데, 이때 패키지 구조를 일관되게 유지하는 것이 중요하다.

1.9.7 학습 진입점(Entry Point) 설계

이제 실제 `train.py`를 구성해 보자.

먼저 학습 진입점이라는 개념을 이해해야 한다.

Entry Point는 프로그램 실행이 시작되는 진입 위치를 의미한다.

이번 프로젝트에서는 다음 명령이 학습 진입점을 실행한다.

```
python -m src.train
```

`src/train.py`를 다음처럼 작성해 보자.

```
# src/train.py

import argparse
import os
import sys
from pathlib import Path


def parse_args():
```

```
    parser = argparse.ArgumentParser(
        description="ML training script"
    )

    parser.add_argument(
        "--data",
        type=Path,
        required=True
    )

    parser.add_argument(
        "--epochs",
        type=int,
        default=10
    )

    parser.add_argument(
        "--batch-size",
        type=int,
        default=32
    )

    parser.add_argument(
        "--output",
        type=Path,
        default=Path("checkpoints/model.pt")
    )

    return parser.parse_args()


def main():
    args = parse_args()

    print("Python:", sys.executable)
    print("CWD:", Path.cwd())
    print("PID:", os.getpid())

    print("Data:", args.data)
    print("Epochs:", args.epochs)
    print("Batch size:", args.batch_size)
    print("Output:", args.output)


if __name__ == "__main__":
    main()
```

이 구조를 자세히 살펴보자.

parse_args()를 분리하는 이유

다음 코드를 별도의 함수로 분리했다.

```
def parse_args():
    ...
```

이유는 커맨드라인 인수 처리와 학습 실행 로직의 책임을 구분하기 위해서다.

예를 들어 추후에 학습 설정을 검증하거나 테스트 코드를 작성할 때 구조를 확장하기 쉬워진다.

또한 더 큰 프로젝트에서는 인수 파싱 자체를 별도의 모듈로 분리할 수도 있다.

main()을 사용하는 이유

다음 함수는 프로그램의 주요 실행 흐름을 담당한다.

```
def main():
    ...
```

Python에서 `main()`이라는 이름은 특별한 예약어가 아니다.

개발자가 관례적으로 사용하는 함수 이름이다.

즉, Python이 자동으로 `main()`을 실행해 주는 것은 아니다.

따라서 다음 코드가 필요하다.

```
if __name__ == "__main__":
    main()
```

이 코드는 현재 모듈이 이 프로그램의 진입점으로 실행되었을 때 `main()`을 호출한다.

반면 다른 모듈에서 다음처럼 import 한다면

```
from src.train import main
```

`main()`이 자동으로 실행되지는 않는다.

이것은 프로젝트를 모듈 단위로 재사용하거나 테스트할 때 중요한 설계다.

1.9.8 입력 파일과 출력 파일의 경로 검증

이제 실제 학습 프로그램에서 필요한 경로 처리를 추가해 보자.

먼저 입력 파일을 확인해야 한다.

```
if not args.data.is_file():
    raise FileNotFoundError(
        f"Dataset not found: {args.data}"
    )
```

입력 파일이 존재하지 않으면 학습을 시작하기 전에 오류를 발생시키는 것이다.

이번에는 출력 경로를 살펴보자.

```
args.output
```

이 객체는 다음과 같은 경로를 가리킬 수 있다.

```
checkpoints/model.pt
```

따라서 다음처럼 작성하여야 한다.

```
args.output.parent.mkdir(
    parents=True,
    exist_ok=True
)
```

이전에 배웠던 내용이다.

```
args.output.mkdir()
```

라고 작성하면 `model.pt`라는 이름의 디렉터리를 만들려고 하기 때문에 의도와 다른 동작이 발생한다.

입력 경로와 출력 경로 처리의 차이

경로	처리
입력 데이터 파일	존재 여부 검증
출력 모델 파일	부모 디렉터리 생성
로그 디렉터리	디렉터리 자체 생성

예를 들어 다음처럼 구분할 수 있다.

```
data_path = args.data.resolve()

output_path = args.output.resolve()

log_dir = Path("logs").resolve()

if not data_path.is_file():
    raise FileNotFoundError(data_path)

output_path.parent.mkdir(
    parents=True,
    exist_ok=True
)

log_dir.mkdir(
    parents=True,
    exist_ok=True
)
```

여기서 `resolve()`는 경로를 절대 경로로 해석하는 데 사용된다.

단, `resolve()` 자체가 파일을 생성하거나 파일 존재를 검증하는 것은 아니다.

파일 존재 여부는 별도로 확인해야 한다.

1.9.9 실행 설정과 프로그램 로직의 분리

이제 ML/DL 프로젝트에서 중요한 설계 원칙을 하나 살펴보자.

바로 설정(Configuration)과 실행 로직(Logic)의 분리다.

예를 들어 다음과 같이 학습 코드를 작성했다고 생각해 보자.

```
epochs = 10
batch_size = 32
learning_rate = 0.0001
```

이 값들을 소스 코드 안에 직접 작성할 수도 있다.

하지만 실험할 때마다 코드를 수정해야 한다는 문제가 발생한다.

그래서 일반적으로 다음 세 가지 방법을 사용한다.

1. 커맨드라인 인수
2. 환경변수
3. 설정 파일

각각의 역할을 비교해 보자.

방식	적합한 용도	예시
CLI	실행마다 변경하는 옵션	epochs, batch size
환경변수	실행 환경에 따른 설정	API token, cache path
설정 파일	여러 설정을 묶어서 관리	optimizer, scheduler, model architecture

물론 엄격하게 구분되는 것은 아니다.

예를 들어 learning rate를 CLI로 전달할 수도 있고, JSON 설정 파일에 저장할 수도 있다.

중요한 것은 설정값의 책임과 변경 주기를 고려해 적절한 위치에 배치하는 것이다.

JSON 설정 파일

다음 파일을 만들어 보자.

`configs/train.json`

내용은 다음과 같다.

```
{
    "model_name": "distilbert-base-uncased",
    "learning_rate": 0.0001,
    "weight_decay": 0.01,
    "max_length": 128
}
```

Python에서는 다음과 같이 읽을 수 있다.

```
import json
from pathlib import Path
```

```
config_path = Path("configs/train.json")

with config_path.open(
    "r",
    encoding="utf-8"
) as f:
    config = json.load(f)

print(config)
```

여기서 `json.load(f)`는 JSON 파일의 내용을 Python 객체로 변환한다.

결과는 다음과 같은 딕셔너리다.

```
{
    "model_name": "distilbert-base-uncased",
    "learning_rate": 0.0001,
    "weight_decay": 0.01,
    "max_length": 128
}
```

이전에 배운 매핑 객체의 개념이 여기서 다시 등장한다.

```
config['learning_rate']
```

는 `'learning_rate'` 키에 대응하는 값을 조회하는 연산이다.

CLI와 설정 파일을 결합하기

이번에는 설정 파일의 경로 자체를 CLI로 전달해 보자.

```
parser.add_argument(
    "--config",
    type=Path,
    default=Path("configs/train.json")
)
```

이제 다음처럼 실행할 수 있다.

```
python -m src.train \
    --config configs/train.json \
    --epochs 10
```

이 구조에서는 다음 두 종류의 설정을 구분할 수 있다.

```
CLI
├── --config
│   └── 설정 파일 위치
└── --epochs
    └── 이번 실행의 epoch 수
```

설정 파일:

```
train.json
├── model_name
├── learning_rate
├── weight_decay
└── max_length
```

즉, CLI는 실행을 제어하고 설정 파일은 모델과 학습에 필요한 여러 설정을 묶어서 제공하는 구조다.

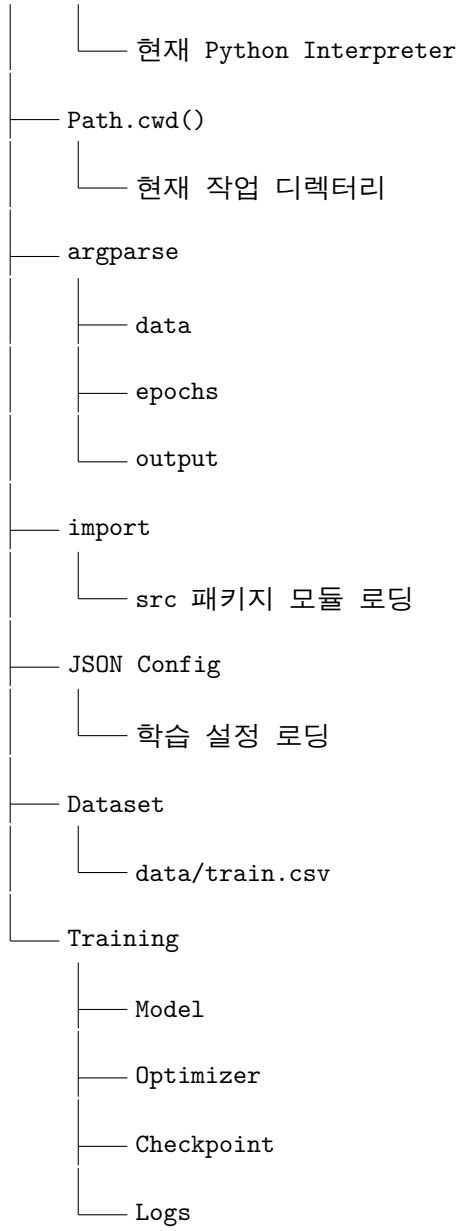
실제 프로젝트에서는 설정 파일과 CLI에 동일한 항목이 존재할 때 어느 쪽이 우선하는지도 명시적으로 설계해야 한다.

예를 들어 CLI 옵션이 설정 파일 값을 덮어쓰도록 만들 수 있다.

1.9.10 지금까지의 실행 흐름 통합

여기까지 배운 내용을 하나의 구조로 정리해 보자.

```
Terminal
   ↓ conda activate transformer
Conda Environment
   ↓ Python Interpreter 선택
Shell
   │ python -m src.train
   │ --data ./data/train.csv
   │ --epochs 10
   ↓ --output ./checkpoints/model.pt
Python Process
   │
   ├── sys.executable
```



이 구조에서 각 기능은 독립적인 책임을 가지고 있다.

그리고 중요한 것은 각 기능이 서로 다른 계층에 존재한다는 점이다.

예를 들어 다음 내용을 구분할 수 있어야 한다.

- Shell은 명령을 해석하고 프로세스를 실행한다.

- Conda는 실행 환경을 구성하고 인터프리터 선택에 영향을 준다.

- Python 인터프리터는 모듈을 로딩하고 코드를 실행한다.

- argparse는 커맨드라인 인수를 Python 객체로 변환한다.

- pathlib는 파일 시스템 경로를 표현하고 조작한다.

- json은 설정 파일의 데이터를 Python 객체로 변환한다.

- 모델 학습 코드는 이 설정들을 이용해 실제 연산을 수행한다.

이 구분이 명확해지면 프로젝트에서 문제가 발생했을 때 어느 계층을 확인해야 하는지도 판단하기 쉬워진다.

이번 학습 내용 정리

학습 항목	핵심
프로젝트 루트	프로젝트 구조의 논리적 기준
CWD	현재 프로세스의 작업 디렉터리
Path(__file__)	소스 파일의 경로
상대 경로	일반적으로 CWD 기준으로 해석
python -m	패키지 구조를 유지하며 모듈 실행
argparse	실행 옵션을 Python 객체로 변환
main()	프로그램의 실행 흐름을 구성하는 관례적 함수
입력 경로 검증	is_file()
출력 경로 준비	output.parent.mkdir()
JSON 설정	json.load()
설정 분리	CLI / 환경변수 / 설정 파일

1.9.11 전체 학습 파이프 라인

지난 학습에서는 프로젝트 루트, CWD, python -m, argparse,

설정파일, 입력·출력 경로를 하나의 프로젝트 구조로 연결했다.

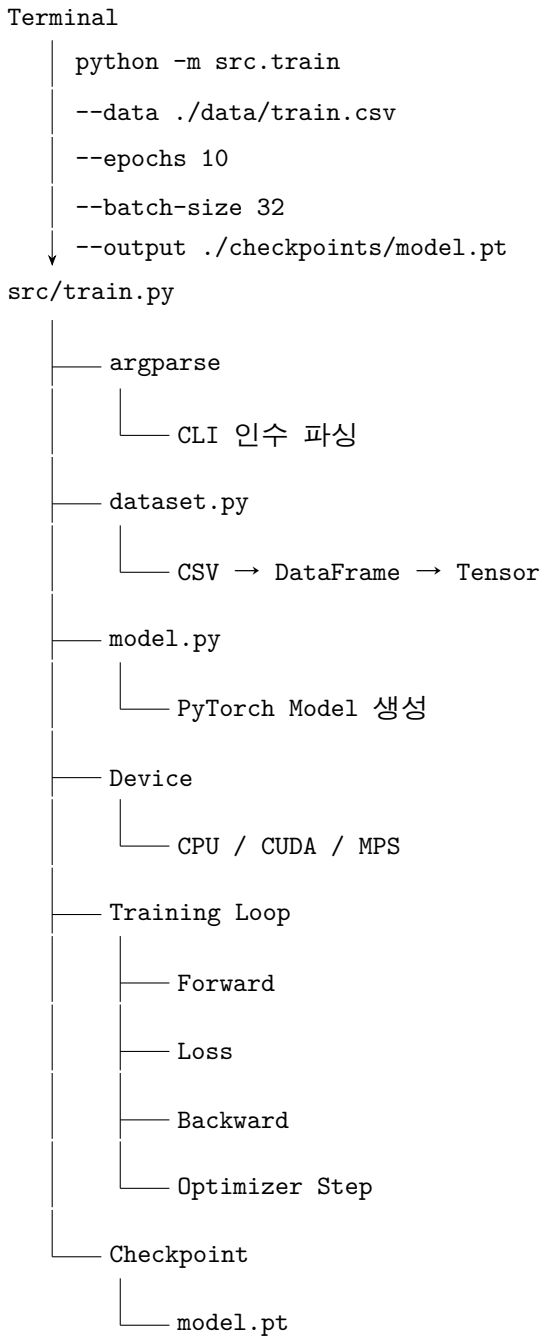
이번에는 그 구조 안에서 데이터가 어떻게 모델에 전달되고,

학습 결과가 어떻게 파일로 저장되는지 살펴볼 것이다.

여기서 중요한 것은 모델의 성능이 아니다.

이번에는 Python 프로그램의 실행 구조와 객체·데이터의 이동에 집중한다.

먼저 우리가 구현할 전체 흐름을 살펴본다.



여기서 눈여겨볼 점은 각 단계가 서로 다른 역할을 수행한다는 것이다.

- `argparse`는 실행 설정을 전달받는다.
- `dataset.py`는 외부 데이터를 모델이 사용할 수 있는 형태로 변환한다.
- `model.py`는 학습할 모델을 정의한다.
- `train.py`는 이 구성 요소들을 조립하고 실행한다.
- `torch.save()`는 학습 결과를 파일로 저장한다.

이번에는 이해하기 쉽도록 CSV 기반의 간단한 회귀 모델을 사용한다.

Transformer 모델을 사용하면 토크나이저, 모델 다운론, Hugging Face 캐시 등

추가적인 개념이 동시에 등장하기 때문이다.

그 부분은 기본 실행 구조를 완성한 뒤 연결할 것이다.

1.9.12 학습 프로젝트 구성

지난 학습에서 만든 프로젝트 구조를 그대로 사용한다.

```
transformer-project/
├── src/
│   ├── __init__.py
│   ├── train.py
│   ├── config.py
│   ├── dataset.py
│   └── model.py
├── data/
│   └── train.csv
├── checkpoints/
├── logs/
├── configs/
│   └── train.json
├── scripts/
│   └── run_experiments.py
├── .env
├── .gitignore
├── environment.yml
└── README.md
```

이번 예제에서 실제로 사용할 파일은 네 개다.

```
data/train.csv
src/dataset.py
src/model.py
src/train.py
```

우선 데이터부터 살펴본다.

CSV 데이터

다음과 같은 데이터가 있다고 가정하자.

```
data/train.csv
```

```
x1,x2,target
1.0,2.0,5.0
2.0,3.0,8.0
3.0,4.0,11.0
4.0,5.0,14.0
5.0,6.0,17.0
6.0,7.0,20.0
7.0,8.0,23.0
```

```
8.0,9.0,26.0
```

이 데이터는 두 개의 입력 피처와 하나의 타깃을 가지고 있다.

수학적으로 표현하면 다음과 같다.

$$X \in \mathbb{R}^{N \times 2}$$
$$y \in \mathbb{R}^{N \times 1}$$

여기서 N은 데이터 샘플 수다.

우리는 간단한 선형 회귀 모델을 학습할 것이다.

$$\hat{y} = XW + b$$

이때

$$W \in \mathbb{R}^{2 \times 1}, \qquad b \in \mathbb{R}$$

이다.

이 예제는 머신러닝 이론을 새롭게 학습하기 위한 것이 아니라,

데이터 파일이 Python 객체와 Tensor로 변환되는 과정을 확인하기 위한 것이다.

1.9.13 dataset.py - 파일에서 Tensor까지

이제 데이터 로딩 코드를 작성해 보자.

```python
# src/dataset.py

import pandas as pd
import torch

from pathlib import Path
from torch.utils.data import TensorDataset

def load_dataset(data_path: Path) -> TensorDataset:
```

```
    df = pd.read_csv(data_path)

    features = df[["x1", "x2"]]
    target = df[["target"]]

    x = torch.tensor(
        features.to_numpy(),
        dtype=torch.float32
    )

    y = torch.tensor(
        target.to_numpy(),
        dtype=torch.float32
    )

    dataset = TensorDataset(x, y)

    return dataset
```

이 코드에는 우리가 지금까지 배운 개념들이 많이 포함되어 있다.

하나씩 살펴보자.

data_path: Path의 의미

함수 정의를 보자.

```
def load_dataset(data_path: Path) -> TensorDataset:
```

여기서 `data_path: Path`는 타입 힌트다.

이 함수가 `Path` 객체를 인수로 받도록 설계되었다는 것을 표현한다.

하지만 타입 힌트 자체가 런타임에서 인수의 타입을 강제로 검사하거나 변환하지는 않는다.

따라서 다음과 같이 호출할 수 있다.

```
from pathlib import Path

dataset = load_dataset(
    Path("data/train.csv")
)
```

이때 `Path("data/train.csv")`는 파일 자체가 아니라 파일 시스템 경로를 표현하는 Python 객체다.

실제 파일을 읽는 작업은 다음 코드에서 발생한다.

```
df = pd.read_csv(data_path)
```

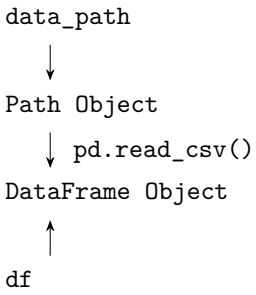
이전에 배운 Path 객체와 파일 입출력의 차이가 여기서 다시 등장한다.

pd.read_csv()는 무엇을 반환할까?

```
df = pd.read_csv(data_path)
```

pd.read_csv()는 CSV 데이터를 읽고 DataFrame 객체를 반환한다.

따라서 다음과 같은 바인딩이 발생한다.



여기서 df는 파일 자체가 아니다.

파일의 내용을 읽어 구성한 메모리상의 DataFrame 객체를 참조하는 이름이다.

이 차이는 이후 체크포인트 저장에도 중요하게 등장한다.

DataFrame에서 Tensor로 변환

다음 코드를 살펴보자.

```
features = df[["x1", "x2"]]
target = df[["target"]]
```

여기서 두 객체의 shape은 다음과 같다.

```
features.shape = (N, 2)
target.shape   = (N, 1)
```

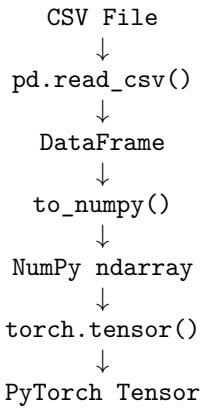
이제 NumPy 배열로 변환한다.

```
features.to_numpy()
```

그리고 PyTorch Tensor로 변환한다.

```
x = torch.tensor(
    features.to_numpy(),
    dtype=torch.float32
)
```

전체 흐름은 다음과 같다.



여기서 `torch.tensor()`는 전달된 데이터를 이용해 새로운 Tensor를 생성한다.

또한 `dtype=torch.float32`를 지정했기 때문에

해당 Tensor의 자료형을 `float32`로 설정한다.

1.9.14 `TensorDataset`과 `DataLoader`

다음 코드를 다시 보자.

```
dataset = TensorDataset(x, y)
```

`TensorDataset`은 여러 Tensor를 같은 인덱스 기준으로 묶어 주는 Dataset 구현체다.

예를 들어 다음처럼 접근할 수 있다.

```
x_sample, y_sample = dataset[0]
```

이때 x_sample은 첫 번째 샘플의 피처를 나타내고,

y_sample은 첫 번째 샘플의 타깃을 나타낸다.

하지만 학습할 때는 일반적으로 데이터를 한 번에 모두 사용하기보다 미니배치 단위로 처리한다.

이때 DataLoader를 사용한다.

```
from torch.utils.data import DataLoader

loader = DataLoader(
    dataset,
    batch_size=32,
    shuffle=True
)
```

이 코드를 객체 관점에서 살펴보자.

```
TensorDataset Object
   ↓ DataLoader(dataset, ...)
DataLoader Object
   ↓ iter(loader)
Iterator
   ↓ next(iterator)
Mini-batch
```

여기서 이전에 배웠던 iterator와 Generator 개념이 연결된다.

정확하게 말하면 DataLoader는 iterable 객체이고,

iter(loader)를 호출하면 배치를 순회하는 iterator를 얻을 수 있다.

따라서 다음 코드가 가능하다.

```
for x_batch, y_batch in loader:
    ...
```

이전에 배웠듯 for문은 내부적으로 iterator protocol을 사용한다.

개념적으로는 다음과 유사하다.

```
iterator = iter(loader)

while True:
    try:
        x_batch, y_batch = next(iterator)

    except StopIteration:
        break
```

즉, DataLoader가 특별한 반복 문법을 사용하는 것이 아니다.

Python의 기존 iterator protocol을 활용하는 것이다.

이 부분은 이전에 학습한 Python 문법이 실제 ML/DL 코드에서 사용되는 대표적 사례다.

1.9.15 model.py - 모델 객체 정의

이제 모델을 정의해 보자.

```
# src/model.py

import torch
from torch import nn

class RegressionModel(nn.Module):

    def __init__(self):
        super().__init__()

        self.linear = nn.Linear(2, 1)

    def forward(self, x):
        return self.linear(x)
```

여기서 모델은 다음과 같은 선형 변환을 수행한다.

$$\hat{y} = XW + b$$

다만 PyTorch의 nn.Linear(2,1)은 내부적으로 가중치를 (1,2) 형태로 저장하고

연산은 다음과 같이 수행한다.

$$Y = XW^T + b$$

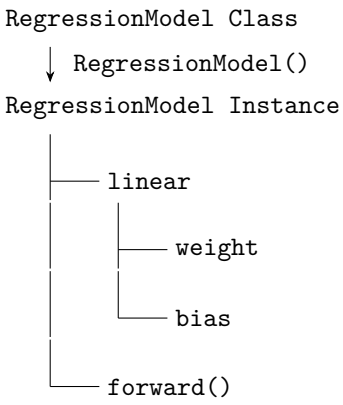
이것은 행렬의 shape을 맞추기 위한 구현상의 표현 차이다.

모델 객체 생성

다음 코드를 보자.

```
model = RegressionModel()
```

이 코드는 RegressionModel 클래스의 인스턴스를 생성한다.



여기서 model은 모델 파일이 아니다.

메모리에 생성된 Python 객체를 참조하는 이름이다.

이 모델 객체가 학습되면서 내부 파라미터의 값이 변경되는 것이다.

그리고 나중에 그 파라미터를 파일로 저장할 것이다.

1.9.16 CPU, CUDA, MPS 장치 선택

이제 모델을 어느 장치에서 실행할지 결정해야 한다.

사용하는 환경이 다양하다면 특히 중요한 부분이다.

- Apple Silicon Mac: MPS 사용 가능 여부 확인
- Windows 데스크톱: CUDA 지원 GPU 여부 확인

- 공통 대안: CPU

먼저 장치 선택 함수를 작성해 보자.

```
import torch


def get_device():

    if torch.cuda.is_available():
        return torch.device("cuda")

    if torch.backends.mps.is_available():
        return torch.device("mps")

    return torch.device("cpu")
```

여기서 `torch.device()`는 연산을 수행할 장치를 표현하는 객체를 생성한다.

```
device = get_device()
```

이후 모델을 해당 장치로 이동할 수 있다.

```
model = model.to(device)
```

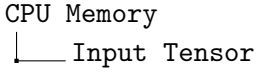
그리고 데이터도 같은 장치로 이동해야 한다.

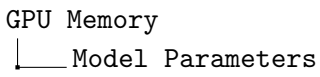
```
x_batch = x_batch.to(device)
y_batch = y_batch.to(device)
```

왜 모델과 데이터를 같은 장치로 이동해야 할까?

PyTorch의 일반적인 연산에서는 연산에 참여하는 Tensor들이 호환되는 장치에 있어야 한다.

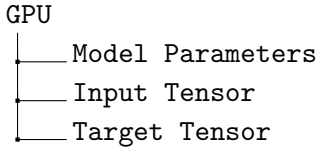
예를 들어 모델의 가중치가 GPU에 있고, 입력 데이터가 CPU에 있으면 장치 불일치 오류가 발생할 수 있다.





이 상태에서는 일반적인 선형 계층 연산을 그대로 수행할 수 없다.

따라서 다음처럼 맞춰 준다.



`model.to(device)`와 `tensor.to(device)`의 차이

여기서 객체 관점에서 주의할 점이 있다.

```
model.to(device)
```

`nn.Module.to()`는 일반적으로 모델 자체의 파라미터와 버퍼를 지정한 장치로 이동시키고, 모델 객체 자신을 반환한다.

반면

```
x_batch.to(device)
```

Tensor의 `.to()`는 필요한 경우 변환된 Tensor를 반환한다.

따라서 Tensor에서는 반환값을 다시 바인딩하는 형태를 사용하는 것이 일반적이다.

```
x_batch = x_batch.to(device)
```

이 차이는 Python 객체의 mutation과 rebinding을 공부했던 내용과도 연결된다.

1.9.17 학습 루프 구성

이제 실제 학습을 수행해 보자.

먼저 필요한 구성 요소를 생성한다.

```
model = RegressionModel().to(device)

criterion = torch.nn.MSELoss()

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001
)
```

여기서 각각의 역할은 다음과 같다.

객체	역할
model	예측값 계산
criterion	손실 계산
optimizer	모델 파라미터 갱신

학습 루프는 다음과 같이 구성할 수 있다.

```
for epoch in range(epochs):

    model.train()

    for x_batch, y_batch in loader:

        x_batch = x_batch.to(device)
        y_batch = y_batch.to(device)

        optimizer.zero_grad()

        predictions = model(x_batch)

        loss = criterion(
            predictions,
            y_batch
        )

        loss.backward()

        optimizer.step()

    print(
        f"Epoch {epoch + 1}/{epochs}, "
        f"Loss: {loss.item():.4f}"
    )
```

이 학습 범위의 초점은 딥러닝의 수학적 원리가 아니므로 설명은 하지 않는다.

대신 Python 실행 구조 관점에서 살펴보자.

`model(x_batch)`는 무엇일까?

```
predictions = model(x_batch)
```

이전에 callable 객체에 대해 배웠다.

`model`은 `RegressionModel`의 인스턴스이고,

`nn.Module`은 호출 가능한 객체로 동작한다.

따라서

```
model(x_batch)
```

처럼 호출할 수 있다.

이 호출은 PyTorch의 `nn.Module` 호출 메커니즘을 거쳐 `forward()`를 실행한다.

여기서 중요한 점은 `model.forward(x_batch)`를 직접 호출하는 것보다

`model(x_batch)`를 사용하는 것이 일반적인 방식이라는 것이다.

PyTorch의 호출 메커니즘은 forward hook 등 추가적인 기능도 처리하기 때문이다.

1.9.18 체크포인트(Checkpoint)란?

이제 학습한 모델을 저장해 보자.

먼저 체크포인트라는 개념부터 구분해야 한다.

체크포인트는 특정 시점의 학습 상태를 저장한 데이터다.

예를 들어 다음 정보를 저장할 수 있다.

- 모델 파라미터
- Optimizer 상태
- 현재 Epoch

- 학습 설정
- 손실값

하지만 모든 체크포인트가 이 정보를 전부 포함하는 것은 아니다.

모델 파라미터만 저장하는 경우도 있고, 학습을 정확하게 이어가기 위한 추가 상태까지 저장하는 경우도 있다.

모델 파라미터만 저장하기

가장 단순한 방법은 다음과 같다.

```
torch.save(
    model.state_dict(),
    "checkpoints/model.pt"
)
```

여기서 중요한 메서드는 다음이다.

```
model.state_dict()
```

이 메서드는 모델의 파라미터와 persistent buffer 등을 담은 딕셔너리 형태의 객체를 반환한다.

따라서 다음 두 개념을 구분해야 한다.

```
Model Object
   ↓ state_dict()
State Dictionary
   ↓ torch.save()
model.pt
```

여기서 `model.pt`는 모델 객체 자체가 아니라, 저장된 데이터를 담고 있는 파일이다.

즉, 메모리의 Python 객체와 디스크의 파일을 구분해야 한다는 5장의 내용이 다시 등장한다.

저장 경로 준비하기

다음 코드를 작성해 보자.

```
from pathlib import Path

output_path = Path("checkpoints/model.pt")

output_path.parent.mkdir(
    parents=True,
    exist_ok=True
)

torch.save(
    model.state_dict(),
    output_path
)
```

여기서 output_path.parent는 checkpoints 디렉터리를 가리킨다.

따라서 모델 파일을 저장하기 전에 부모 디렉터리를 생성한다.

1.9.19 체크포인트 불러오기

이번에는 저장한 모델 파라미터를 불러와 보자.

```
model = RegressionModel()

state_dict = torch.load(
    "checkpoints/model.pt",
    map_location="cpu",
    weights_only=True
)

model.load_state_dict(state_dict)
```

여기서 중요한 부분은 두 단계로 나뉜다는 것이다.

첫 번째:

```
state_dict = torch.load(...)
```

파일에 저장된 데이터를 읽어 Python 객체로 복원한다.

두 번째:

```
model.load_state_dict(state_dict)
```

복원한 파라미터 값을 모델 객체에 적용한다.

따라서 다음 구조가 성립한다.

```
model.pt
   ↓ torch.load()
State Dictionary
   ↓ model.load_state_dict()
Model Object
```

여기서 `torch.load()`만 실행한다고 기존 모델 객체가 자동으로 생성되고 구성되는 것은 아니다.

우리는 먼저 다음 코드를 실행했다.

```
model = RegressionModel()
```

즉, 모델의 구조를 정의하는 Python 코드가 필요하다.

그 구조에 저장된 파라미터를 적용하는 방식이다.

`weights_only=True`는 체크포인트를 불러올 때 허용되는 역직렬화 대상을 제한하는 옵션이다.

그렇더라도 신뢰할 수 없는 체크포인트를 무조건 안전하다고 가정해서는 안 된다.

1.9.20 학습 재개용 체크포인트

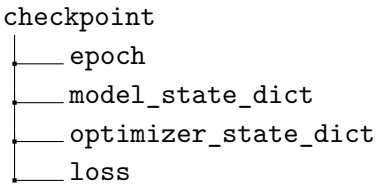
이번에는 모델 파라미터뿐 아니라 Optimizer 상태도 저장해 보자.

```
checkpoint = {
    "epoch": epoch,
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
    "loss": loss.item()
}

torch.save(
    checkpoint,
    "checkpoints/checkpoint.pt"
)
```

여기서 checkpoint는 Python 딕셔너리다.

각 키에 학습 상태를 저장하고 있다.



이제 다시 불러오면 다음과 같다.

```
checkpoint = torch.load(
    "checkpoints/checkpoint.pt",
    map_location="cpu",
    weights_only=True
)

model.load_state_dict(
    checkpoint["model_state_dict"]
)

optimizer.load_state_dict(
    checkpoint["optimizer_state_dict"]
)

start_epoch = checkpoint["epoch"] + 1
```

이렇게 하면 저장된 Epoch 이후부터 학습을 이어갈 수 있는 기본 구조가 만들어진다.

다만 정확하게 동일한 학습 상태를 재현하려면 학습률 스케줄러, 난수 생성기 상태, 데이터 샘플링 상태 등 추가적인 정보가 필요할 수도 있다.

따라서 모델 파라미터만 저장하는 것과 학습을 완전히 재개하기 위한 체크포인트를 저장하는 것은 구분해야 한다.

1.9.21 train.py에서 전체 구성 요소 조립하기

이제 지금까지 작성한 코드를 하나로 연결해 보자.

다음 코드는 학습 실행 구조를 이해하기 위한 최소 예제이다.

```
# src/train.py

import argparse
import sys

from pathlib import Path

import torch

from torch.utils.data import DataLoader

from .dataset import load_dataset
from .model import RegressionModel


def parse_args():

    parser = argparse.ArgumentParser()

    parser.add_argument(
        "--data",
        type=Path,
        required=True
    )

    parser.add_argument(
        "--epochs",
        type=int,
        default=10
    )

    parser.add_argument(
        "--batch-size",
        type=int,
        default=32
    )

    parser.add_argument(
        "--output",
        type=Path,
        default=Path("checkpoints/model.pt")
    )

    return parser.parse_args()


def get_device():

    if torch.cuda.is_available():
        return torch.device("cuda")
```

```
    if torch.backends.mps.is_available():
        return torch.device("mps")

    return torch.device("cpu")


def main():

    args = parse_args()

    data_path = args.data.resolve()
    output_path = args.output.resolve()

    if not data_path.is_file():
        raise FileNotFoundError(data_path)

    output_path.parent.mkdir(
        parents=True,
        exist_ok=True
    )

    device = get_device()

    dataset = load_dataset(data_path)

    loader = DataLoader(
        dataset,
        batch_size=args.batch_size,
        shuffle=True
    )

    model = RegressionModel().to(device)

    criterion = torch.nn.MSELoss()

    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=0.001
    )

    print("Python:", sys.executable)
    print("Device:", device)
    print("Data:", data_path)

    for epoch in range(args.epochs):

        model.train()

        total_loss = 0.0
```

```
        for x_batch, y_batch in loader:

            x_batch = x_batch.to(device)
            y_batch = y_batch.to(device)

            optimizer.zero_grad()

            predictions = model(x_batch)

            loss = criterion(
                predictions,
                y_batch
            )

            loss.backward()

            optimizer.step()

            total_loss += loss.item() * len(x_batch)

        avg_loss = total_loss / len(dataset)

        print(
            f"Epoch {epoch + 1}/{args.epochs} "
            f"Loss: {avg_loss:.4f}"
        )

    torch.save(
        model.state_dict(),
        output_path
    )

    print("Model saved:", output_path)


if __name__ == "__main__":
    main()
```

이 코드는 다음과 같이 실행할 수 있다.

프로젝트 루트에서:

```
python -m src.train \
    --data ./data/train.csv \
    --epochs 10 \
    --batch-size 32 \
    --output ./checkpoints/model.pt
```

실행 결과는 대략 다음과 같다.

```
Python: /path/to/conda/envs/transformer/bin/python
Device: cpu
Data: /path/to/project/data/train.csv

Epoch 1/10 Loss: ...
Epoch 2/10 Loss: ...
...
Epoch 10/10 Loss: ...

Model saved: /path/to/project/checkpoints/model.pt
```

실제 손실값은 모델의 초기 파라미터와 학습 환경에 따라 달라진다.

1.9.22 지금까지 배운 Python 개념이 어디에 사용되었을까?

이제 전체 코드를 Python 기초 개념과 연결해 보자.

Python 개념	실제 사용 위치
Module / Package	`from .dataset import load_dataset`
python -m	`src.train`을 패키지 모듈로 실행
argparse	학습 인수 처리
Path	데이터·체크포인트 경로
Path.resolve()	상대 경로를 절대 경로로 해석
Path.is_file()	입력 데이터 검증
Path.mkdir()	출력 디렉터리 생성
Function	parse_args(), get_device(), main()
Class / Instance	RegressionModel()
Callable	model(x_batch)
Iterator Protocol	for x_batch, y_batch in loader
Mapping	model.state_dict()
File I/O	pd.read_csv(), torch.save()
Object Binding	model = RegressionModel()
Python Interpreter	sys.executable

여기서 주목할 점은 ML/DL 라이브러리가 Python의 기본 객체 모델 위에서 동작한다는 사실이다.

예를 들어 DataLoader는 반복 가능한 객체이고, 모델은 호출 가능한 객체이며,

state_dict()는 매핑 형태의 데이터를 반환한다.

즉, 지금까지 배운 Python 문법과 객체 모델이 실제 딥러닝 프로젝트의 기반이 되는 것이다.

1.9.23 모델 파일과 학습 환경은 별개다

마지막으로 중요한 개념 하나를 짚고 넘어간다.

우리가 다음 코드를 실행했다고 생각해 보자.

```
torch.save(
    model.state_dict(),
    "checkpoints/model.pt"
)
```

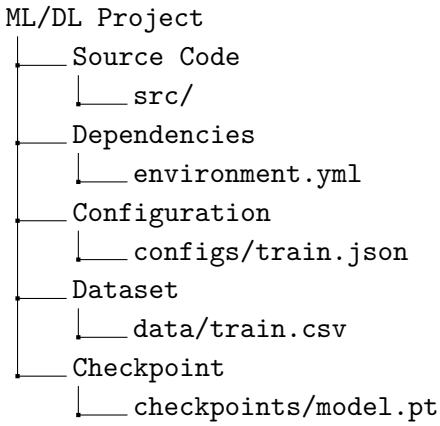
그러면 모델 파라미터가 파일에 저장된다.

하지만 이 파일에 다음 정보가 모두 포함되는 것은 아니다.

- Python 인터프리터의 위치
- Conda 환경 전체
- 설치된 라이브러리 버전
- 학습 데이터
- 모델 클래스의 Python 소스 코드
- 프로젝트 디렉터리 구조

즉, 모델 파일 하나만 가지고 언제나 동일한 실행 환경을 복원할 수 있는 것은 아니다.

이를 구분하면 다음과 같다.



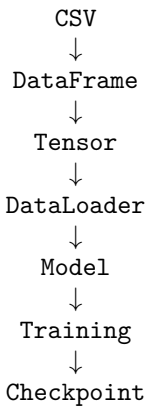
각 구성 요소는 서로 다른 책임을 가지고 있다.

그래서 실제 ML/DL 프로젝트에서는 모델 파라미터뿐 아니라 콘, 환경, 설정, 데이터의 버전까지 함께 관리하는 것이 중요하다.

이 부분은 추후 실험 재현성과 모델 배포를 공부할 때 다시 등장하게 될 것이다.

이번 학습 정리

이번에는 다음 흐름을 실제 코드로 연결했다.



그리고 이 전체 과정은 다음 실행 명령에서 시작했다.

```
python -m src.train \
    --data ./data/train.csv \
    --epochs 10 \
    --batch-size 32 \
    --output ./checkpoints/model.pt
```

이제 우리가 처음에 배웠던 경로, CLI, import, 객체, 반복자, 파일 입출력이

실제 학습 파이프라인 안에서 어떻게 연결되는지 확인하였다.

1.9.24 Hugging Face의 `from_pretrained()` 이해하기

여기서부터는 Hugging Face의 transformers, Conda 환경, 모델 캐시,

환경변수, 실험 자동화를 하나의 구조로 연결한다.

앞에서 작성한 PyTorch 회귀 모델은 로컬 CSV 파일을 읽고 학습하는 구조였다.

하지만 Hugging Face를 사용하면 모델과 토크나이저를 외부 저장소에서 가져오고,

다운로드한 파일을 캐시에 저장하며, 모델에 따라 인증 토큰도 필요해진다.

즉, 이번 학습에서는 다음 질문에 답하는 것이 목표이다.

Python 프로그램이 외부 모델을 가져와 학습하고,

그 실행 환경과 결과를 재현 가능하게 관리하려면 어떤 구조가 필요할까?

먼저 아래의 코드를 살펴보자.

```
from transformers import AutoTokenizer, AutoModel

model_name = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)
```

우리는 이 코드를 실행하면 사전학습된 모델과 토크나이저를 사용할 수 있다는 것을 알 수 있다.

그런데 Python 실행 환경 관점에서는 조금 더 깊게 이해할 필요가 있다.

`from_pretrained()`는 무엇을 하는가?

다음 코드를 보자.

```
model = AutoModel.from_pretrained(
```

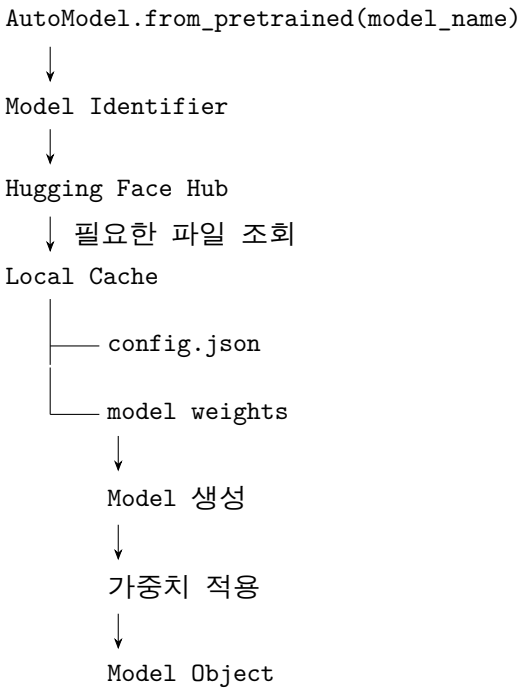
```
    "distilbert-base-uncased"
)
```

이때 from_pretrained()는 단순히 모델 객체를 생성하는 함수가 아니다.

일반적으로 다음 작업을 수행한다.

1. 모델 식별자를 해석한다.
2. 필요한 설정 파일과 가중치 파일을 찾는다.
3. 로컬 캐시에 사용할 수 있는 파일이 있는지 확인한다.
4. 필요한 파일이 없다면 Hugging Face Hub에서 다운로드한다.
5. 설정을 이용해 모델 구조를 생성한다.
6. 저장된 가중치를 모델에 적용한다.
7. 초기화된 모델 객체를 반환한다.

개념적으로 표현하면 다음과 같다.



단, 이미 캐시에 필요한 파일이 있으면 다운로드 하지 않고 사용할 수 있다.

실제 파일 조회 및 다운로드 여부는 캐시 상태와 라이브러리 옵션에 따라 달라진다.

여기서 중요한 점은 다음이다.

`from_pretrained()`가 반환하는 것은

메모리에 생성된 Python 모델 객체이지, 다운로드한 파일 경로가 아니다.

이전에 배운 `pd.read_csv()`와 비교해 보자.

코드	외부 데이터	반환 객체
`pd.read_csv(path)`	CSV 파일	DataFrame
`torch.load(path)`	체크포인트 파일	역직렬화된 객체
`AutoModel .from_pretrained( name)`	모델 설정과 가중치	모델 객체
`AutoTokenizer .from_pretrained( name)`	토크나이저 설정과 관련 파일	토크나이저 객체

외부 데이터를 읽어서 Python 객체를 구성한다는 점에서는 비슷하지만, 각 함수가 수행하는 구체적인 작업은 다르다.

1.9.25 모델 식별자와 파일 경로의 차이

다음 문자열을 보자.

```
model_name = "distilbert-base-uncased"
```

이 문자열은 일반적으로 Hugging Face Hub의 모델 저장소를 식별하는 데 사용된다.

반면 다음 코드는 어떨까?

```
model_path = "./models/distilbert"
```

이것은 로컬 파일 시스템 경로로 사용할 수 있는 문자열이다.

`from_pretrained()`는 모델 식별자뿐 아니라 적절한 구조를 가진 로컬 디렉터리도 입력받을 수 있다.

예를 들어 다음과 같은 디렉터리가 있다고 가정하자.

```
model/
└── distilbert/
    ├── config.json
    ├── model.safetensors
    ├── tokenizer_config.json
    ├── tokenizer.json
    └── vocab.txt
```

실제 파일 구성은 모델과 토크나이저 종류에 따라 달라질 수 있다.

이제 다음처럼 사용할 수 있다.

```
from pathlib import Path
from transformers import AutoTokenizer, AutoModel

model_dir = Path("models/distilbert")

tokenizer = AutoTokenizer.from_pretrained(
    model_dir
)

model = AutoModel.from_pretrained(
    model_dir
)
```

여기서 구분해야 할 개념은 세 가지다.

개념	의미
모델 식별자	Hub에서 모델 저장소를 식별하는 문자열
로컬 모델 디렉터리	모델 파일이 저장된 파일 시스템 경로
모델 객체	Python 프로세스의 메모리에 생성된 모델 인스턴스

이 구분은 이전에 공부했던 이름, 객체, 경로, 파일의 차이와 직접 연결된다.

1.9.26 Hugging Face 캐시(Cache)의 역할

이제 캐시를 살펴보자.

이전에 Hugging Face 모델을 사용하면서 파일 다운로드 오류나 토크나이저 파일 문제를 경험한 적이 있을 것이다.

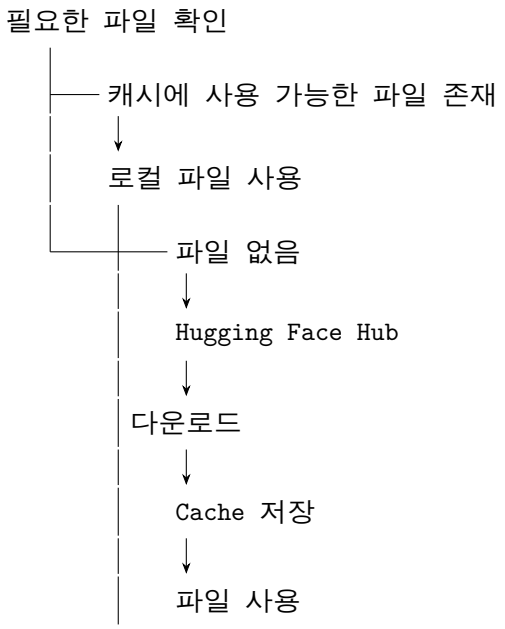
그런 문제를 이해하려면 캐시의 구조를 알아야 한다.

캐시가 필요한 이유

모델을 실행할 때마다 수백 MB 또는 수 GB의 파일을 다운로드한다면 비효율적이다.

그래서 Hugging Face는 다운로드한 파일을 로컬 캐시에 저장해 재사용할 수 있다.

일반적인 흐름은 다음과 같다.



역시 ㅓ캐시는 모델 객체 자체를 저장하는 공간이 아니다.

모델 객체를 구성하는 데 필요한 파일을 저장하는 공간이다.

따라서 Python 프로세스가 종료되더라도 캐시 파일은 일반적으로 디스크에 남아 있다.

반면 메모리에 생성된 모델 객체는 프로세스 종료와 함께 사라진다.

기본 캐시 위치

Hugging Face Hub의 일반적인 기본 캐시 위치는 다음과 같다.

macOS/Linux:

```
~/.cache/huggingface/hub/
```

Windows:

```
C:\Users\<username>\.cache\huggingface\hub\
```

다만 환경변수와 라이브러리 설정에 따라 실제 위치는 달라질 수 있다.

여기서 ~는 사용자 홈 디렉터리를 나타내는 표현이다.

Python에서는 다음과 같이 확인할 수 있다.

```python
from pathlib import Path

print(Path.home())
```

그리고 일반적인 기본 경로를 구성하면 다음과 같다.

```python
cache_dir = (
    Path.home()
    / ".cache"
    / "huggingface"
    / "hub"
)

print(cache_dir)
```

다만 이 코드는 기본 경로를 계산할 뿐, 현재 Hugging Face 라이브러리가 실제로 사용하는 캐시 위치를 조회하는 코드는 아니다.

실제 캐시 위치는 관련 환경변수와 라이브러리 설정을 확인해야 한다.

1.9.27 Hugging Face Hub 캐시의 내부 구조

Hub 캐시는 일반적으로 모델 저장소와 버전을 구분할 수 있는 구조를 사용한다.

예를 들어 다음과 같은 형태다.

```
hub/
└── models--distilbert--distilbert-base-uncased/
    ├── blobs/
    ├── refs/
    └── snapshots/
```

각 디렉터리의 역할을 살펴보자.

디렉터리	역할
blobs/	다운로드한 파일 내용 저장
refs/	브랜치나 태그 등의 참조 정보
snapshots/	특정 리비전의 파일 구성을 나타내는 경로

일반적인 캐시 구조에서는 snapshots의 파일들이 blobs에 저장된 파일을 가리키는 심볼릭 링크로 구성될 수 있다.

환경에 따라 링크를 사용할 수 없는 경우에는 저장 방식이 달라질 수 있다.

여기서 Git과 비슷한 구조가 등장하는 이유는 모델 저장소도 버전이 존재하기 때문이다.

예를 들어 같은 모델 이름이라도 저장소가 업데이트되면 서로 다른 리비전의 파일이 존재할 수 있다.

따라서 모델 이름만으로 모든 실행 환경이 완전히 고정되는 것은 아니다.

1.9.28 HF_HOME과 HF_HUB_CACHE

이제 환경변수를 연결해 보자.

Hugging Face의 캐시 위치는 환경변수를 통해 설정할 수 있다.

대표적인 환경변수는 다음과 같다.

환경변수	역할
HF_HOME	Hugging Face 관련 로컬 데이터의 기본 위치
HF_HUB_CACHE	Hugging Face Hub 저장소 캐시 위치
HF_TOKEN	Hub 인증에 사용할 토큰
HF_HUB_OFFLINE	Hub의 네트워크 접근을 비활성화하는 설정

HF_HOME

예를 들어 macOS 터미널에서 다음과 같이 설정할 수 있다.

```
export HF_HOME="$HOME/huggingface-data"
```

그러면 Hugging Face 라이브러리는 해당 위치를 기본 디렉터리로 사용할 수 있다.

Hub 캐시의 기본 위치도 일반적으로 그 아래에 구성된다.

```
huggingface-data/
└── hub/
```

HF_HUB_CACHE

Hub 캐시 위치만 별도로 지정할 수도 있다.

```
export HF_HUB_CACHE="$HOME/model-cache"
```

이 경우 Hub 모델 저장소 캐시의 위치를 별도로 관리할 수 있다.

두 변수를 동시에 지정했다면 HF_HUB_CACHE가 Hub 캐시 위치를 직접 결정하는 설정이다.

Python에서 설정하기

이전에 배운 os.environ을 사용할 수도 있다.

```
import os

os.environ["HF_HOME"] = "/path/to/huggingface-data"
```

하지만 여기서 중요한 점이 있다.

Hugging Face 관련 라이브러리를 import하기 전에 환경변수를 설정하는 것이 안전하다.

일부 라이브러리는 import 과정에서 환경변수를 읽어 내부 설정을 초기화하기 때문이다.

예를 들어 다음과 같은 순서가 좋다.

```
import os

os.environ["HF_HOME"] = "/path/to/huggingface-data"

from transformers import AutoModel
```

반대로 이미 라이브러리를 import한 뒤 환경변수를 변경하면,

변경한 설정이 현재 프로세스의 라이브러리 동작에 즉시 반영되지 않을 수 있다.

이것이 7장에서 배웠던 환경변수와 Python 객체의 초기화 시점이 연결되는 부분이다.

1.9.29 환경변수와 .env 파일

이번에는 .env 파일을 살펴보자.

먼저 중요한 사실부터 짚고 넘어간다.

.env 파일은 Python이나 운영체제가 자동으로 읽어 환경변수로 등록해 주는 특별한 파일이 아니다.

일반적으로 애플리케이션이나 별도의 도구가 .env 파일을 읽고 환경변수를 설정한다.

예를 들어 프로젝트에 다음 파일이 있다고 가정하자.

```
transformer-project/
└── .env
```

내용

```
HF_HOME=/Users/user/huggingface-data
HF_TOKEN=your_token_here
```

이 파일은 단순한 텍스트 파일이다.

Python이 실행되었다고 해서 자동으로 다음 값이 설정되는 것은 아니다.

```
os.environ["HF_TOKEN"]
```

.env를 사용하려면 별도의 로딩 과정이 필요하다.

python-dotenv 사용하기

대표적인 방법은 python-dotenv 라이브러리를 사용하는 것이다.

```
python -m pip install python-dotenv
```

그다음에 아래처럼 작성할 수 있다.

```
from pathlib import Path
from dotenv import load_dotenv

PROJECT_ROOT = Path(__file__).resolve().parent.parent

load_dotenv(
    PROJECT_ROOT / ".env"
)
```

이제 .env 파일의 설정을 현재 Python 프로세스의 환경변수로 읽어 들일 수 있다.

```
import os

token = os.environ.get("HF_TOKEN")
```

여기서 중요한 구분이 있다.

```
load_dotenv(...)
```

는 .env 파일의 값을 환경변수로 로딩하는 작업이고,

```
os.environ.get("HF_TOKEN")
```

은 현재 프로세스의 환경변수에서 값을 조회하는 작업이다.

또한 load_dotenv()는 기본적으로 이미 존재하는 환경변수를 덮어쓰지 않는다.

필요한 경우 override=True를 지정할 수 있다.

인증 토큰을 다룰 때의 주의사항

Hugging Face 토큰은 인증 정보이므로 소스 코드에 직접 작성하거나 로그에 출력하지 않는 것이 좋다.

특히 다음 코드는 피해야 한다.

```
print(os.environ["HF_TOKEN"])
```

그리고 .env 파일을 Git 저장소에 올리지 않도록 설정해야 한다.

.gitignore에 다음 항목을 추가할 수 있다.

```
.env
```

다만 .gitignore는 아직 추적되지 않은 파일이 Git에 새로 추가되는 것을 방지하는 용도다.

이미 Git에서 추적 중인 파일을 자동으로 제거해 주지는 않는다.

1.9.30 Hugging Face 인증과 공개 모델

여기서 인증이 항상 필요한 것은 아니라는 점도 알아두자.

예를 들어 다음과 같은 공개 모델은 일반적으로 별도의 인증 토큰 없이 다운로드할 수 있다.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained(
    "distilbert-base-uncased"
)
```

반면 접근 제한이 있는 모델이나 비공개 저장소에 접근하려면 인증이 필요할 수 있다.

토큰을 명시적으로 전달하는 방식도 있지만, 라이브러리 버전에 따라 인수 이름이 다를 수 있다.

예를 들어 최근 버전에서는 다음과 같은 형태를 사용할 수 있다.

```
import os

from transformers import AutoModel

model = AutoModel.from_pretrained(
    "some-org/private-model",
    token=os.environ["HF_TOKEN"]
)
```

Hugging Face CLI로 로그인한 경우에는 저장된 인증 정보를 라이브러리가 사용할 수도 있으므로, 반드시 매번 토큰을 코드에 전달해야 하는 것은 아니다.

1.9.31 오프라인에서 모델 사용하기

이번에는 다운로드한 모델을 네트워크 없이 사용하는 방법을 알아보자.

앞에서 캐시가 존재한다고 했다.

그렇다면 이미 다운로드한 모델은 인터넷 연결 없이도 사용할 수 있을까?

필요한 파일이 로컬에 모두 존재한다면 가능하다.

대표적으로 다음 옵션을 사용할 수 있다.

```
from transformers import AutoTokenizer, AutoModel

model_name = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(
    model_name,
    local_files_only=True
)

model = AutoModel.from_pretrained(
    model_name,
    local_files_only=True
)
```

여기서

```
local_files_only=True
```

는 로컬에 존재하는 파일만 사용하도록 요청하는 옵션이다.

필요한 파일이 로컬에 없으면 다운로드하는 대신 오류가 발생한다.

따라서 다음 두 상황을 구분해야 한다.

```
상황 A
├── 필요한 파일이 로컬에 있음
└── 오프라인 로딩 가능

상황 B
├── 필요한 파일이 로컬에 없음
└── 오프라인 로딩 실패
```

환경변수로 Hub의 오프라인 모드를 활성화할 수도 있다.

macOS/Linux:

```
export HF_HUB_OFFLINE=1
```

다만 HF_HUB_OFFLINE은 Hub 라이브러리의 네트워크 접근을 제한하는 설정이므로, 애플리케이션 전체의 모든 네트워크 접근을 차단하는 것은 아니다.

1.9.32 모델 캐시와 모델 저장 디렉터리의 차이

여기서 혼동하기 쉬운 두 개념을 구분한다.

첫 번째는 Hugging Face가 관리하는 캐시다.

```
~/.cache/huggingface/hub/
```

두 번째는 개발자가 직접 저장하는 모델 디렉터리다.

```
transformer-project/
├── models/
    └── my-model/
```

예를 들어 다음 코드를 보자.

```
model.save_pretrained(
    "models/my-model"
```

```
)

tokenizer.save_pretrained(
    "models/my-model"
)
```

이 코드는 모델과 토크나이저를 지정한 디렉터리에 저장한다.

그다음 아래처럼 다시 불러올 수 있다.

```
model = AutoModel.from_pretrained(
    "models/my-model"
)

tokenizer = AutoTokenizer.from_pretrained(
    "models/my-model"
)
```

두 방식의 차이를 정리하면 다음과 같다.

구분	**Hugging Face 캐시**	**save_pretrained() 디렉터리**
주요 목적	다운로드 파일 재사용	모델 결과물 저장 및 재사용
관리 주체	Hugging Face 라이브러리	개발자
위치	캐시 설정에 따라 결정	개발자가 지정
버전 구조	Hub 리비전 기반	개발자가 관리
프로젝트 산출물	일반적으로 아님	일반적으로 해당

특히 모델을 파인튜닝했다면 다운로드 캐시와 파인튜닝 결과물을 구분해서 관리하는 것이 좋다.

1.9.33 environment.yml

과 실행 환경 재현

이제 Conda 환경으로 넘어가자.

우리는 2장에서 Conda 환경을 만들고 활성화하는 방법을 배웠다.

그런데 다른 컴퓨터에서 동일한 프로젝트를 실행하려면 어떻게 해야 할까?

프로젝트 소스 코드만 복사한다고 충분하지는 않다.

Python 버전과 필요한 라이브러리도 설치해야 한다.

이때 사용할 수 있는 것이 `environment.yml`이다.

예를 들어 다음과 같은 파일을 작성할 수 있다.

YAML :

```
name: transformer-project

channels:
  - conda-forge

dependencies:
  - python=3.10
  - pip
  - pandas
  - numpy
  - pip:
      - torch
      - transformers
      - datasets
      - accelerate
      - python-dotenv
```

이 파일은 Conda 환경을 구성하기 위한 선언적인 설정 파일이다.

다음 명령으로 환경을 생성할 수 있다.

```
conda env create -f environment.yml
```

그리고 활성화한다.

```
conda activate transformer-project
```

여기서 중요한 점은 `environment.yml` 자체가 Python 인터프리터나 라이브러리를 포함하는 것은 아니라는 것이다.

환경을 구성하는 데 필요한 정보를 기술한 파일이다.

실제 패키지는 Conda와 pip가 설치한다.

패키지 버전 고정

위 예제에서는 라이브러리 버전을 명시하지 않았다.

그러면 설치 시점에 따라 서로 다른 버전이 설치될 수 있다.

따라서 재현성이 중요한 프로젝트에서는 다음과 같이 버전을 명시하는 것이 좋다.

```
dependencies:
  - python=3.10
  - pip
  - pip:
      - transformers==4.30.2
      - datasets==2.14.7
      - accelerate==0.20.3
```

이 버전들은 이전에 공부했던 Transformer 학습 환경과 관련된 예시이다.

다만 실제 환경을 재구성할 때는 PyTorch, NumPy, Hugging Face Hub 등

다른 의존성의 버전 호환성도 함께 검증해야 한다.

특히 macOS Apple Silicon과 Windows에서는 사용할 수 있는 패키지 빌드가 다를 수 있다.

따라서 동일한 `environment.yml`을 사용하더라도 모든 운영체제에서 완전히 동일한 바이너리 환경이 만들어진다고 보장할 수는 없다.

1.9.34 `sys.executable`로 실제 환경 검증하기

여기서 이전에 배운 `sys.executable`이 다시 등장한다.

Conda 환경을 활성화했다고 생각해 보자.

```
conda activate transformer
```

그다음 Python을 실행하면 일반적으로 활성화된 환경의 인터프리터가 선택된다.

하지만 VS Code, Jupyter, 터미널 설정 등에 따라 실제 사용 중인 인터프리터가 다를 수 있다.

따라서 다음 코드로 확인하는 것이 좋다.

```
import sys
```

```
print(sys.executable)
```

예를 들어 macOS에서는 다음과 같은 경로가 나올 수 있다.

```
/Users/user/anaconda3/envs/transformer/bin/python
```

Windows에서는 다음과 같을 수 있다.

```
C:\Users\user\anaconda3\envs\transformer\python.exe
```

여기서 중요한 것은 환경 이름이 아니라 실제로 실행 중인 인터프리터의 경로다.

또한 설치된 라이브러리의 버전도 확인할 수 있다.

```
import torch
import transformers

print(torch.__version__)
print(transformers.__version__)
```

이렇게 하면 현재 프로세스가 실제로 어떤 환경에서 실행되고 있는지 확인할 수 있다.

1.9.35 Jupyter 커널과 Conda 환경의 관계

Windows에서 Jupyter 커널 문제를 겪어본 경험이 있을 것이다.

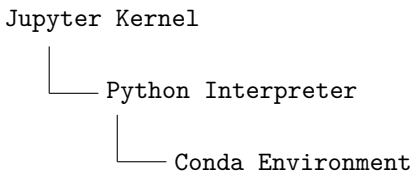
Jupyter Notebook을 실행할 때는 다음 두 개념을 구분해야 한다.

1. Jupyter 서버를 실행하는 환경
2. Notebook 코드가 실행되는 커널 환경

이 둘은 반드시 동일할 필요가 없다.

예를 들어 다음과 같은 구조가 가능하다.

```
Jupyter Server
    ↓ Notebook 실행 요청
```



따라서 터미널에서

```
conda activate transformer
```

를 실행했다고 해서 이미 열려 있는 모든 Notebook이 자동으로 해당 환경을 사용하는 것은 아니다.

Notebook 내부에서 다음 코드를 실행해 확인해야 한다.

```
import sys

print(sys.executable)
```

이 경로가 의도한 Conda 환경으이 Python을 가리키는지 확인하는 것이다.

또한 커널에 필요한 라이브러리가 설치되어 있어야 한다.

예를 들어 `ipykernel`이 필요하다면 해당 환경에 설치할 수 있다.

```
conda activate transformer

python -m pip install ipykernel
```

필요한 경우 커널을 등록할 수도 있다.

```
python -m ipykernel install \
    --user \
    --name transformer \
    --display-name "Python (transformer)"
```

이렇게 등록하면 Jupyter에서 해당 커널을 선택할 수 있다.

여기서 `--name`은 커널의 내부 식별자이고,

`--display-name`은 사용자에게 표시되는 이름이다.

1.9.36 실험 자동화와 subprocess

이제 8장에서 배웠던 subprocess를 연결해 보자.

우리가 다음과 같은 학습 명령을 실행한다고 가정하자.

```
python -m src.train \
    --data ./data/train.csv \
    --epochs 10 \
    --batch-size 32 \
    --output ./checkpoints/model.pt
```

그런데 배치 크기를 변경하면서 여러 실험을 수행하고 싶다면 어떻게 해야 할까?

예를 들어 다음 세 가지 실험을 수행한다고 생각해 보자.

실험	Batch Size	Epoch
Experiment 1	16	10
Experiment 2	32	10
Experiment 3	64	10

이때 매번 터미널에서 명령을 입력할 수도 있지만 Python으로 자동화할 수도 있다.

프로젝트에 다음 파일을 추가하자.

```
scripts/
└── run_experimnets.py
```

코드는 다음과 같다.

```
# scripts/run_experiments.py

import subprocess
import sys

from pathlib import Path


PROJECT_ROOT = Path(__file__).resolve().parent.parent
```

```
BATCH_SIZES = [16, 32, 64]


def main():

    for batch_size in BATCH_SIZES:

        command = [
            sys.executable,
            "-m",
            "src.train",
            "--data",
            "data/train.csv",
            "--epochs",
            "10",
            "--batch-size",
            str(batch_size),
            "--output",
            f"checkpoints/model_bs{batch_size}.pt"
        ]

        result = subprocess.run(
            command,
            cwd=PROJECT_ROOT,
            check=True
        )

        print(
            f"Batch size {batch_size}: "
            f"Exit code {result.returncode}"
        )


if __name__ == "__main__":
    main()
```

이 코드를 실행하면 세 개의 학습 실험이 순서대로 실행된다.

```
python scripts/run_experiments.py
```

sys.executable을 사용하는 이유

다음 코드를 보자.

```
sys.executable
```

이것은 현재 실행 중인 Python 인터프리터의 경로다.

따라서 자식 프로세스에서도 동일한 Python 인터프리터를 사용하도록 할 수 있다.

반면 다음처럼 작성하면

```
"python"
```

Shell이나 운영체제의 실행 파일 검색 경로에 따라 다른 인터프리터가 선택될 가능성이 있다.

그래서 실험 자동화에서는 `sys.executable`을 사용하는 것이 유용하다.

cwd=PROJECT_ROOT의 역할

다음 코드를 보자.

```
cwd=PROJECT_ROOT
```

이것은 자식 프로세스의 CWD를 프로젝트 루트로 지정한다.

따라서 자식 프로세서으 ㅔ서 다음 상대 경로를 사용하면

```
data/train.csv
```

프로젝트 루트 기준으로 해석된다.

또한 `python -m src.train`이 프로젝트 루트에서 실행되므로

`src` 패키지를 찾는 데도 도움이 된다.

check=True의 역할

다음 코드는

```
check=True
```

자식 프로세스가 0이 아닌 종료 코드를 반환했을 때 `CalledProcessError`를 발생시킨다.

따라서 학습 실험이 실패하면 오류를 무시한 채 다음 실험으로 진행하는 것을 방지할 수 있다.

물론 실무에서는 실패한 실험을 기록하고 나머지 실험을 계속 실행하도록 설계할 수도 있다.

그것은 실험 관리 정책에 따라 결정하면 된다.

1.9.37 실험 로그 관리하기

이제 학습 로그를 파일로 저장해 보자.

다음처럼 프로젝트에 로그 디렉터리를 생성할 수 있다.

```
log_dir = PROJECT_ROOT / "logs"

log_dir.mkdir(
    parents=True,
    exist_ok=True
)
```

그리고 각 실험의 로그 파일 경로를 만들자.

```
log_path = log_dir / f"train_bs{batch_size}.log"
```

이제 다음처럼 실행할 수 있다.

```
with log_path.open("w", encoding="utf-8") as log_file:

    result = subprocess.run(
        command,
        cwd=PROJECT_ROOT,
        stdout=log_file,
        stderr=subprocess.STDOUT,
        check=True
    )
```

여기서 `stdout=log_file`은 자식 프로세스의 표준 출력을 파일에 연결한다.

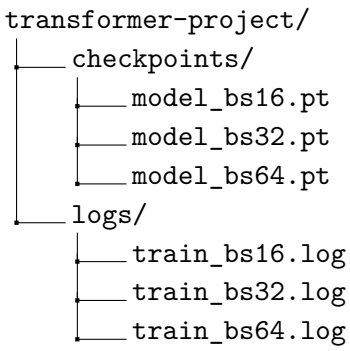
그리고

```
stderr=subprocess.STDOUT
```

는 표준 오류 출력도 같은 대상으로 연결한다.

따라서 정상 출력과 오류 메시지를 하나의 로그 파일에 저장할 수 있다.

결과적으로 다음과 같은 구조가 만들어진다.



여기서 중요한 점은 모델 파일과 로그 파일이 서로 다른 목적을 가진다는 것이다.

모델 파일은 학습 결과의 파라미터를 저장하고, 로그 파일은 실행 과정에서 발생한 메시지를 기록한다.

1.9.38 실험 재현성(Reproducibility)

이제 마지막으로 실험 재현성을 살펴보자.

머신러닝에서 재현성은 매우 중요한 개념이다.

그런데 단순히 난수 시드만 고정한다고 모든 실험이 재현되는 것은 아니다.

예를 들어 다음과 같이 작성할 수 있다.

```
import random
import numpy as np
import torch

seed = 42

random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
```

이 코드는 Python, NumPy, PyTorch의 주요 난수 생성기를 초기화한다.

하지만 실제 학습 결과에는 다음과 같은 요소들도 영향을 줄 수 있다.

- 데이터셋 버전

- 모델 초기 가중치
- 라이브러리 버전
- 하드웨어 종류
- 연산의 결정성
- 데이터 샘플링 순서
- 학습 설정
- 체크포인트의 상태

따라서 실험 재현성을 확보하려면 다음 정보들을 함께 관리하는 것이 좋다.

구분	기록할 정보
코드	Git commit
환경	Python 및 라이브러리 버전
데이터	데이터셋 버전 또는 식별자
모델	모델 이름과 리비전
설정	Learning rate, batch size, epochs
난수	Random seed
장치	CPU, CUDA, MPS
결과	Checkpoint, metrics, logs

특히 Hugging Face 모델을 사용할 때는 모델 이름뿐 아니라 리비전도 기록하는 것이 좋다.

예를 들어 다음처럼 작성할 수 있다.

```
model = AutoModel.from_pretrained(
    model_name,
    revision=revision
)
```

여기서 `revision`에 특정 커밋 해시를 지정하면 해당 리비전의 모델 파일을 요청할 수 있다.

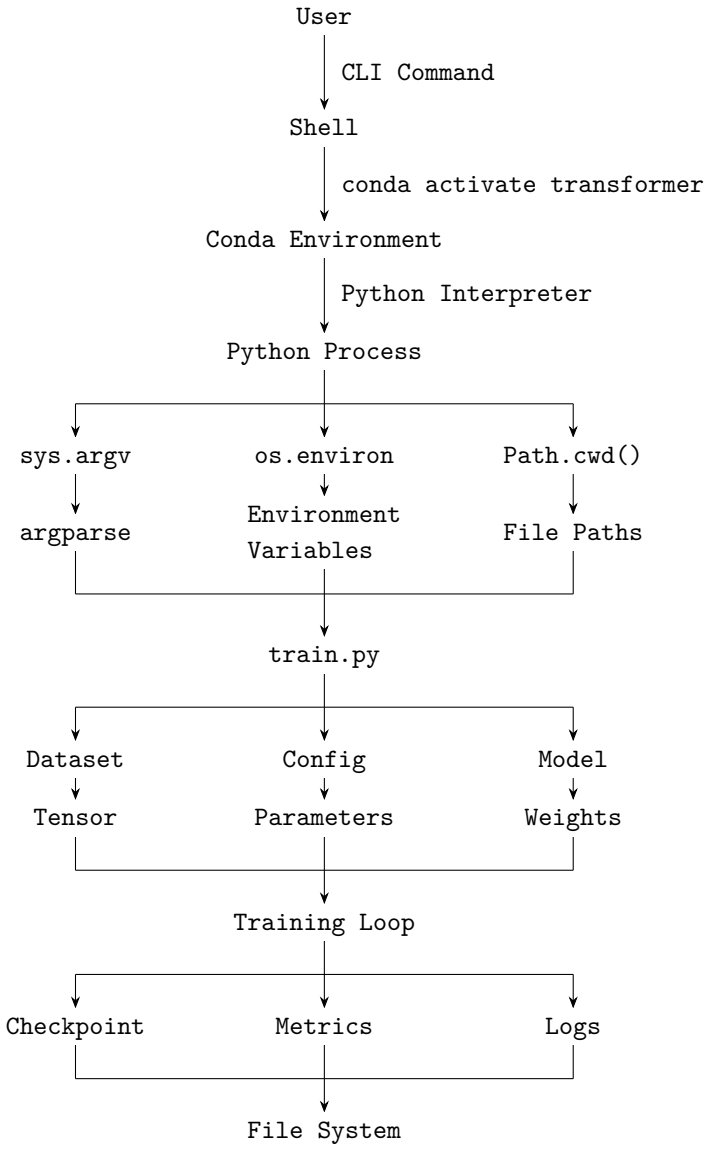
모델 저장소가 이후에 변경되더라도 동일한 리비전을 참조할 수 있다는 장점이 있다.

다만 정확한 재현성을 위해서는 토크나이저, 데이터셋, 라이브러리 환경도

함께 고정해야 한다.

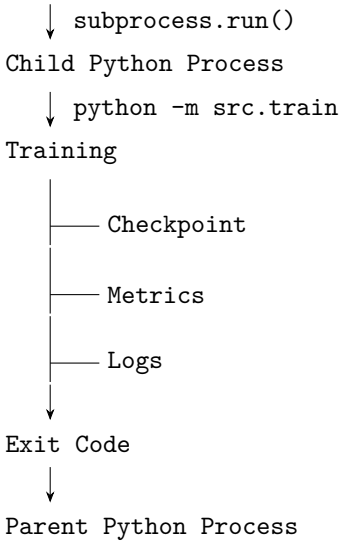
1.9.39 전체 프로젝트 실행 구조

이제 1 9장에서 배운 내용을 하나의 구조로 정리해 본다.



그리고 실험 자동화까지 포함하면 다음 구조가 추가된다.

```
run_experiments.py
```



이것이 지금까지 배운 Python 실행 환경의 전체 구조다.

1.9.40 전체 커리큘럼 최종 정리

장	학습 주제	핵심 개념
1장	Python 실행 환경과 외부 시스템 개요	sys, os, pathlib, 실행 환경과 외부 시스템의 관계
2장	파일 시스템과 경로	Path, CWD, 절대·상대 경로, pathlib
3장	경로의 기준점과 커맨드라인 실행	Path.cwd(), __file__, sys.path, Interpreter, Shell, sys.argv, argparse
4장	Python 모듈·패키지와 Import 시스템	module, package, sys.path, python -m, sys.modules, namespace, binding
5장	파일 입출력	Text, Binary, CSV, JSON
6장	URL과 네트워크	HTTP, requests, 다운로드
7장	환경변수	os.environ, PATH, PYTHONPATH
8장	프로세스	subprocess, stdin/stdout/stderr
9장	ML/DL 프로젝트 통합	학습 실행, 체크포인트, 캐시, 자동화

처음 이 커리큘럼을 구성한 이유는 Python의 기능을 하나씩 외우는 데 있지 않았다.

> Python 프로그램이 어떤 환경에서 시작되고,
>
> 외부 데이터를 어떻게 가져오며,
>
> 객체를 어떻게 구성하고,
>
> 다른 프로세스와 어떻게 상호작용하고,
>
> 최종 결과를 어떻게 저장하는지 하나의 실행흐름으로 이해하는 것

이 목표였다.

그 목표에 필요한 핵심 개념들을 이번 9장까지 모두 연결하였다.